StellarEdge AI

Auriga Technical Report: A Task-Agnostic Agent Harness for Zero-Shot Embodied Manipulation

StellarEdge AI Technical Team

Date Nov. 1, 2026

https://auriga.stelledge.com/

Abstract

Vision-language models can interpret instructions and plan manipulation tasks, yet struggle when acting directly as robot policies. On RoboDojo, a large *planning–execution gap* separates high-level task decisions from physical execution: removing arm execution raises GPT-6 Astra's success rate from 22.5% under the reference control interface to 76.2%. We present **Auriga**, a task-agnostic agent harness that closes 68.3% of this gap in zero-shot manipulation without target-task trials or resets. In Auriga, the model makes all task decisions over an explicit task belief and acts through either direct tool calls or synthesized multi-step Python programs, while the harness supplies calibrated 3D measurements, finger-referenced dual-arm control, and a single admission gateway to the robot. Before motion, the model inspects a visual trajectory preview while simulation is paused; during execution, the harness checks declared stage tolerances and grasp bounds, stopping the robot on divergence and returning the measured discrepancy for in-place recovery. Developed across 13 cross-task capability iterations and frozen for evaluation, Auriga ranks first on the RoboDojo leaderboard with a 57.7% success rate (64.0 Score), more than doubling the reference harness on the same model, and remains first under scene randomization while learned policies drop by up to 33.9%. On RoboCasa365, the same frozen harness reaches a 73.2% success rate, outperforming VLA-assisted agents by up to 23.1% on composite tasks.

Contents

1 Introduction . . . 3
2 Problem Setting . . . 5
3 Related Work . . . 6
3.1 LLMs and VLMs for Embodied Reasoning . . . 6
3.2 Agentic Harnesses for Robotics . . . 7
3.3 Vision-Language-Action and World Action Models . . . 7
4 General-Purpose Harness Evolution . . . 8
5 System Overview . . . 9
5.1 Manipulation as Closed-Loop Decision Making . . . 9
5.2 Separating Decision, Capability, and Admission . . . 11
5.3 Deliberation over an Explicit Task Belief . . . 12
5.4 Closing the Loop at Three Time Scales . . . 12
6 Execution Preview and Verification . . . 13
6.1 The Planning–Execution Gap . . . 14
6.2 Preview Before Acting . . . 14
6.3 Verify While Acting . . . 15
7 Perception and Action Interfaces . . . 16
7.1 Readable Physical State . . . 16
7.2 Expressive Actions . . . 18
8 Experimental Setup . . . 18
8.1 RoboDojo . . . 18
8.2 RoboCasa365 . . . 19
9 Main Results . . . 20
9.1 Results on RoboDojo . . . 20
9.2 Results on RoboCasa365 . . . 22
9.3 Episode Cost . . . 24
9.4 Tool Usage . . . 25
9.5 Ablation of the Mechanisms . . . 25
9.6 Cross-Task Harness Evolution . . . 28
10 Execution Trace Analysis . . . 28
10.1 Preserving Object Goals across Changes in Execution . . . 28
10.2 Using Feedback to Revise Feasible Operations . . . 29
10.3 Comparison with RoboProbe . . . 31
11 Case Studies . . . 32
11.1 Grasp Failure and Re-Grasp after Slippage . . . 32
11.2 Replanning after Contact and Execution Disruptions . . . 35
11.3 Geometry-Guided Manipulation with VGGT/SAM . . . 38
11.4 Failure after Repeated Corrections . . . 40
12 Conclusion . . . 42
Author List . . . 42
Appendix . . . 50
A Benchmark Tasks . . . 50
B Ablation Details . . . 50
C RoboCasa365 Results . . . 51
D Additional Results . . . 52
E Per-Task Results . . . 53

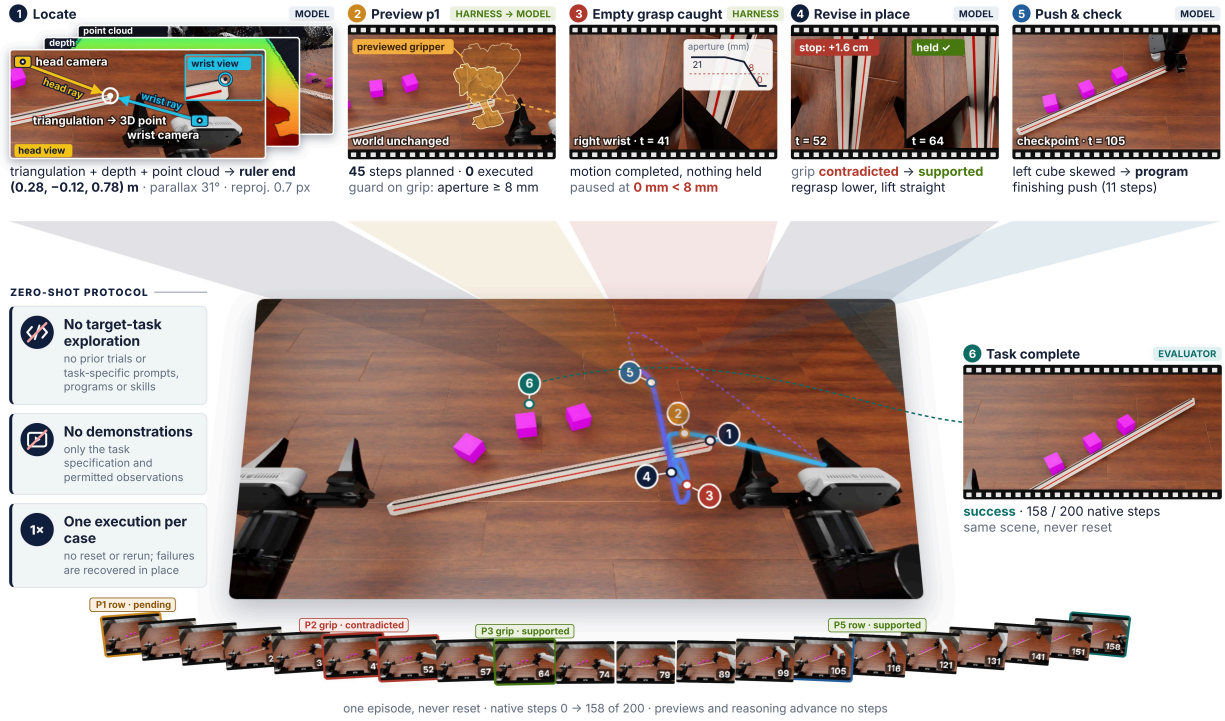


Figure 1 Auriga completes manipulation tasks zero-shot within a single episode. A general-purpose model plans and acts through a shared harness, with no target-task exploration, demonstrations, or resets. The model localizes targets by multi-view triangulation, complemented by depth and point-cloud estimates (1), and reviews candidate motions before they are executed (2). The harness verifies each execution and stops when an action fails (3); the model then revises its plan and recovers from the current state (4–5) until the task succeeds (6).

1 Introduction

In physical manipulation, a failed attempt changes the scene, often irreversibly. Unlike simulated settings where an agent can call `env.reset` repeatedly on a target task to tune coordinate offsets, control scripts, or failure notes before evaluation [11, 20, 31, 50], a deployed robot must complete an unseen task within a single episode. Tuning procedures across repeated trials on a fixed scene also fits to that scene's layout, and performance drops when object positions are randomized or the task changes. Generalization in manipulation therefore separates two problems: across tasks during development, using failures to improve the shared perception, control, and verification tools rather than task-specific scripts; and during execution on an unseen task, completing the task in one episode without prior trials, demonstrations, or resets (Sec. 2).

On a 210-episode diagnostic across all 42 tasks of the RoboDojo benchmark [4], the main bottleneck is physical execution rather than task reasoning (Sec. 6.1 and Tab. 3). When GPT-6 Astra is given a privileged action that moves objects along chosen waypoints without the robot arm, it succeeds in **76.2%** of episodes; when it drives the arm through the benchmark's reference end-effector interface (RoboProbe [61]), it succeeds in **22.5%**. The model usually identifies the right object and destination, but the arm misses a grasp by a few millimeters, loses the object in flight, or hits a container rim during transport. Common control interfaces do not catch these errors in time. Step-wise and chunked action interfaces [36, 61, 63] issue motions from the latest frames without showing the full route in advance, so collisions appear only after the arm has moved. Program generation [9, 21] hands execution to a script that cannot see the scene, where normal program exit does not mean the intended contact succeeded.

We present **Auriga**, a task-agnostic agent harness that narrows this planning–execution gap within a single

zero-shot episode (Figs. 1 and 2). In Auriga, the model makes all task decisions, while the harness measures the scene, checks and executes motions, and reports what happened:

1. **Task-agnostic harness and explicit task belief (Sec. 5).** Auriga contains no task-specific code, prompts, or heuristics, and exposes no privileged simulator state. One persistent model session plans the task, chooses actions, and recovers from errors. As an episode grows long, the model records its current plan, open questions, and task premises (`pending`, `supported`, `contradicted`, or `unknown`) in a concise *task belief* B_t, while full interaction logs stay in an off-context *evidence record* E_t retrieved on demand.
2. **Preview and verification through a single admission gateway (Sec. 6).** Every physical motion, whether called directly or from a model-written program, passes through the admission gateway $\mathcal{A}$ in a *preview–execute–verify* loop. While simulation is paused, $\mathcal{A}$ plans a multi-stage dual-arm sequence into a trajectory preview showing the path, step cost, and clearance warnings. During execution, the previewed trajectory runs while $\mathcal{A}$ checks model-set arrival tolerances $\tau_k = (\delta_p, \delta_\theta)$, gripper openings, and observation checkpoints. If a stage misses its tolerance, $\mathcal{A}$ stops the robot and returns a receipt r_t with the completed stages, measured deviations, unexecuted stages, and fresh camera views so the model can recover from the current state (Fig. 2).
3. **3D geometric measurements and finger-referenced actions (Sec. 7).** On the input side, camera calibration, VGGT-Omega [47] depth, and SAM 3.1 [2] segmentation provide 3D object estimates, ray–plane intersection with height sensitivity, and two-view triangulation, together with in-hand slip measurements and warnings when a motion re-enters a past failure region. On the output side, the model sets goals at the finger contact region rather than the wrist, chooses tolerances per stage, and coordinates both arms on a shared timeline.

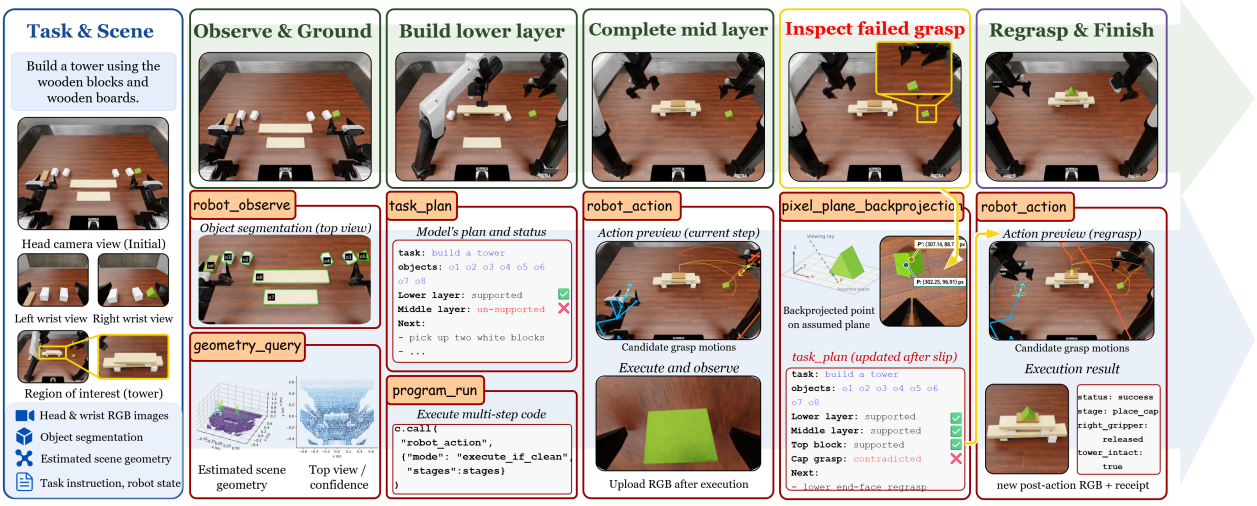


Figure 2 Planning, execution, and recovery within a single tower-building episode. From left to right, the model grounds the task in visual and geometric observations, constructs a plan, and builds the tower through direct tool calls and generated programs. Action previews and execution feedback support a preview–execute–verify loop. After a failed grasp, the model inspects the scene, updates its plan, and regrasps the object to complete the tower. The harness provides measurements and execution checks; the model selects actions and recovery strategies. The scene is never reset.

Auriga separates cross-task tool development from single-episode execution (Secs. 4 and 9.6). Over 13 development iterations (Fig. 3), manipulation failures were used to identify missing capabilities in 3D localization, contact checking, and state tracking, raising zero-shot performance across all 42 RoboDojo tasks from 36.2% SR (45.5 Score) at iteration `#4` to 57.7% SR (64.0 Score) in the final harness. During

evaluation, the harness is frozen and runs under the single-episode zero-shot protocol (Secs. 2, 8 and 9), with no target-task exploration, demonstrations, or resets:

- **RoboDojo [4].** Across all 42 tasks and five dimensions (6,300 episodes over three seeds), Auriga with GPT-6 Astra reaches an average **Score of 64.0** and **SR of 57.7%**, ranking **first overall on the leaderboard** across learned policies and agent methods, more than doubling the GPT-6 Astra reference harness (**29.0 Score**, **22.5% SR**), and leading all 12 columns of the agent block (Tab. 4). On the 210-episode diagnostic set, Auriga closes **68.3%** of the gap between reference arm control and arm-free object motion (Tab. 3).
- **Scene randomization.** Across the 12 Generalization tasks (Fig. 9 and Tab. 22), learned policies drop by up to 33.9% in SR on randomized layouts, whereas **Auriga ranks first on both standard scenes (52.3% SR) and random scenes (45.6% SR)** and achieves the highest generalization score ($G = 46.3$).
- **RoboCasa365 [32].** On the 50 Target50 mobile manipulation tasks in unseen kitchens (250 episodes), Auriga reaches **73.2% Overall SR** (**92.2%** Atomic-Seen, **67.5%** Composite-Seen, **57.5%** Composite-Unseen) without a learned policy or cross-episode memory, outperforming VLA-assisted agents RPent [62] (**59.2%**) and PyRUA-Lean [44] (**56.4%**) by **19.4%** and **23.1%** on composite tasks (Tab. 5).

2 Problem Setting

We study zero-shot manipulation, in which a general-purpose model uses a shared robot harness to construct and execute a task procedure from the task specification and interaction feedback.

Task and interaction. The unit of evaluation is a *case*, defined by a task instruction and a specified scene variant, random seed, and layout. Each case starts from an initial scene and is evaluated under the benchmark's execution budget and success criteria. The agent receives the task specification and accesses observations and robot actions through the shared harness. It uses the available observations and interaction history to choose subsequent tool calls or programs, then receives feedback from their execution. This interaction continues until success, termination, or budget exhaustion. The benchmark's evaluator determines task success from the resulting execution.

Zero-shot protocol. We define zero-shot evaluation through three conditions:

1. **No target-task exploration or development before evaluation.** Evaluation starts with a pretrained model, a fixed shared harness, and task-independent operating guidance. No preparatory exploration or trials are conducted on the target task, including its other layouts, to develop or select task-specific prompts, programs, skills, or policies. Model parameters remain fixed throughout evaluation.
2. **No demonstrations at evaluation time.** The agent receives the task specification and permitted observations, without additional solution examples or expert demonstrations. For memory tasks, the benchmark-defined event sequence forms part of the case's observation input.
3. **One execution per case.** Each case is initialized and executed once, without a reset or rerun. Its outcome is recorded whether the episode succeeds or fails.

Within-episode adaptation. An episode comprises successive model decisions, observations, and physical actions. During this interaction, the agent may inspect the scene, write and revise programs, preview motions, revisit its history, and recover from a failed grasp or placement. Recovery continues from the physical state left by earlier actions under the remaining execution budget, rather than resetting the scene to start over.

Plans, generated code, and interaction memory evolve within the current episode and are not transferred to subsequent evaluation cases.

Tab. 1 compares six aspects of target-task preparation and information available to the test agent across representative manipulation settings.

Comparison scope. Tab. 1 distinguishes target-task environment trials from offline demonstration analysis. Prepared code/skills include procedures later generalized across tasks; trial-derived memory denotes separate notes inherited from earlier trials. Demo-derived artifacts are distilled text or code, whereas demo references are recorded images or motions, including those retrieved from frozen packages. Task-independent tools, public task specifications, current-episode history, benchmark-required memory events, and training data encoded in model weights are outside these categories.

A fixed evaluation agent does not establish the absence of earlier task-driven tool development. PyRUA-Lean uses shared primitives without cross-episode memory, but does not report whether target-task trials informed their development; its two preparation entries are therefore marked as not reported [44]. HexaShell is shown for non-Open tasks; its Open setting omits task-specific development and notes [11]. The R^{2}SI row describes the inspected public release: outer-loop trials exclude Open tasks, and the snapshot supplies no separate memory files or demo context, although the framework supports memory updates and its WAMs use trained weights [29].

Table 1 Target-task preparation and evaluation inputs across the specified settings.

	Before evaluation		Available to the test agent			
Method / setting	Task exploration	Cross-trial refinement	Task code / skills	Trial-derived memory	Demo-derived artifacts	Demo references
GPT-6 Astra (zero-shot) [61]	✗	✗	✗	✗	✗	✗
GPT-6 Astra (image-demo ICL) [61]	✗	✗	✗	✗	✗	✓
GPT-Policy (video ICL) [5]	✗	✗	✗	✗	✗	✓
GPT-GEAR (full-context ICL) [65]	✗	✗	✗	✗	✓	✓
SenseNova (RoboDojo ICL) [43]	✗	✗	✓	✗	✓	✓
PyRUA-Lean (RoboDojo) [44]	–	–	✗	✗	✗	✗
SEAR (hardware AaP) [50]	✓	✓	✓	✓	✗	✗
HexaShell (RoboDojo) [11]	✓	✓	✓	✓	✗	✗
EmbodiedEvo (frozen harness) [20]	✓	✓	✓	✓	✗	✗
WorldScape R^{2}SI (release) [29]	✓	✓	✓	✗	✗	✗
Auriga (ours)	✗	✗	✗	✗	✗	✗

Notes. ✓: included; ✗: absent from the specified setting; –: not reported. Scope and qualifications are discussed in the text.

3 Related Work

3.1 LLMs and VLMs for Embodied Reasoning

Robotic applications of language and vision-language models span task planning, spatial grounding, and direct action generation. Alongside general-purpose models such as GPT-6 [34] and Claude Opus 5.5 [1], embodied foundation models specialize perception and reasoning for physical interaction [6, 12, 19, 46, 59]. Evaluations of GPT-6 Astra [10, 61] investigate direct numerical control and cooperation with learned policies, revealing a gap between semantic task understanding and precise physical execution. Other work adapts robot behavior through context: GPT-Policy [5] preserves task-relevant demonstration transitions, while

RoboICL [24] organizes demonstrations and interaction memory around observations, actions, and execution receipts.

3.2 Agentic Harnesses for Robotics

Agentic harnesses organize model interaction with robot tools, programs, and execution feedback. *Trial-and-error interaction* operates within a task: Agent as Policy [16] and CaP-Agent0 [9] within CaP-X allow agents to construct programs and revise execution using observations and tool results, while RoboHarness [38] emphasizes a visual-geometric control interface. The agent may also delegate action generation: PhysicalRSI [31] provides task-aware memory and skills backed by code policies or $\pi_{0.5}$ [36], and FastHarness [22] adapts the frequency of VLM supervision over a frozen vision-language-action (VLA) model.

Cross-trial evolution uses rollout feedback to revise programs, tools, or policies for subsequent executions. HarnessPAI [49] keeps the generated program fixed within a rollout, including its observation-dependent checks, and revises it across rollouts. HexaShell [11] separates an acting agent from an optimizer that repairs and tests executable tools; EmbodiedEvo [20] separates exploration from evaluation, passing distilled code skills and failure lessons to independent test agents. Both freeze the acquired experience for evaluation without updating model weights. Related approaches evolve skills and context code, attribute execution failures, or select informative physical trials [7, 45, 48]; rollout scale and candidate selection also affect harness improvement [52]. WorldScape-R^{2}SI [29] jointly improves reusable skills and a reactive action model, while ENPIRE [54] lets agents revise policy-training algorithms and infrastructure using real-robot evaluations.

Skill libraries complement cross-trial improvement by storing reusable procedures for later retrieval and composition. ASPIRE [26] stores validated repair knowledge as reusable in-context guidance, including applicability conditions, repair strategies, and optional code sketches; RATs [60] stores code skills acquired through self-directed play and retrieves them from a frozen library for downstream tasks. Teach-and-Grow [33] derives closed-loop Skill Blocks from subgoals shared across demonstrations and retains execution conditions and repairs in a separate experience memory. Real2Gym [39] retains task procedures, object-relative motions, and recovery strategies from interaction in simulation environments reconstructed from videos, then reuses them on physical robots. GaP [3] focuses on composition, assembling existing perception, planning, and control modules into computation graphs refined through simulation. Auriga organizes interaction within a persistent model session, where generated programs and direct tool calls share a physical execution interface and return observations and outcomes to guide subsequent decisions.

3.3 Vision-Language-Action and World Action Models

Learned action models map visual and language inputs to robot actions and can serve as policies invoked by an agent. Recent VLA work explores heterogeneous-data co-training and cross-embodiment adaptation [36, 37, 53, 55, 57, 63], experience-driven policy improvement [35], and joint reasoning and action generation [25]. InternVLA-A1.5 [28] further introduces latent foresight supervised by a pretrained video model while retaining semantic training objectives.

World action models (WAMs) incorporate predictive modeling or supervision from future observations into action learning. Video-based approaches jointly model actions and future observations [18, 51, 56]; VPP2 [14] strengthens manipulation-specific video prediction, and X-WAM [13] adds multi-view RGB-D prediction with asynchronous action decoding. Future supervision can also shape action representations without requiring future-video generation at deployment, as in Being-H0.7, Fast-WAM, and InternW0-Δ [27, 30, 58]. JEPA-WAM [23] couples latent transition prediction to action generation, while Long-WAM [15] studies causal pretraining and longer observation histories. When these policies are used within a harness, the agent coordinates their invocation, evaluates execution outcomes, and revises the task procedure.

4 General-Purpose Harness Evolution

Improvement in embodied agents can target either a procedure for a particular task or the shared capabilities used across many tasks [8, 64].

Within-task trial search versus cross-task tool evolution. Recent work on recursive self-improvement (RSI) groups autonomous improvement into six levels from L0 to L5 [8]:

- **L0–L2 (within-task search and refinement):** Under a single-task objective, the system repeats trials to adjust a trajectory, script, or task note, so the resulting gains stay mainly on the practiced task.
- **L3–L4 (cross-task experience and tool evolution):** The system turns recurring failures across different tasks into reusable tools, interfaces, and verification steps that apply to unseen tasks.
- **L5 (meta-improvement):** The system revises its own perception, control, and verification mechanisms so that later tasks are solved more reliably.

Target-task refinement methods such as HexaShell, EmbodiedEvo, and SEAR [11, 20, 50] work in the L0–L2 regime: they reset the target scene across trials to tune coordinate offsets, scripts, or failure notes for that task, which improves performance on the practiced layout but can overfit when layouts change. Auriga instead focuses on L3–L5 evolution during development. Failures such as a jammed pin insertion or a slipped cloth grasp are used not to add a coordinate offset for that scene, but to diagnose missing capabilities in 3D localization, contact checking, or in-hand state tracking and to upgrade the shared tools accordingly.

Role of the harness. An agent harness connects the model to the robot: it translates the model's intent into motion and stops the robot when contact or slippage occurs, without making task decisions on the model's behalf. Existing harnesses often either run generated commands open-loop without checking the physical outcome, or add task-specific rules and thresholds in the backend to pass particular tasks. Because manipulation involves both tight rigid clearances and deformable objects, an offset or grasp rule tuned for one task often interferes with another.

Auriga keeps the harness task-agnostic through three rules: (1) *no task-specific logic or privileged state*, so the same code and prompts run across all tasks and randomized layouts; (2) *general measurement and control tools*, including calibrated multi-view triangulation, paused-world trajectory preview, stage tolerances, and in-hand slip checks; and (3) *in-place recovery*, stopping when a motion leaves its tolerance and returning the measured deviation so the model can continue from the current state.

Two complementary timescales. Fig. 3 highlights the key stages in strengthening Auriga's general-purpose harness. The early stages establish a calibrated, previewable action interface and organize task information through harness refactoring and official task descriptions. Geometric queries, action-history retrieval, and observation checkpoints then extend the model's ability to measure the scene, inspect intermediate outcomes, and revise its next action, while held-object perception adds explicit feedback about the object's relation to the gripper. System development and within-episode adaptation thus operate at complementary timescales. Across development versions, the shared tools and their interfaces are upgraded from cross-task failure evidence (Sec. 9.6); during evaluation, the harness version is completely frozen across all cases. Within each evaluated episode, the model uses those shared tools to inspect the scene, diagnose discrepancies, and revise its plan or generated programs from execution feedback, continuing from the physical state left by previous actions under the remaining budget without an environment reset or cross-episode memory transfer (Secs. 2 and 5 to 7).

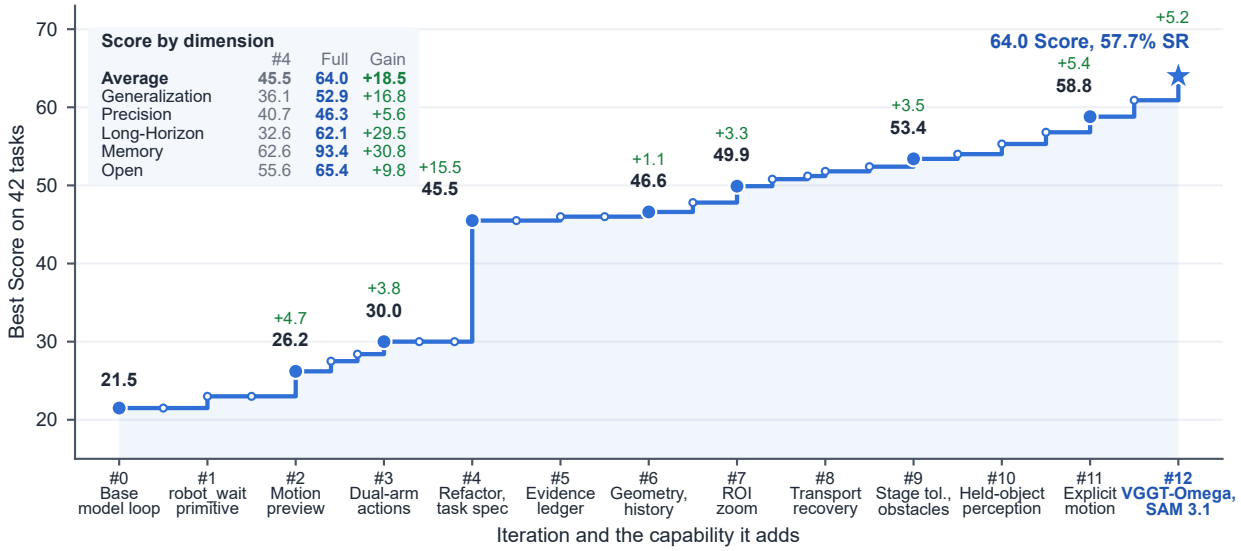


Figure 3 General-purpose harness evolution of Auriga across 13 capability iterations (`#0` to `#12`). The horizontal axis lists the newly added task-agnostic tools and harness capabilities introduced at each iteration, and the vertical axis tracks the resulting performance (best overall Score on the 42 RoboDojo tasks). Larger nodes, labeled with their Score and its gain, mark major capability milestones, including motion preview (`#2`), unified dual-arm actions (`#3`), harness refactoring with verbatim task specifications (`#4`), calibrated 3D geometry and history (`#6`), ROI zoom (`#7`), stage tolerances and obstacle checks (`#9`), and explicit motion control (`#11`), culminating in the frozen Auriga harness with VGGT-Omega and SAM 3.1 perception (`#12`, star marker: 64.0 Score, 57.7% SR across 6,300 episodes). Top-left inset: 42-task Score by capability dimension from iteration `#4` to the full frozen harness.

5 System Overview

Large vision-language models can reason about task structure, but reliable manipulation also requires metric scene information, contact-sensitive execution, and feedback after each action. Auriga addresses this gap with a simple closed-loop workflow: the model measures and previews while the simulation is paused, submits a physical action through a single admission gateway, and then uses the returned receipt and observation to decide what to do next (Fig. 4). The model retains authority over planning and recovery; the harness supplies measurements, enforces the execution boundary, and records what occurred. This section first formalizes the decision process (Sec. 5.1), then describes the three system layers (Sec. 5.2), the explicit task belief maintained by the model (Sec. 5.3), and the three feedback loops that connect planning to execution (Sec. 5.4). Sec. 6 details execution preview and verification through the admission gateway, and Sec. 7 specifies the perception and action interfaces.

5.1 Manipulation as Closed-Loop Decision Making

For each benchmark episode, the model must construct a procedure from the task description and the evidence gathered during that episode; no task-specific solution is prepared in advance. Consequently, each decision uses the information available so far, and each outcome must be returned in a form that can inform the next decision.

Formulation. A case specifies a task description g and an initial world state s_0 (Sec. 2). The world state s_t is hidden from the model, which instead receives observations o_t comprising calibrated multi-view RGB images

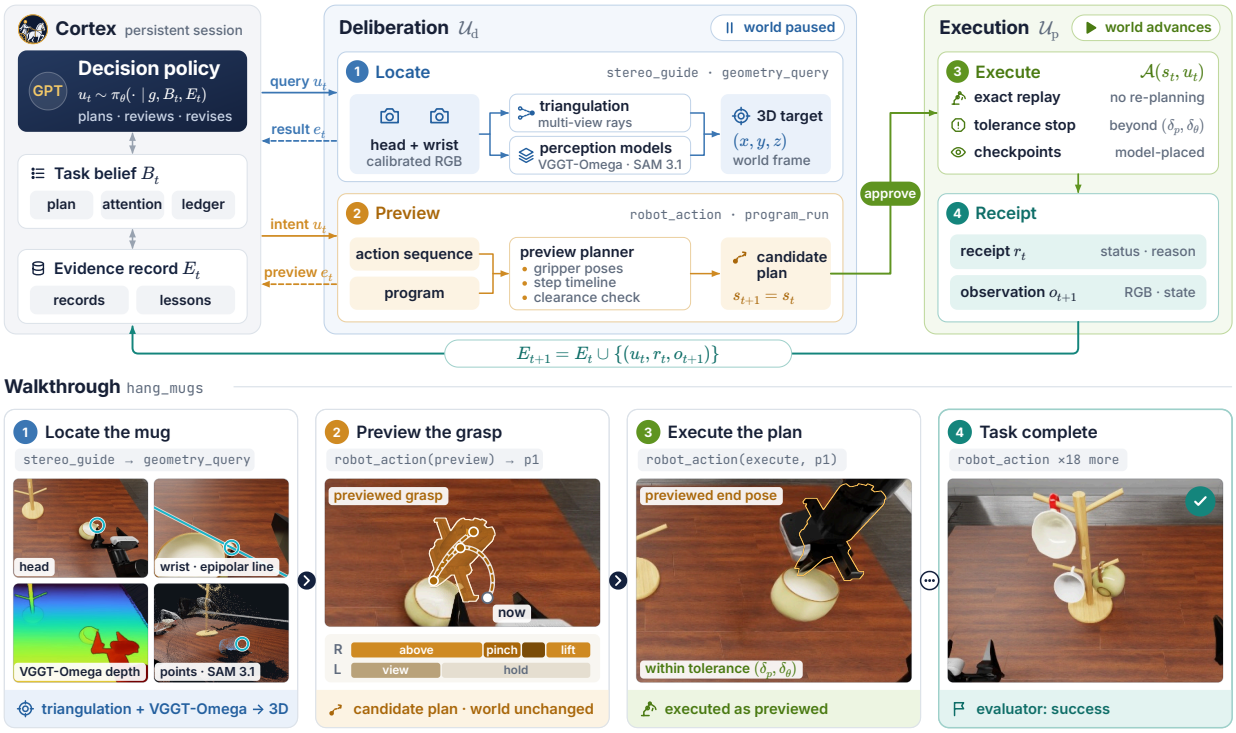


Figure 4 Overview of Auriga. *Top:* The model maintains a task belief and an evidence record, uses observations to localize targets, and previews actions while the world is paused. Every physical motion passes through the admission gateway $\mathcal{A}$, which returns a receipt and a new observation. *Bottom:* Closed-loop execution on `hang_mugs`: (1) localization, (2) grasp preview, (3) execution, and (4) placement on the rack.

and the measured robot state. At decision step t, a pretrained model with fixed parameters θ selects a tool call

$$u_t \sim \pi_\theta(\cdot \mid g, B_t, E_t), \tag{1}$$

where B_t is its *task belief* (Sec. 5.3) and E_t is the episode's *evidence record*, the interactions accumulated so far. The available calls have two roles. *Deliberative* calls $u_t \in \mathcal{U}_\mathrm{d}$ inspect the scene, measure quantities, compute, retrieve records, revise the belief, or preview a motion. They return a result e_t while leaving the world unchanged:

$$s_{t+1} = s_t, \qquad E_{t+1} = E_t \cup \{(u_t, e_t)\}. \tag{2}$$

For example, before hanging a mug, the model can localize the mug and rack, inspect the handle orientation, and preview a grasp without advancing simulated time. *Physical* calls $u_t \in \mathcal{U}_\mathrm{p}$ request robot motion or the passage of simulated time. They reach the world only through the admission gateway, written as the operator $\mathcal{A}$, which executes an admissible request and returns a receipt r_t and a fresh observation:

$$(s_{t+1}, r_t, o_{t+1}) = \mathcal{A}(s_t, u_t), \qquad E_{t+1} = E_t \cup \{(u_t, r_t, o_{t+1})\}. \tag{3}$$

If $\mathcal{A}$ rejects a request, the world remains unchanged and the receipt reports the reason. The model parameters θ, tool set, and task-independent operating guidance remain fixed throughout an episode; adaptation comes from the evolving B_t and E_t. Because the simulation is paused during deliberation, reasoning and deliberative calls consume no simulated time.

Quantitative error feedback. A physical request specifies both an intended result and the required precision. Each stage k has a target $\hat{x}_k$ and a model-chosen tolerance τ_k (Sec. 7.2). During execution, $\mathcal{A}$ measures

the reached state x_k and halts when the deviation $\epsilon_k = x_k - \hat{x}_k$ leaves τ_k. The resulting receipt reports the completed stages, measured deviations, active tolerance, and unexecuted stages (Sec. 6.3). Thus, if a grasp approach in `hang_mugs` stops short or the gripper opening differs from the expected value, the model receives the measured discrepancy rather than a generic failure flag. It can then decide whether to inspect the scene, revise the approach, or execute the remaining stages; $\mathcal{A}$ does not choose the recovery.

Information boundary. The harness exposes only the published task text, calibrated RGB images, the robot's measured state, and the public robot model and camera calibration. From these inputs, it may compute learned depth, per-object 3D estimates, and point clouds, reporting each as an estimate with its reliability (Sec. 7.1). The model and its tools cannot access simulator depth, hidden object poses, contact state, or evaluator internals. The benchmark evaluator, rather than the model's own assessment, determines task success.

5.2 Separating Decision, Capability, and Admission

Auriga separates task reasoning from measurement and execution so that a general-purpose model can build a procedure while every physical action remains reviewable.

Division of responsibility. The harness is organized into three layers (Fig. 4): a model session that makes every task decision, a capability layer of general tools, and an admission gateway that is the only path to the robot. Each layer has a distinct responsibility:

- *Cortex* is one persistent model session for the entire episode. It receives task-independent operating guidance, the official task text, and the robot's capabilities and coordinate conventions. It chooses task decompositions, measurements, action targets, and recovery steps without human intervention or a task-specific prompt, program, or skill.
- The *capability layer* provides general tools to measure, act, compute, and recall (Tab. 2). For `hang_mugs`, these tools can estimate mug and rack locations, compare views, preview a grasp, or retrieve an earlier receipt; they do not encode a solution for hanging mugs.
- The *admission gateway* implements $\mathcal{A}$. It is the only path that moves the robot. It receives a reviewed physical request, determines whether the request is admissible, monitors execution, and records the outcome.

Table 2 Capabilities available to the model. Only physical calls advance the world, and they do so only through the admission gateway $\mathcal{A}$.

Capability	Role	Representative tools	World	Details
Measure	Return images, robot state, object estimates, and geometric or state quantities with their reliability	`robot_observe`, `geometry_query`, `stereo_guide`	–	Sec. 7.1
Act	Preview and execute multi-stage action sequences for one or both arms, or let simulated time pass	`robot_action`, `robot_wait`	$\mathcal{A}$	Secs. 6.2, 6.3 and 7.2
Compute	Run programs for loops, search, and numerical work; their physical requests are reviewed by the model	`program_run`	$\mathcal{A}$	Sec. 6.2
Recall	Revise the task belief, retrieve earlier interactions, and record or query operation lessons	`task_plan`, `history_*`, `operation_memory`	–	Secs. 5.3 and 7.1

One-step dataflow. At each decision step, the model first gathers the evidence needed for the next choice

through the capability layer (Sec. 7.1). It then submits either a complete multi-stage action sequence or a program that requests one (Sec. 7.2). Both routes converge on $\mathcal{A}$, so a program-generated motion is previewed and reviewed in the same way as a direct request (Sec. 6.2). After a physical call, the harness stores the reviewed request, receipt, observations before and after execution, and measured displacement in one interaction record (Sec. 7.1). These records make completed motion, normal program termination, and benchmark success separately observable instead of conflating them.

Transparent execution. The model sets task decomposition, targets, and the need for additional evidence; the harness exposes the thresholds that admit, pause, or stop an action and reports their effects. Every physical request, whether direct or program-generated, follows the same admission path. The harness neither substitutes a task-specific rule nor silently changes a parameter supplied by the model. The records constitute E_t, remain outside the model context, and can be retrieved on demand (Sec. 7.1). The model may attach short operation lessons to them, but it rechecks each lesson against the current scene before using it.

5.3 Deliberation over an Explicit Task Belief

Long-horizon manipulation accumulates observations, receipts, and intermediate decisions. Without a compact representation of the current task understanding, a model can lose track of an earlier objective or confuse an issued command with a verified outcome. Auriga therefore maintains a revisable task belief B_t alongside the detailed evidence record E_t.

Structure. The belief has three concise components. The *plan* records the model's current understanding of the goal, its intended approach, and the visible evidence that would indicate completion. The *attention* list records the uncertainties that matter now, the evidence that could resolve each one, and the part of the procedure that depends on it. The *ledger* records a small number of task-critical premises. Each premise includes an expected event, a status (*pending*, *supported*, *contradicted*, or *unknown*), the permitted evidence for that status, and the next observation or action that could change it (the labeled premises in Fig. 1). The harness admits no physical action until the model has submitted a belief.

For example, in `hang_mugs`, the plan can state "grasp the mug by its handle, lift it, and place it on the rack hook." An attention item can require a view that resolves the handle orientation before the approach. A ledger premise can state that the mug is retained by the gripper: it is initially *pending*, and becomes *supported* only after permitted evidence, such as the expected gripper opening together with a confirming lift observation, is available.

Belief versus evidence. B_t is a compact, revisable working state rather than an action script or a source of evidence. Concrete poses, arm timing, and gripper commands remain part of individual actions, so the model can change them as new measurements arrive. Likewise, a belief entry must be checked against fresh observations before it supports a decision: a closed gripper alone does not verify that a mug is retained. When the conversation is compacted to fit the context window, the harness reinserts the latest B_t as the model's provisional judgment, while the details of earlier interactions remain retrievable from E_t.

5.4 Closing the Loop at Three Time Scales

Errors can arise within a movement, between successive actions, or only after a task-level premise has been tested. Auriga addresses these cases with three nested loops that have distinct responsibilities: the admission gateway stops unsafe or off-target execution, while the model diagnoses the result and chooses the recovery. Fig. 5 shows how the three loops interleave within one episode.

Stage loop. Within a single action, $\mathcal{A}$ contains errors: applying the stage check of Sec. 5.1 during execution, it halts at the first stage whose deviation ϵ_k leaves τ_k, and also pauses at a failed grasp check or at a checkpoint placed by the model (Sec. 6.3). For a `hang_mugs` grasp, it can stop the sequence immediately when the

measured gripper opening indicates that the mug handle was not secured. The stage loop keeps an error from propagating to later stages; it does not repair the motion.

Action loop. Between actions, the model diagnoses outcomes: after each physical call, it compares the receipt and fresh observation with its intended result and forms a cause hypothesis before acting again. It may continue the remaining previewed stages, replan from the stopped stage without repeating completed stages, change the contact, route, target, arm, or tolerance, or collect an observation that distinguishes competing explanations. For example, after an unsuccessful mug grasp, the model can inspect whether the handle orientation, approach route, or target localization caused the discrepancy before choosing a new approach. Whenever possible, normal task progress tests the hypothesis; a separate probe is used only when its result would alter the next decision.

Belief loop. Across the episode, the model revises its premises: action outcomes update the ledger and, when necessary, the plan in B_t. If repeated evidence contradicts a premise, the model changes its task-level explanation and selects a different mechanism, such as another contact, route, or target, rather than simply restating the same hypothesis. In `hang_mugs`, evidence that a top-down approach repeatedly fails can update the belief so that the model seeks a handle-oriented grasp or a different approach route. This loop keeps the model's working understanding aligned with the evidence accumulated during the episode. Sec. 6 next describes the admission path in which the stage loop runs.

Input: task description g, initial observation o_0, task-independent operating guidance, fixed model π_θ.
Initialize: $E_0 \leftarrow \{o_0\}$; the model observes, measures, or previews and submits a belief B_0 before any physical call.
repeat *belief loop*

1. *Decide:* the model selects $u_t \sim \pi_\theta(\cdot \mid g, B_t, E_t)$ (equation (1)).
2. *Deliberate:* if $u_t \in \mathcal{U}_{\mathrm{d}}$, return an observation, measurement, computation, retrieved record, or preview e_t; the world is unchanged (equation (2)).
3. *Act:* if $u_t \in \mathcal{U}_{\mathrm{p}}$, issued directly or by a program, apply $\mathcal{A}$ (equation (3)). *stage loop*
   For each stage k: execute, measure x_k, and halt if ϵ_k leaves τ_k, if the grasp check fails, or at a model checkpoint. Return r_t and o_{t+1}, and append the record to E_{t+1}.
4. *Assess:* the model compares r_t and o_{t+1} with its intent and states a cause hypothesis for any deviation. *action loop*
   It continues, replans from the stopped stage, changes the contact, route, target, arm, or tolerance, or gathers a distinguishing observation.
5. *Revise:* the model updates the status of affected premises and, if its understanding has changed, the plan in B_{t+1}.

until the evaluator reports termination or the execution budget is exhausted.

Figure 5 The three nested loops of Auriga within one episode. The stage loop runs inside $\mathcal{A}$ and stops execution when a declared condition is violated; the model closes the action and belief loops. Deliberative calls extend the evidence record without advancing the world.

6 Execution Preview and Verification

A general-purpose model can usually tell what a manipulation task requires, yet it often cannot make the robot carry it out. We first quantify this *planning–execution gap* (Sec. 6.1) and then describe how Auriga narrows it by binding execution to a preview the model can inspect (Secs. 6.2 and 6.3). Auriga follows a single rule, enforced by the admission operator $\mathcal{A}$. The motion that reaches the robot is exactly the motion

compiled into its preview, and whenever execution departs from it, the harness stops and reports the measured state rather than repairing it.

6.1 The Planning–Execution Gap

Deciding without the arm. To separate what the model decides from how the arm executes it, we run the same harness with one additional privileged action, which moves an object along waypoints given by the model without using the arm. The model still chooses the object and every target pose from its own perception. The object moves under simulated physics, stops at a collision, and settles under gravity at the end, after which the action reports where the object came to rest. Success under this action therefore estimates what the model's own decisions achieve when execution by the arm is taken out of the loop. We compare it with two executed systems on the same 210 episodes, five for each of the 42 tasks (Tab. 3). The first is RoboProbe [61], the reference harness released with the evaluation of GPT-6 Astra on RoboDojo [4], and the second is Auriga. Averaged over the five dimensions, the success rate is 76.2% with the privileged action, 22.5% for RoboProbe, and 59.2% for Auriga. The model therefore decides considerably better than it executes.

Table 3 Planning–execution gap on 210 episodes, five for each of the 42 tasks: SR (%) and Score (0–100) per dimension and their average over the five dimensions, as in Tab. 4. With direct object motion, the model moves objects through a privileged action instead of the arm. All methods use GPT-6 Astra, RoboProbe at medium and the others at high reasoning effort. [§]Memory over 29 episodes, since one episode has no counterpart in the reference results.

	Average		Generalization		Precision		Long-Horizon		Memory		Open	
Method	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score
Direct object motion (privileged)	76.2	79.6	81.7	83.3	65.0	72.0	70.0	75.4	96.7	96.7	67.5	70.6
RoboProbe [61]	22.5	28.5	31.7	34.1	2.5	8.5	7.5	22.1	40.8[§]	45.8[§]	30.0	32.1
Auriga	59.2	64.6	43.3	46.2	50.0	55.6	45.0	59.5	90.0	90.5	67.5	71.2

Why common paradigms leave the gap open. Two common ways of placing a general model on a robot address execution only indirectly. In *step-wise action output*, as in the reference harness and in VLA policies [36, 61, 63], the model emits one motion step or chunk per call from the latest frames. It never sees the route of a motion before committing to it, and a failed motion is noticed only when the model reads the next frame, by which time the arm has already moved. In *code as policy* [9, 21], the model writes a program that commands the control interface directly. Execution judgment then moves into a program that cannot see the scene. A failure is detected only if the program was written to check for it, and a program may exit normally even after the motions it requested have failed. Auriga keeps the judgment with the model, which describes a complete action sequence in each call rather than a single step. Before execution, the model can see the whole trajectory, its cost, and its projection on the camera image. Exactly the previewed motion then reaches the robot (Sec. 6.2), and the harness checks the measured outcome after every movement (Sec. 6.3).

6.2 Preview Before Acting

Preview gives the model a first guarantee, namely that every motion, whether requested by the model or by a program it has written, is compiled into a preview before it runs, and that nothing else is executed. The model reviews this preview by default and can skip the review to save a call. Below, we describe how the model sees a motion and how execution is held to it.

Seeing the whole motion. Rather than issuing one movement per call, the model describes a complete action sequence for one arm or for both arms at once, such as approaching an object, closing the gripper, lifting, transporting, and releasing it. The harness plans the entire sequence into a robot trajectory before anything

moves and returns a preview of what would happen. Any adjustment the harness makes to how a movement is realized happens at this stage, so the preview already shows the motion that will run. The preview draws the planned gripper paths (Fig. 6) and arm configurations on the camera image and reports how long each part of the sequence would take on each arm. It also estimates the clearance between the robot, the objects it holds, and the obstacles the model has declared, such as whether the two arms come close to each other. Such estimates are given as advice, and the decision whether to proceed stays with the model. If any part of the sequence cannot be planned, nothing is executed, and the preview states which part failed and why.

Committing to it. Once the model accepts a preview, the harness replays the previewed trajectory without replanning it. Because the simulation is paused while the model inspects the preview, the robot is still in the state from which the preview was planned, and looking ahead costs no execution time. The robot therefore performs the previewed motion, so every later report refers to a motion the model can inspect. In free space the robot follows the previewed trajectory closely, which means that a deviation during execution almost always indicates contact, the case that Sec. 6.3 is built around.

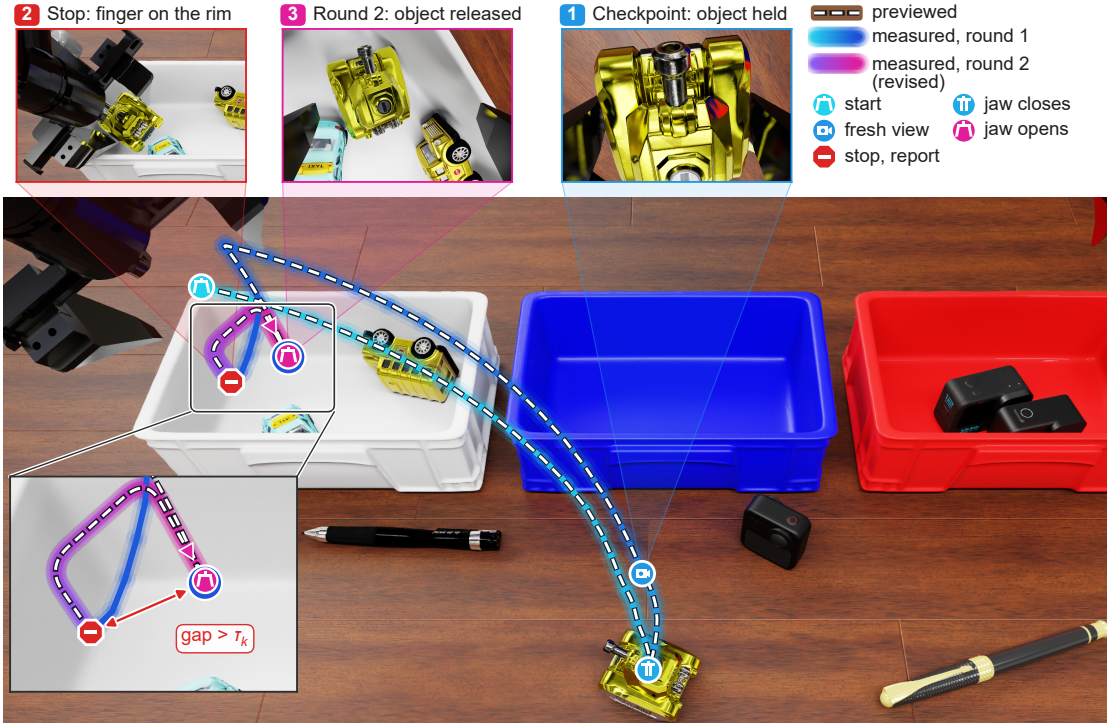


Figure 6 Preview and verification in two rounds, drawn on the head-camera image. In the first round, the previewed gripper path moves to the object, closes on it, and lifts it to a checkpoint placed by the model, where the robot pauses and the model receives a fresh wrist view (inset 1). The measured path follows the preview until a finger lands on the basket rim while the object is lowered. Once the gap exceeds the tolerance, the harness stops the arm and reports with a fresh view (inset 2), so the gripper does not open. In the second round, the model previews a revised motion from the stop that lifts clear of the rim and lowers into the basket, where the gripper opens (inset 3). The rim contact and the second round are illustrative.

6.3 Verify While Acting

Preview fixes what will be executed, but not how the world will respond. Contact, slip, and objects displaced by earlier actions can still make the outcome differ from the plan. Verification gives the second guarantee, namely that every divergence of the robot from its previewed motion beyond the tolerance is caught at the end of the movement in which it occurs and handed back to the model together with the facts needed to act on it.

Catching divergence. After each movement, the harness compares where the gripper actually is with where the preview placed it. The tolerance τ_k of a movement (Sec. 5.1) consists of a position part δ_p and an orientation part δ_θ. The model can set both for each movement, by default δ_p=10 mm and δ_θ=0.15 rad. If the difference exceeds the tolerance set for that movement, the robot stops and holds its pose instead of continuing with the rest of the sequence. The receipt r_t that follows states which movement failed, by how much it missed, and which parts did not run. It also adds a fresh camera view from the stopping point and images of the measured and the previewed route. Movements that already completed are kept rather than undone. In Fig. 6, a finger lands on a basket rim while the object is lowered, and the measured path leaves the previewed one beyond its tolerance. The harness stops the arm and reports only this miss, and the model identifies the rim as the cause from the fresh view. It then previews a revised motion that clears the rim, which completes the placement in a second round. This check targets the main source of divergence, contact with an object or fixture, which the planner does not model, so the harness detects its consequence instead of predicting it.

Looking where the model chooses. Some divergences leave the robot on its planned path and show only in the scene, such as an object that slips or shifts during a grasp. The model can therefore insert checkpoints at moments it considers critical, either unconditionally or only when the measured gripper opening after a grasp falls outside the range it expects, for example when the fingers close on nothing. At a checkpoint the robot stops and the model receives a fresh view, after which it either continues the remaining sequence as previewed or replaces it. In Fig. 6, the model places a checkpoint after the lift, and the sequence continues as previewed once the fresh wrist view shows the object still held. Checkpoints are placed by the model rather than at fixed points, since pausing after every gripper action by default fragmented long tasks. Acting on what a stop or a checkpoint reveals, however, often requires facts that the model cannot read reliably from RGB, such as whether an object is held and where it sits relative to the gripper, which the following section provides.

7 Perception and Action Interfaces

Faithful execution helps only if the motion the model chooses is the right one, which depends on what the model can perceive and on how it can express what it wants. A general-purpose vision-language model reads a scene and plans a task well, but it is weak at the quantitative side of manipulation. This section describes how the harness compensates on both sides, by turning physical state into facts the model can read (Sec. 7.1) and by letting the model state its intent in terms of the task (Sec. 7.2), as summarized in Fig. 7. Two principles guide both sides. The harness *informs but does not decide*, reporting measurements and estimates together with how reliable they are and leaving the choice of what to do with them to the model. It also *attaches facts at the point of decision*, so that information that should change the next command arrives with the preview or the report instead of waiting to be requested.

7.1 Readable Physical State

On the input side, every quantity the model needs for its next decision reaches it as a measured, computed, or estimated fact together with how reliable it is, rather than as something the model has to infer from pixels. We describe how the model locates things, learns what an action did, and recalls what happened earlier.

3D localization. The model understands the layout of a scene well from RGB, but an image does not show metric 3D positions directly, so any estimate made from appearance alone remains uncertain. A deployed robot, however, comes with physical quantities that are known exactly, namely the intrinsic and extrinsic calibration of every camera, the height of the work surface, and the kinematic model of the robot itself. With the calibration, every pixel (u, v) of a camera with intrinsics K, orientation R, and center $\mathbf{c}$ lies on a known ray with direction $\mathbf{d} = R\,K^{-1}(u, v, 1)^\top$, so locating it in 3D reduces to finding its distance along this ray. The harness provides two tools that supply this distance at different levels of precision (Fig. 7a). The first estimates every visible object in 3D. VGGT-Omega [47] predicts a depth for every pixel of the head-camera

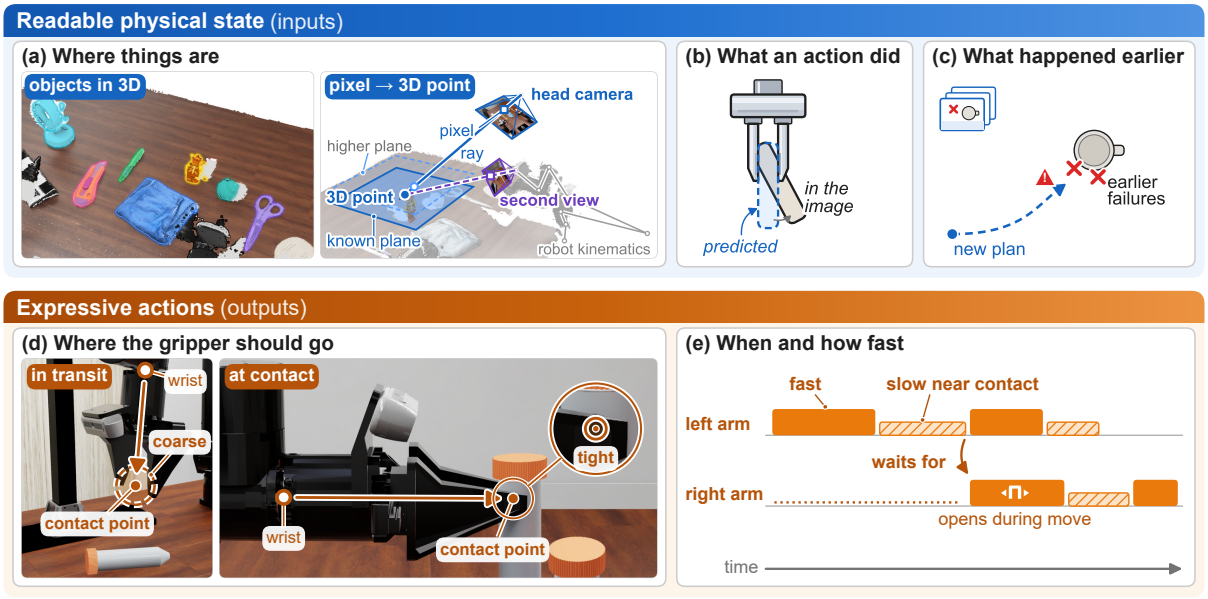


Figure 7 Perception and action interfaces in Auriga. (a) Objects separated and placed in 3D from the head-camera image, and the position of a selected point from its pixel ray and a plane of known height or a second view. (b) The object estimates keep a held object at its pose in the hand at the grasp, and a measurement against this reference shows that it has turned in the hand. (c) A new plan that returns to where earlier motions failed carries a warning, and earlier frames remain retrievable. (d) A goal refers to the contact point on the fingers rather than the wrist, with a coarse tolerance in transit and a tight one at contact. (e) Both arms share one timeline, with declared waits, a speed per movement that slows near contact, and a gripper that opens during a movement.

image, which places the pixel on its ray, and the depths are scaled so that the table lies at its known height. SAM 3.1 [2] separates the objects in these points, and each object is reported with its 3D position, orientation, and size and with how reliable the estimate is, in every full observation and every action report. Where such an estimate is not precise enough, as for placing the fingers on a narrow edge, the second tool computes the position of a point that the model selects in an image. The model fixes its distance either with a plane $\mathbf{n}^\top \mathbf{p} = h$ of known height,

$$\mathbf{p} = \mathbf{c} + \frac{h - \mathbf{n}^\top \mathbf{c}}{\mathbf{n}^\top \mathbf{d}} \mathbf{d}, \qquad \frac{\partial \mathbf{p}}{\partial h} = \frac{\mathbf{d}}{\mathbf{n}^\top \mathbf{d}}, \tag{4}$$

or by marking the same point in a second view, such as the wrist camera, from which the harness triangulates it. The two tools meet in the plane height. For a point on the table, the height is known exactly. For a point on an object, the model either takes the height from the learned depth of that object, in which case the harness reports how far the point would move if this height were off, or marks the point in a second view, which needs no height. Any 3D point, including the fingers placed by the kinematic model, can be projected back into the image to check it.

Action outcomes and in-hand state. After an action, the model needs its physical outcome. For most of the scene, the next image shows it, and each report also lists the objects whose estimated position changed during the action. A grasp is harder, since the fingers often hide the object, so the image may not show the state of the object and how it interacts with the gripper. The harness therefore complements the image with measurements of both. For the gripper, every report includes its measured opening. For the object, the model can measure on request where a visible part of it, such as its lower end, lies relative to the gripper, the axis of the object in the 3D points around the fingers, and how far a part marked in a wrist image before an action has moved after it. The object estimates also keep the pose that the object had in the hand at the grasp, as a reference for these measurements. Together, these tell the model whether the object is held, how it sits in the

hand, and whether it has turned or slipped since the grasp.

Episode history and warnings. In long episodes the context is compacted and early images are dropped, while recovery often depends on what earlier attempts actually achieved. The model can therefore retrieve from the evidence record E_t (Sec. 5.1) any earlier camera frame, including a magnified region of it, a list of past actions with the displacement that each one measured, and the planned and measured trajectories of any action. Full history is retrieved on demand rather than pushed into every turn, since forced reviews of history produced idle calls. The exception is history that bears directly on the next command. A report of a stopped or failed action carries a short summary of recent outcomes, and a preview that returns to a place where a motion has failed carries a warning, so that the model does not unknowingly repeat a failed attempt.

7.2 Expressive Actions

On the output side, the model states what it wants in terms of the task, such as where the fingers should touch and how the two arms should cooperate, and the harness turns these statements into motion. We describe how the model specifies the goal of each movement and how it sets the timing and speed of a sequence.

Finger-referenced goals. A robot interface typically takes the pose of the wrist as its goal, whereas what matters for a task is where the fingers touch the object and how precisely they get there. In Auriga, each movement states its goal in these terms. The model gives where the gripper should end up and how it should be turned in a frame fixed to the workspace, together with the part of the fingers that should make contact, such as the fingertip or the pad center, so that the goal refers to the point that touches the object rather than to the wrist. It also states how precisely the gripper must arrive, so that a tight insertion and a coarse transit are checked differently.

Shared timeline and speed. Goals alone do not fix how a sequence unfolds in time. When both arms take part, or when a movement passes from free space to contact, the order and the speed of its parts matter as much as its goals. In Auriga, the stages of both arms belong to the same sequence and advance on a shared timeline. Stages of the two arms run at the same time unless the model declares that one must wait for another. The model also sets the speed of each movement, typically slower near contact, and can let the gripper open or close during a movement, for example opening it on the way to an object. Speed and gripper timing are fixed in the preview like the path itself, so they are part of the reviewed motion.

8 Experimental Setup

We evaluate Auriga under the zero-shot protocol of Sec. 2 on two simulation benchmarks: dual-arm tabletop manipulation on **RoboDojo** [4] (Sec. 8.1) and kitchen mobile manipulation on **RoboCasa365** [32] (Sec. 8.2).

8.1 RoboDojo

RoboDojo [4] simulates a dual-arm ARX X5 robot with one head camera and two wrist cameras in Isaac Sim.

Capability dimensions. RoboDojo groups its 42 tasks into five capability dimensions: Generalization (12 tasks, each with standard and random scenes), Precision (8), Long-Horizon (8), Memory (6), and Open (8); Tab. 18 in appendix A lists what each dimension covers and its tasks. As on the official leaderboard, results are first averaged across the tasks of each dimension and then across the five dimensions, with equal weight at both levels. For Generalization, the value of each task and of the dimension is the equal-weight mean of its standard-scene average and its random-scene average.

Evaluation set. Following Sec. 2, a case specifies a task, a scene variant (standard or random), an evaluation seed, and a layout, and an episode is one execution of a case by the agent. The evaluation set follows the protocol of the official leaderboard, with three evaluation seeds and, in each seed, 50 cases for each of the

42 tasks, that is, 2,100 cases per seed and 6,300 in all. In each seed, every Generalization task contributes 25 cases with standard scenes and 25 with random scenes, and every other task contributes 50 with standard scenes. Each case counts with its first official score, including scores of zero, and any later episode of the same case is ignored. When an infrastructure fault ends an episode before the evaluator has scored it, the case is re-run under the same configuration.

Metrics. *SR* is the share of cases that the benchmark evaluator judges fully successful, over a fixed denominator: a case without an official score stays in the denominator and does not count as a success. *Score* is the official graded process score of the evaluator, from 0 to 100, averaged over the cases with an official score. Both metrics are first computed for each task in each seed, and a task's value is the mean of its three per-seed values; the dimension values and the five-dimension average then follow from the task values as in Sec. 8.1. *Steps* are the native steps that the simulator executes in an episode, also reported as a percentage of the official native-step limit of the task, which ranges from 200 to 1,900 steps; a single action request may use at most 250 steps. *Calls* are the model calls (the model responses in an episode) and the direct tool calls issued by the model; tool calls issued by model-written programs are counted separately. *Cost* comprises the input tokens of the model session (total, share served from the prompt cache in %, and uncached input tokens), the output tokens, and the wall-clock time of the episode; Steps, Calls, and Cost are reported as means per episode.

Agent configuration. The RoboDojo results (Secs. 9.1, 9.3 and 9.4) use Auriga as described in Secs. 5 to 7, with the model `gpt-6-astra` at reasoning effort high. An episode may use at most 170 model responses and the official native-step limit of its task, and the simulation is paused while the model deliberates (Sec. 5.1). The model observes RGB images from the three cameras, proprioception, and the public camera calibration, together with the object estimates that the harness computes from the head-camera image (Sec. 7.1); simulator depth and hidden object state are withheld. Its task text is the native instruction of the task, the official task page with its description and scoring rules, and the task-independent operating guidance of the harness (Secs. 5.1 and 5.2). The ablations in Sec. 9.5 remove or restrict individual mechanisms of this configuration while keeping the rest fixed.

8.2 RoboCasa365

RoboCasa365 [32] evaluates household mobile manipulation with a single Franka Panda arm mounted on an Omron mobile base across kitchen environments, spanning 365 tasks (65 atomic and 300 composite).

Tasks and task groups. Evaluation focuses on the benchmark's 50 Target50 tasks, organized into three groups: 18 *Atomic-Seen* tasks involving single manipulation skills (e.g., opening a drawer or operating a microwave); 16 *Composite-Seen* tasks that chain two to fifteen subtasks whose templates appear in the benchmark's pretraining demonstrations; and 16 *Composite-Unseen* tasks of comparable horizon whose templates are absent from the pretraining demonstrations.

Evaluation protocol and metrics. Whereas the standard policy leaderboard [40] evaluates demonstration-trained policies in the 2,500 pretraining kitchens, agent evaluations are conducted in ten unseen target kitchens with novel layouts, fixtures, and objects. We follow the protocol of the published agent baselines [44, 62]: every task is evaluated at seeds 1–5, giving 250 episodes (90 Atomic-Seen, 80 Composite-Seen, and 80 Composite-Unseen). We report the success rate (SR, %) of each group, of all 160 composite episodes, and *Overall* as the mean over the 50 tasks, which equals the SR over all 250 episodes. The benchmark environment provides camera images, robot proprioception, world-coordinate maps derived from rendered depth, the source code of the task's success check, and live values of the predicates tested by that check.

Baselines. Both published agent baselines plan with an LLM agent and hand contact-rich phases, such as grasping an object or turning a knob, to the RLDX-1 vision-language-action (VLA) policy [17]. RPent [62] also retrieves task memory, built by exploring each task on seed 0 before evaluation. PyRUA-Lean [44] uses no cross-episode memory. We also list RLDX-1 on its own and, as in the comparison of RPent, three learned

policies evaluated in the pretraining kitchens.

Agent configuration. Auriga runs `gpt-6-astra` at `high` reasoning effort with at most 100 model requests per episode. Unlike both agent baselines, it uses no VLA policy and no cross-episode memory. Alongside the benchmark-provided observations and task predicates, Auriga segments camera images via SAM 3.1 [2] from model-chosen text prompts and projects each mask onto the world coordinate map, without reading ground-truth object poses. All physical motions are executed through multi-stage action programs under the preview-and-verification gateway of Sec. 6, coordinating stages for the arm, gripper, torso, and mobile base.

9 Main Results

This section reports the results on RoboDojo (Sec. 9.1) and on RoboCasa365 (Sec. 9.2), followed on RoboDojo by episode cost (Sec. 9.3), tool usage across capabilities and dimensions (Sec. 9.4), the controlled ablation of the harness mechanisms (Sec. 9.5), and cross-task harness evolution (Sec. 9.6).

9.1 Results on RoboDojo

Tab. 4 places Auriga on the official leaderboard [41], reporting Score and SR on average and in each dimension. The table has two blocks: the eight leading entries that use a learned policy, either a world action model (WAM) or a vision-language-action (VLA) policy, alone or with an agent; and four agent methods, which act through tools or code, together with Auriga. Averaged across the five dimensions, Auriga reaches a Score of 64.00 and an SR of 57.74%. Across both blocks of Tab. 4, Auriga achieves the highest overall averages, ahead of GO-3 Nano Preview, which combines an agent with a world action model, at 61.90 and 56.42%. Within the agent block, Auriga leads all twelve columns, including both averages (ahead of PyRUA-Lean at 55.26 and 50.35%). According to their reports [11, 20, 44, 61], PyRUA-Lean, HexaShell-Preview, EmbodiedEvo, and RoboProbe all use `gpt-6-astra`, the same base model as Auriga (marked † in Tab. 4), with reasoning effort high for PyRUA-Lean, xhigh for EmbodiedEvo, and medium for RoboProbe; Auriga uses high. Against RoboProbe [61], the reference harness released with the evaluation of GPT-6 Astra on RoboDojo [4], Auriga raises the average Score from 28.97 to 64.00 and the average SR from 22.48% to 57.74% on the same model (Tab. 4).

By dimension. Across both blocks of Tab. 4, Auriga has the highest Score and SR in Generalization, Memory, and Open, ahead of the next entry by 5.72, 11.99, and 4.83 percentage points in SR. Precision and Long-Horizon remain the most challenging dimensions for Auriga, at 38.50% and 45.24% SR, against 76.00% and 53.50% for GO-3 Nano Preview, which leads both, though Auriga ranks second overall in both Score and SR across Precision (46.30 and 38.50%) and Long-Horizon (62.10 and 45.24%).

Profile against other methods. Fig. 8 compares the five-dimension profile of Auriga with seven other methods of Tab. 4: the four highest-scoring entries of the WAM / VLA block (GO-3 Nano Preview, WorldScape-R^{2}SI, WALL-OSS-1.0-lite, and VPP2) and the three highest-scoring other entries of the agent block (PyRUA-Lean, HexaShell-Preview, and EmbodiedEvo); Fig. 24 in appendix D plots the same values as bars, together with RoboProbe. GO-3 Nano Preview has the opposite profile to Auriga: its Score exceeds that of Auriga by 32.35 in Precision and 6.11 in Long-Horizon, while falling below Auriga in the other three dimensions, most in Memory (64.50 vs. 93.40) and Generalization (36.45 vs. 52.90). Against PyRUA-Lean, Auriga achieves a higher Score in all five dimensions, with gains of 1.53 in Precision, 4.63 in Generalization, 4.84 in Open, 7.80 in Long-Horizon, and 25.00 in Memory.

Per task. Tab. 23 and Fig. 25 in appendix E compare Auriga task by task with PyRUA-Lean, HexaShell-Preview, EmbodiedEvo, and RoboProbe. Against each of the four, Auriga is higher in Score on more of the 42 tasks than it is lower, from 24 vs. 17 tasks against PyRUA-Lean to 41 vs. 1 against RoboProbe. In SR, it is likewise higher on more tasks against all four methods: PyRUA-Lean (22 vs. 16), HexaShell-Preview (28 vs.

Table 4 Results on the RoboDojo simulation benchmark, with external entries from the official leaderboard [41] (Sec. 8.1), reporting Score (0–100) and SR (%) in two blocks ordered by ascending average Score: the eight leading entries that use a world action model (WAM) or a vision-language-action (VLA) policy, alone or with an agent, and four agent methods on the leaderboard (Agent, acting through tools or code) together with Auriga. In each column, the highest value is in bold and the second highest is underlined. †Uses `gpt-6-astra`.

		Average		Generalization		Precision		Long-Horizon		Memory		Open	
Method	Type	Score	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score	SR
Learned policies: WAM or VLA, alone or with an agent													
Rex-M1 Preview	VLA	33.79	27.44	34.05	26.05	32.82	23.83	46.89	33.00	48.53	48.22	6.67	6.08
Simate-beta	VLA	33.95	27.96	35.09	27.95	34.35	26.92	57.84	43.42	33.33	33.00	9.12	8.50
Awomo-0.5	WAM	35.34	29.64	30.31	23.84	39.65	33.50	40.36	26.75	42.27	41.22	24.09	22.92
PhysicalRSI	Agent + VLA	36.27	31.38	21.30	15.62	38.05	32.50	46.37	37.33	46.74	46.56	28.88	24.92
VPP2	WAM	39.26	32.26	36.38	29.50	43.03	35.17	56.28	37.00	56.01	55.56	4.61	4.08
WALL-OSS-1.0-lite	VLA	42.59	34.97	46.66	37.94	38.16	29.08	49.94	32.25	57.42	56.00	20.76	19.58
WorldScape-R^{2}SI	Agent + WAM	46.58	39.35	42.95	35.83	23.51	15.50	51.48	34.25	65.78	63.67	49.16	47.50
GO-3 Nano Preview	Agent + WAM	61.90	56.42	36.45	33.66	**78.65**	**76.00**	**68.21**	**53.50**	64.50	63.67	61.71	55.25
Agent methods, acting through tools or code													
RoboProbe†	Agent	28.97	22.48	33.36	30.50	12.65	4.00	21.45	8.25	43.04	38.67	34.36	31.00
EmbodiedEvo†	Agent	45.08	38.95	35.22	30.83	25.15	16.25	33.83	21.75	79.40	76.67	51.81	49.25
HexaShell-Preview†	Agent	53.18	46.85	39.22	34.50	41.25	32.00	42.91	31.50	84.53	81.00	57.97	55.25
PyRUA-Lean†	Agent	55.26	50.35	48.27	43.17	44.77	38.25	54.30	43.75	68.40	68.33	60.56	58.25
Auriga†	Agent	**64.00**	**57.74**	**52.90**	**48.89**	46.30	38.50	62.10	45.24	**93.40**	**92.99**	**65.40**	**63.08**

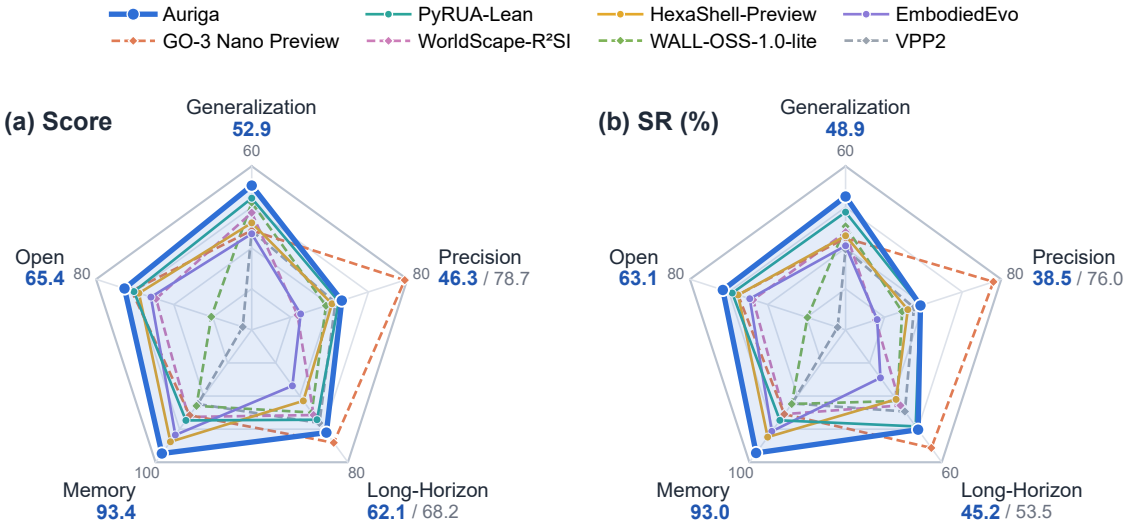


Figure 8 Five-dimension profile of Auriga and seven other methods of Tab. 4, in Score (a) and SR in % (b). Each axis runs from zero at the center to the value at its outer vertex, with four evenly spaced rings. Below each dimension name is the value of Auriga, followed after a slash by the highest value of the other methods where it is higher. Auriga is the thick blue line with the shaded area; the other agent methods are drawn with solid lines and the WAM / VLA methods with dashed lines.

9), EmbodiedEvo (32 vs. 6), and RoboProbe (38 vs. 1).

Generalization under randomization. Fig. 9 compares the entries of Tab. 4 on the 12 Generalization tasks

across the standard and random scenes, reporting the SR change $\Delta\text{SR} = \text{SR}_{\text{rand}} - \text{SR}_{\text{std}}$ in panel (a) and the composite generalization metric $G = \frac{1}{2}(S_{\text{std}} + S_{\text{rand}}) - |S_{\text{rand}} - S_{\text{std}}|$ in panel (b), which rewards a high mean Score and penalizes the gap between the two scene types (Tab. 22 in appendix D lists the SR and Score on both scene types). Auriga achieves the highest G in Fig. 9 at 46.27, ahead of PyRUA-Lean at 43.87 and WorldScape-R^{2}SI at 35.61. Across both scene types, Auriga achieves the highest SR and Score of all 13 entries: 52.28% SR and 56.22 Score on the standard scenes, and 45.61% SR and 49.58 Score on the random scenes, with a change of -6.7 percentage points in SR (-13% relative) and -6.6 points in Score (Tab. 22). By contrast, the learned WAM and VLA entries suffer steep drops under scene randomization, losing up to 33.9 percentage points in SR (31.5 points in Score) for VPP2 and 18.7 percentage points (-43% relative) for GO-3 Nano Preview (Fig. 9 and Tab. 22).

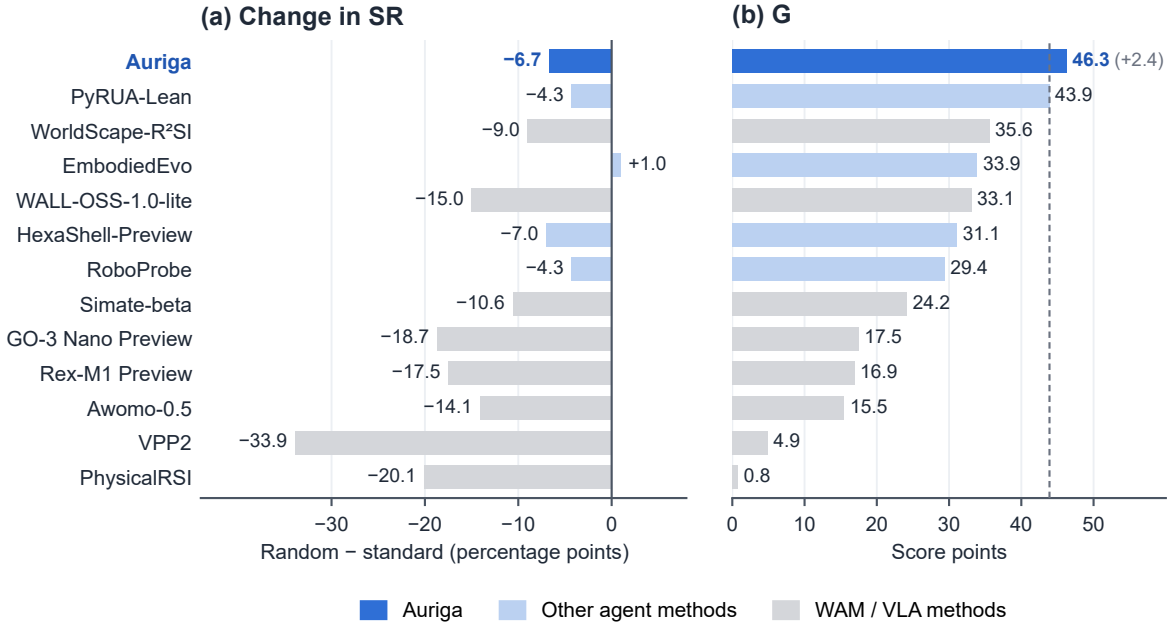


Figure 9 Generalization of the entries of Tab. 4 on the 12 Generalization tasks under scene randomization, ordered by G. (a) Change in SR (percentage points, $\Delta\text{SR} = \text{SR}_{\text{rand}} - \text{SR}_{\text{std}}$) from standard to random scenes. (b) $G = \frac{1}{2}(S_{\text{std}} + S_{\text{rand}}) - |S_{\text{rand}} - S_{\text{std}}|$, the mean of the standard- and random-scene Scores minus their absolute difference.

9.2 Results on RoboCasa365

RoboCasa365 shifts the evaluation to a mobile manipulator operating doors, drawers, knobs, and appliances across unseen kitchens over chains of up to fifteen subtasks (Sec. 8.2). Tab. 5 and Fig. 10 compare Auriga with the published agent baselines and with RLDX-1 on the 250 Target50 episodes (Sec. 8.2). All three agent methods use `gpt-6-astra` at high reasoning effort, and the baseline results are those reported by RPent [62] and PyRUA-Lean [44]. As in the comparison of RPent, Tab. 5 also lists WorldDreamer, $\pi_{0.5}$, and π_0, which are evaluated in the pretraining kitchens; for these policies, Composite All is the mean of the two composite groups. These policies, together with the leaderboard entry Xiaomi-Robotics-1, are also listed in Tab. 20 in appendix C.

Comparison with agent baselines. Auriga succeeds in 73.2% of the 250 episodes, against 59.2% for RPent and 56.4% for PyRUA-Lean with the same model and reasoning effort, and 30.0% for RLDX-1 alone. It has the highest success rate in every group of Tab. 5. The margin is largest on the composite tasks, which chain several subtasks: Auriga completes 62.5% of these episodes, 19.4 percentage points more than RPent and

Table 5 Success rates (%) on the 50 Target50 tasks of RoboCasa365 [32], at seeds 1–5 per task (250 episodes), ordered by ascending Overall SR within each evaluation setting. Overall is the mean over the 50 tasks, and Composite All covers the 160 composite episodes. Agent + VLA methods hand contact-rich phases to the RLDX-1 policy. WorldDreamer, $\pi_{0.5}$, and π_0 are evaluated in the pretraining kitchens, all other methods in the target kitchens. –: not reported. Bold: highest; underlined: second highest.

				Composite		
Method	Type	Overall	Atomic-Seen	Seen	Unseen	All
Pretraining kitchens: learned WAM or VLA policies						
π_0	VLA	14.8	34.6	6.1	1.1	3.6
$\pi_{0.5}$	VLA	16.9	39.6	7.1	1.2	4.2
WorldDreamer	WAM	35.3	66.3	26.7	9.0	17.9
Target kitchens: VLA policy and agent methods						
RLDX-1	VLA	30.0	60.0	21.3	5.0	13.1
PyRUA-Lean	Agent + VLA	56.4	86.7	–	–	39.4
RPent	Agent + VLA	<u>59.2</u>	<u>87.8</u>	<u>43.8</u>	<u>42.5</u>	<u>43.1</u>
Auriga	Agent	**73.2**	**92.2**	**67.5**	**57.5**	**62.5**

23.1 more than PyRUA-Lean. Both baselines rely on the RLDX-1 policy for contact-rich phases, and RPent also on task memory gathered before evaluation; Auriga uses neither.

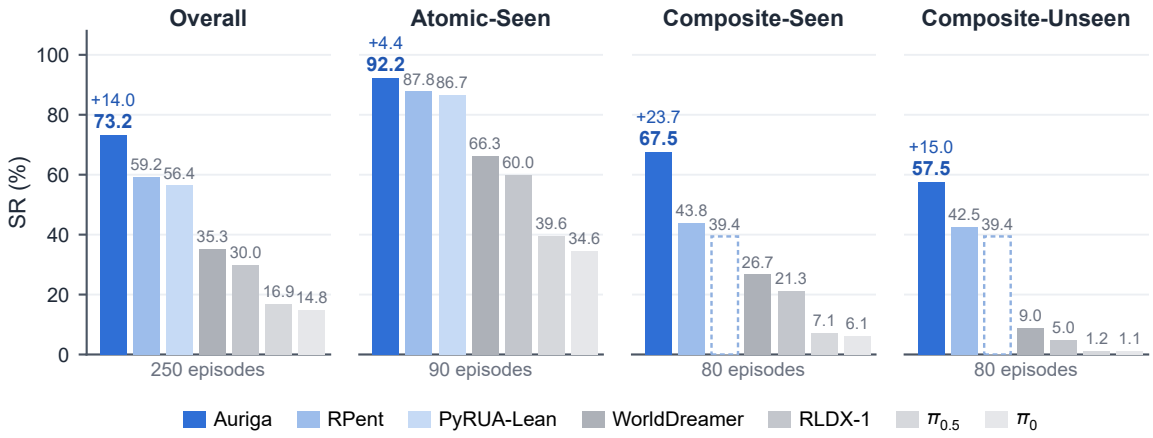


Figure 10 Success rates (%) of the methods of Tab. 5 on RoboCasa365 Target50, in descending order of Overall SR. Grey bars are learned policies, and light blue bars are agent methods that hand contact-rich phases to the RLDX-1 policy. PyRUA-Lean reports composite tasks only as a whole, so its two composite bars are outlined and both show that rate. Above the bar of Auriga is its difference from the highest other method of the group.

Where Auriga gains and where it fails. The composite gap reflects how contact-rich phases are executed. The baselines hand them to RLDX-1, which on its own completes 21.3% of the Composite-Seen and 5.0% of the Composite-Unseen tasks, while Auriga builds every phase, including grasps, placements, and fixture operations, from action programs whose stages are checked as they execute (Sec. 6). Of the 67 failed episodes of Auriga, 23 ended when the 100 model requests were used and 44 after the model stopped acting for several turns (Fig. 23 in appendix C). Four composite tasks fail in all five episodes, PackIdenticalLunches and StoreLeftoversInBowl among the seen and MakeIceLemonade and SeparateFreezerRack among the unseen, while every Atomic-Seen task succeeds in at least three of its five episodes.

9.3 Episode Cost

Of the other methods in Tab. 4, PyRUA-Lean [44], HexaShell-Preview [11], and EmbodiedEvo [20] report the test-episode cost of their RoboDojo runs. Tab. 6 lists the cost per test episode of these three methods and of Auriga (measured over the 6,299 scored episodes of Tab. 21), together with each method's overall average Score in Tab. 4. Of the methods that report input tokens, Auriga achieves the highest average Score (64.00) and uses 5.63 million input tokens per episode, of which 85.4% are served from the prompt cache, leaving 0.82 million uncached input tokens per episode (against 3.94 million total and 0.09 million uncached for EmbodiedEvo at 45.08 Score, and 0.75 million total for PyRUA-Lean at 55.26 Score). An Auriga episode averages 47.5 model calls and 18.8 minutes, compared with 25.7 calls and 12.7 minutes for PyRUA-Lean and a median of 12.3 minutes for HexaShell-Preview. Note that cross-trial evolution methods (Tab. 1) also incur pre-evaluation trial costs that are not included in their per-test-episode numbers, whereas Auriga runs entirely within the evaluated episode; on matched zero-shot episodes against the reference harness (Tab. 15), Auriga uses fewer model calls (25.4 vs. 36.6) and 90.6% fewer physical execution batches.

Fig. 11 compares the five-dimension Score of Auriga (Tab. 4) against the mean input tokens and uncached input tokens per episode across dimensions (Tab. 21 in appendix D). Memory, the dimension with the highest Score (93.40), is also the cheapest, with 30.0 model calls, 3.11 million input tokens (0.45 million uncached), and 11.3 minutes per episode, and Open, with the second-highest Score (65.40), is the second cheapest, with 40.7 model calls, 4.67 million input tokens (0.65 million uncached), and 14.9 minutes. Long-Horizon and Generalization episodes are the most expensive, with 54.5 and 54.2 model calls, 6.70 and 6.57 million input tokens, and 23.0 and 21.6 minutes, and across all 6,299 episodes Auriga uses 79.7% of the native-step limit on average (Tab. 21).

	Auriga	PyRUA-Lean	HexaShell-Preview	Embodied-Evo
Reasoning effort	high	high	–	xhigh
Score	**64.00**	55.26	53.18	45.08
Model calls	47.5	25.7	–	–
Input (M)	5.63	0.75	–	3.94
Cached (%)	85.4	–	–	97.6
Uncached (M)	0.82	–	–	0.09‡
Output (k)	18.3	9.0	–	19‡
Time (min)	18.8	12.7	12.3§	–

Table 6 Cost per RoboDojo episode of Auriga (over the 6,299 episodes in Tab. 21) and of the three methods in Tab. 4 that report it, with each method's average Score from Tab. 4; the quantities are defined in Sec. 8.1. All four use `gpt-6-astra`; a dash marks a quantity the method does not report. ‡Derived from reported totals or shares. §Median.

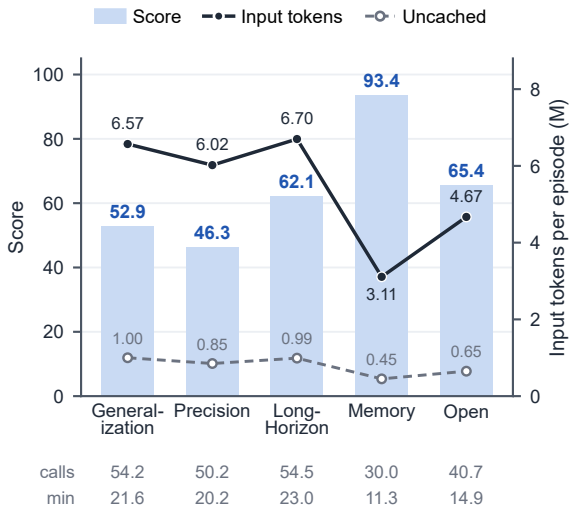


Figure 11 Five-dimension Score (Tab. 4) and per-episode cost of Auriga by dimension (over the 6,299 episodes in Tab. 21): five-dimension Score (bars, left axis), mean input tokens per episode in millions (solid line, right axis), and uncached input tokens in millions (dashed line, right axis). Below the axis: model calls and wall-clock minutes per episode.

9.4 Tool Usage

Figs. 12 and 13 group the tool calls of the model by the four capabilities of Tab. 2: Measure, Act, Compute, and Recall.

Calls by capability. Fig. 12 reports the mean number of direct tool calls per episode and the share of each capability across dimensions. An episode makes 38 tool calls on average, from 45 in Long-Horizon and Generalization episodes to 23 in Memory episodes. Act takes the largest share, 45% of the calls, followed by Measure with 24%, Compute with 17%, and Recall with 15%. Within each capability (Tab. 2), spatial measurement (`geometry_query`) accounts for 16% of all direct tool calls and scene observation (`robot_observe`, alongside the reference overlay `stereo_guide`) for 8% under Measure; under Act, trajectory preview ($\mathcal{U}_{\mathrm{d}}$, equation (2)) accounts for 19% and physical execution ($\mathcal{U}_{\mathrm{p}}$, equation (3)) for 24% in `robot_action` (with `robot_wait` accounting for the remaining 2%), so 44% of all `robot_action` calls preview a candidate sequence without advancing the world; under Compute, running programs (`program_run`) accounts for 17%; and under Recall, updating the task belief (`task_plan`, Sec. 5.3) accounts for 13% while querying evidence records and operation lessons (`history_*` and `operation_memory`, Sec. 5.2) accounts for 2%. In Memory and Long-Horizon episodes, Compute rises to 24% and 20% of the calls, respectively. In addition to the model's direct calls, the programs that the model runs issue another 7 tool calls per episode on average, rising to 10 in Long-Horizon episodes.

Distribution over episode progress. Fig. 13 follows the four capabilities from the first tool call of an episode ($t = 0$) to the last ($t = 1$), with every episode weighted equally. Before $t = 0.05$, Measure takes 87% of the calls (against 10% for Recall, 2% for Act, and 1% for Compute) as the model inspects the scene and localizes targets before acting. From $t = 0.15$ onward, Act and Compute together account for more than 60% of the calls: Act holds between 46% and 54% from $t = 0.15$ to $t = 0.90$ and Compute between 17% and 22% from $t = 0.20$ to $t = 0.90$, while Recall stays between 13% and 17% after $t = 0.15$. In the final segment ($t > 0.95$), Act rises to 63% and Measure drops to 3% (with Recall at 18% and Compute at 16%).

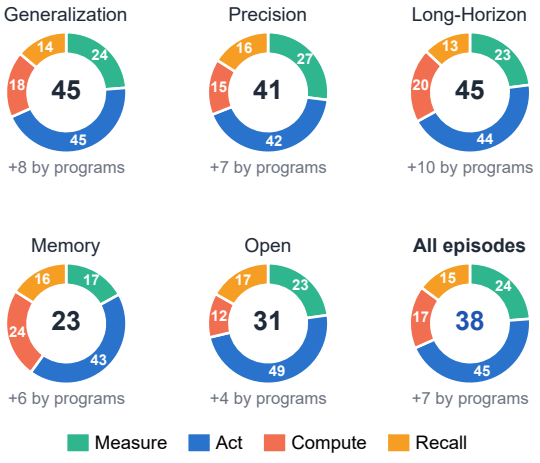


Figure 12 Tool calls per episode by dimension (6,299 episodes): mean number of direct tool calls (center), share of each capability in % (ring segments), and mean number of additional calls made by programs (below each ring).

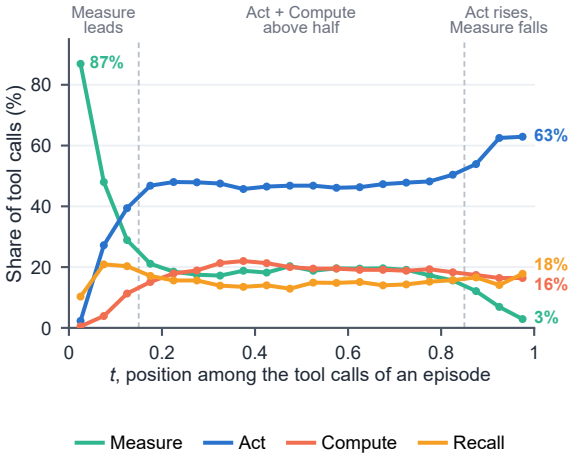


Figure 13 Share of each capability (%) along an episode: the equal-weight mean across episodes (6,298 episodes). The tool calls of each episode are spread in order over $t \in [0, 1]$ and split into 20 equal segments.

9.5 Ablation of the Mechanisms

Tabs. 7 to 11 ablate the mechanisms of Secs. 5 to 7 on the configuration of Sec. 8.1. Each table groups

related mechanisms and evaluates its variants on ten tasks (Tab. 19 in appendix B): Tabs. 7 to 10 on five standard layouts of each task (50 cases), and Tab. 11 on 30 cases. Each variant removes or restricts the mechanisms marked in its row while keeping the rest of the configuration fixed, and the *Full* row reports the main configuration on the same cases. Alongside the number of successes and the mean Score, each table reports a mechanism-specific diagnostic per episode for the ablated variants (and for all rows in Tab. 11), taken from the execution receipts r_t, the evidence record E_t, or the model calls (Secs. 5.1 and 5.2).

Spatial measurement. Tab. 7 evaluates the mechanisms that provide metric 3D positions (Sec. 7.1): the camera calibration and 3D object estimates, together with `geometry_query` for ray–plane intersection (equation (4)), two-view triangulation, and 3D-to-image projection. In *No geometry tools*, the calibration, the 3D object estimates, and `geometry_query` are all removed, so the model locates targets from the RGB images and the robot state alone (Sec. 7.1). In *No triangulation*, the calibration, the 3D object estimates, two-view triangulation, and 3D-to-image projection are removed while ray–plane intersection (equation (4)) remains available at one plane height per pixel. Compared with *Full* (46/50 successes, mean Score 94.9), *No geometry tools* reaches 30/50 successes and a Score of 67.1, with 65.3% of first grasps held. With ray–plane intersection available, *No triangulation* reaches 40/50 successes and a Score of 84.0, with 85.7% of first grasps held; *Full* adds two-view triangulation, 3D-to-image projection, the calibration, and the 3D object estimates (Sec. 7.1).

Table 7 Spatial measurement (Sec. 7.1 and equation (4)): successes over the 50 cases, mean Score, and the share of first grasps that hold (%). The highest successes and Score are in bold. ✓: available; ×: removed.

Variant	Plane intersection	Triangulation	Calibration and object estimates	Successes	Score	First grasps held (%)
No geometry tools	×	×	×	30/50	67.1	65.3
No triangulation	✓	×	×	40/50	84.0	85.7
Full	✓	✓	✓	**46/50**	**94.9**	–

Collision handling. Tab. 8 evaluates two mechanisms that keep execution clear of unintended contact (Secs. 6.2 and 6.3). In *No clearance review*, the preview no longer estimates the clearance between the arms, held objects, and declared obstacles before motion (Sec. 6.2). In *No divergence stop*, the stage loop in $\mathcal{A}$ no longer halts the sequence when a movement exceeds its arrival tolerance $\tau_k = (\delta_p, \delta_\theta)$, so later stages continue from the displaced pose (Secs. 5.4 and 6.3). Compared with *Full* (36/50 successes, Score 79.8), *No clearance review* reaches 31/50 and 69.1, with 1.20 tolerance stops per episode; without the clearance review, the stage check during execution is the remaining guard against contact (Sec. 6.3). Compared with *Full*, *No divergence stop* reaches 34/50 and 75.1, with 0.00 stops and 0.92 out-of-tolerance movements per episode as sequences run on, whereas stage verification in *Full* halts at the first exceeded tolerance and returns a receipt r_t and fresh view (Secs. 5.4 and 6.3).

Table 8 Collision handling (Secs. 6.2 and 6.3): successes over the 50 cases, mean Score, and the stops per episode at a movement that exceeded its arrival tolerance (δ_p, δ_θ). The highest successes and Score are in bold. ✓: available; ×: removed.

Variant	Clearance review	Divergence stop	Successes	Score	Stops per episode
No clearance review	×	✓	31/50	69.1	1.20
No divergence stop	✓	×	34/50	75.1	0.00
Full	✓	✓	**36/50**	**79.8**	–

Preview and checkpoints. Tab. 9 evaluates two mechanisms of the admission operator $\mathcal{A}$ (Secs. 5.2 and 6): previewing a trajectory before execution (Sec. 6.2) and pausing only at model-placed checkpoints (Sec. 6.3).

In *No preview*, an action request is planned and executed in one call, so the model sees the trajectory only after it has run (Sec. 6.2). In *Pause after every gripper action*, $\mathcal{A}$ inserts a checkpoint after every gripper stage that a later movement depends on, rather than only where the model placed one (Sec. 6.3). Compared with *Full* (42/50 successes, Score 88.5), *No preview* reaches 36/50 and 74.7, with 0.96 checkpoint pauses and 4.31 unreviewed motions per episode, 0.98 of which carry a clearance risk that preview reports before motion (Sec. 6.2). Compared with *Full*, *Pause after every gripper action* reaches 37/50 and 78.7 with 13.82 pauses per episode, whereas model-placed checkpoints in *Full* pause only at stages selected for inspection (Sec. 6.3).

Table 9 Preview and checkpoints in $\mathcal{A}$ (Secs. 5.2, 6.2 and 6.3): successes over the 50 cases, mean Score, and the checkpoint pauses per episode. The highest successes and Score are in bold. ✓: available; ×: removed.

Variant	Preview	Checkpoints	Successes	Score	Pauses per episode
No preview	×	placed by the model	36/50	74.7	0.96
Pause after every gripper action	✓	after every gripper action	37/50	78.7	13.82
Full	✓	placed by the model	**42/50**	**88.5**	–

Expressive actions. Tab. 10 evaluates the two output-side mechanisms of Sec. 7.2: finger-referenced movement goals and the shared dual-arm timeline. In *Wrist-flange goals*, each movement specifies the wrist-flange position and quaternion rather than a finger contact point and workspace rotations (Sec. 7.2), and the overlay that projects the finger positions into the image (Sec. 7.1) is removed. In *One arm at a time*, an action request may contain stages for only one arm, so the two arms no longer advance on a shared timeline (Sec. 7.2). Compared with *Full* (39/50 successes, Score 83.9), *Wrist-flange goals* reaches 37/50 and 77.1 with 3.42 unplannable stages per episode, whereas finger-referenced goals in *Full* let the model specify the finger contact region and workspace orientation directly (Sec. 7.2). Compared with *Full*, *One arm at a time* reaches 35/50 and 74.8 with 1.14 unplannable stages per episode, whereas the shared timeline in *Full* coordinates both arms with declared waits in one sequence (Sec. 7.2).

Table 10 Expressive actions (Sec. 7.2): successes over the 50 cases, mean Score, and the planning failures per episode (requested stages that could not be planned). The highest successes and Score are in bold. ✓: available; ×: removed.

Variant	Finger-referenced goals	Shared timeline	Successes	Score	Planning failures per episode
One arm at a time	✓	×	35/50	74.8	1.14
Wrist-flange goals	×	✓	37/50	77.1	3.42
Full	✓	✓	**39/50**	**83.9**	–

Programs. Tab. 11 evaluates how model-written programs interact with $\mathcal{A}$ (Secs. 5.2 and 6.2): in *Full*, a program computes and calls deliberative tools ($\mathcal{U}_{\mathrm{d}}$) freely, while any physical request ($\mathcal{U}_{\mathrm{p}}$) enters $\mathcal{A}$, waits for the model to review its preview (Sec. 6.2), and returns a receipt r_t (Secs. 5.1 and 5.2). In *No programs*, the Compute capability (`program_run`, Tab. 2) is removed, so the model issues every call directly. In *Programs cannot act*, programs still run for computation and deliberative queries, but $\mathcal{A}$ rejects their physical requests, so the model issues every physical action directly (Sec. 5.2). In *Code as policy*, physical requests from a program run without preview review and return no receipts r_t to the program or the model, leaving only the program's output and exit status, as in the paradigm contrasted with Auriga in Secs. 5.2 and 6.1. Compared with *Full* (30/30 successes, Score 100.0, 46.2 model calls per episode), *No programs* reaches 19/30 and 75.3 with 66.0 calls per episode (one episode exhausting the 170-response limit), and *Programs cannot act* reaches 19/30 and 73.0 with 67.1 calls per episode; in *Full*, programs issue 28.8 tool calls per episode while the model executes 5.9 physical actions directly (Sec. 5.2). Compared with *Full*, *Code as policy* reaches 20/30 and 77.2 with 71.8 calls per episode. Its programs issue 120 physical actions over the 30 episodes, of

which 2 halt when a movement exceeds its arrival tolerance and 1 stops at a checkpoint; without receipts r_t, each of these programs still exits with code 0, one of them after issuing 8 further actions (Secs. 5.2 and 6.3).

Table 11 Programs (Secs. 5.2, 6.1 and 6.2): successes over the 30 cases, mean Score, and the model calls per episode. The highest successes and Score are in bold. ✓: available; ×: removed.

Variant	Programs	Physical actions from programs	Review and receipts	Successes	Score	Model calls per episode
Programs cannot act	✓	×	✓	19/30	73.0	67.1
No programs	×	×	✓	19/30	75.3	66.0
Code as policy	✓	✓	×	20/30	77.2	71.8
Full	✓	✓	✓	**30/30**	**100.0**	46.2

9.6 Cross-Task Harness Evolution

Fig. 3 (Sec. 4) tracks Auriga across 13 capability iterations (`#0` to `#12`). From the initial base-model loop (`#0`, 21.5 Score, 4.0% SR on the 50-case set), adding motion preview (`#2`, 26.2 Score), unified dual-arm actions (`#3`, 30.0 Score), and harness refactoring with verbatim task specifications (`#4`, 45.5 Score, 36.2% SR across all 42 tasks) formed the core execution interface. Later iterations added the evidence ledger (`#5`), calibrated 3D geometry and history retrieval (`#6`, 46.6 Score), ROI zoom (`#7`, 49.9 Score), transport recovery (`#8`), stage tolerances and obstacle checks (`#9`, 53.4 Score), held-object perception (`#10`), explicit motion control (`#11`, 58.8 Score), and VGGT-Omega / SAM 3.1 3D perception (`#12`), raising the 42-task Score from 45.5 at `#4` to 64.0 (57.7% SR) for the frozen harness across all five capability dimensions ($+16.8$ Score in Generalization, $+5.6$ in Precision, $+29.5$ in Long-Horizon, $+30.8$ in Memory, and $+9.8$ in Open; inset of Fig. 3 and Tab. 4).

10 Execution Trace Analysis

We examine `swap_T`, a pose-exchange task in the Memory dimension, to study how a remembered objective guides manipulation after the scene changes. The robot must exchange two T-shaped blocks so that each occupies the other's initial position and orientation. This task combines three demands in a compact setting: preserving a spatial reference after its source object moves, coordinating access to a shared workspace, and correcting the object pose when grasping or release changes it. The asymmetric T shape makes orientation part of the goal, so exchanging positions alone is insufficient. We analyze the 15-scene standard-variant diagnostic set for this task, using their task-belief records, action specifications, execution receipts, and evaluator outcomes. The analysis follows how these records connect a persistent goal to changing physical evidence, and then compares task performance with the published RoboProbe results on the same scenes.

10.1 Preserving Object Goals across Changes in Execution

The destination persists while the means of reaching it change. For analysis, let $\hat{T}_{A,0}$ and $\hat{T}_{B,0}$ denote the two initial planar poses comprising planar position and yaw estimated from the calibrated images. The exchange assigns the object goals

$$G_A = \hat{T}_{B,0}, \qquad G_B = \hat{T}_{A,0}. \tag{5}$$

These goals remain fixed when the model changes a grasp or assigns the remaining transfer to another arm. The current object location and its relation to the gripper, however, must be updated from subsequent observations. This distinction matters after slippage: re-localizing the fallen block supplies a new pickup pose, while the saved initial pose still supplies its destination. The recorded `task_plan` separates these

roles through the plan, attention, and ledger described in Sec. 5.3. Its updates retain the original placement requirement while revising which physical conditions are supported and which remain unresolved.

A completed motion can still require a task-level correction. Tab. 12 follows a successful exchange in which the pink block has already reached the blue block's initial pose. During the remaining right-arm transfer, the blue block slips onto the table. The crossing stage reports `completed`, but its grip feedback records a zero final aperture, and the returned image places the block to the left of its target. The next belief revision marks blue retention as `contradicted` while retaining the pink placement as `supported`. The model then selects the nearby left arm, re-localizes the blue stem, and specifies a short, slow transfer to the original target. Thus the correction changes the arm, pickup coordinates, and transport motion together, while preserving both the completed placement and the outstanding object goal.

Table 12 Task state, execution feedback, and correction in a successful T-block exchange. Entries condense the recorded task belief and actions; steps are cumulative native actions from receipt budget counters. The destination remains the opposite block's initial pose throughout.

Step	Recorded state and feedback	Decision or verified outcome
72	Both blocks are lifted; the belief records retention as `supported` and the exchange as `pending`.	Preserve both initial outlines; place pink first and inspect its release.
169	Pink has been released at blue's initial pose; blue remains to be transferred.	Keep the completed placement and move blue with the right arm.
278	The transfer motion completes, but the final aperture is zero and the image shows blue on the table.	Mark blue retention as `contradicted`; preserve pink as `supported`; switch recovery to the left arm.
332	The left-arm re-grasp and slow lift complete; the right arm returns.	Use a short, slow placement at the saved target, followed by release and withdrawal.
380	The evaluator reports task success within the 400-step budget.	Both object goals are satisfied after the local recovery.

Recovery preserves completed work. The two ledger entries distinguish an unresolved blue transfer from an already supported pink placement. Consequently, the model does not repeat the entire exchange or derive a new destination from the displaced scene. It repairs the remaining relation between one object and its saved goal. This is the operational value of maintaining task state: evidence can invalidate a local assumption without erasing the broader objective or the progress already achieved. It also explains why counting only `tracking_error` receipts would miss an important class of recovery. Here, the discrepancy lies between successful robot motion and unsuccessful object transport, and is exposed by combining the receipt with the resulting observation.

10.2 Using Feedback to Revise Feasible Operations

Different records support different decisions. Tab. 13 summarizes mechanism use across the 15 episodes. Geometric queries provide coordinates for pickup and placement; belief updates retain the goal and revise progress; previews evaluate candidate operations before execution. All episodes use each of these capabilities. Their roles become concrete in the preceding trace: the saved target survives the transfer failure, a fresh localization supplies the new grasp, and the revised action specification changes which arm performs the remaining work. The table counts usage at the interface where it occurs, including calls made through model-written programs.

Coordination is expressed at the scale of an operation. Multi-stage plans account for 73 of the 74 generated plans. The sequence can include pickup, lift, transfer, release, and withdrawal, allowing the model to consider the entire operation before committing to it. The 54 plans involving both arms include staggered motion and one-arm holding while the other places an object. This is well suited to pose exchange: both destinations are

Table 13 Mechanism use in the 15 T-block exchange episodes. Belief updates exclude the initial submission. Preview evaluations include unsuccessful candidates; generated plans are the candidates returned with a plan identifier. Plan and guard counts refer to specifications, including revisions. Observation pauses count actual receipt-reported stops.

Mechanism	Interface	Episodes	Count and unit
Task-belief updates	`task_plan`	15/15	25 updates
Geometric queries	`geometry_query`	15/15	17 calls
Preview evaluations	`robot_action`	15/15	106 evaluations
Generated action plans	`robot_action`	15/15	74 plans
Multi-stage plans	`move`, `gripper`, `observe`	15/15	73 plans
Plans involving both arms	`arm`	15/15	54 plans
Grasp-guard specifications	`grip_guard_preset`[†]	15/15	70 stages
Observation pauses	`observation_checkpoint`[‡]	1/15	1 pause

[†]Includes explicit aperture bounds as well as guard presets. [‡]The receipt reason identifying an observation pause; a pause returns control for inspection and does not itself imply an error.

initially occupied, and the arms need access to the same central region. The recorded plans repeatedly resolve this dependency by holding one block clear, delivering the other, and then completing the exchange. The relevant coordination is the ordering of access and the preservation of held objects, which can be expressed within one plan even when the arms move at different times.

Preview makes correction cost available before motion. A second successful exchange shows how the model uses the remaining budget when a late observation contradicts placement. After 345 native steps, the blue block is correctly placed, but the red block's stem points in the wrong direction. The next belief revision preserves the required destination and calls for a right-arm re-grasp and approximately 94-degree clockwise correction. Tab. 14 records the subsequent candidate costs. With 55 steps remaining, the model shortens the route and coordinates gripper motion with the approach and return. The executed specification also contains an explicit release stage. Task success is reached at step 396. The useful change is therefore more specific than issuing another grasp: the correction is reorganized to include the outstanding orientation change, release, and return within the available budget.

Table 14 Recorded preview costs during the late orientation correction. The world remains at native step 345 during these previews, with 55 steps available. The first two candidates are rejected because their planned lengths exceed the remaining 55-step native budget. Candidate rows give planned lengths; the last row gives steps actually used before the evaluator signals success.

Candidate	Native steps	Status
Initial correction	95	Rejected: insufficient budget
Revised route	57	Rejected: insufficient budget
Coordinated approach and return	54	Planned
Final plan with explicit release	55	Planned
Executed correction to task success	51	Success at step 396

The 106 preview evaluations and 74 generated plans in Tab. 13 reflect this distinction between evaluating a candidate and obtaining a plan for execution. The remaining 32 evaluations report unsuccessful candidates without generating such a plan. Likewise, the observation pause provides an opportunity to inspect intermediate results; it is counted separately from grasp-guard specifications and execution errors. The two traces show how deliberation and physical execution interact: one revises the physical route after object loss, while the other revises the cost and timing of a correction before moving.

10.3 Comparison with RoboProbe

We compare the 15 standard-variant scenes with the corresponding episodes in the published RoboProbe release [61]. The reference uses end-effector commands, including `move_eef`; Auriga uses the action-specification interface analyzed above. Both configurations use GPT-6 Astra. The comparison aligns the task instruction, evaluation seed, and initial scenes, and uses the released RoboProbe records rather than new baseline runs. For Auriga, the recorded configuration uses reasoning effort high, at most 170 model responses, and the task's 400-native-step limit. Tab. 15 compares task completion, model interaction, and physical execution.

Table 15 Task outcomes and execution profiles on 15 matched standard-variant T-block exchange scenes.

	Task completion			Model calls / ep.		Physical execution		
System	Successes	SR (%)	Exclusive	Mean	Median	Batches / ep.	Steps /ep.	Steps / batch
RoboProbe	3/15	20.0	0	36.6	38	33.4	365.1	10.9
Auriga	14/15	**93.3**	**11**	25.4	24	3.1	336.2	107.3

RoboProbe denotes the official GPT-6 Astra reference harness [61]. SR: success rate; Exclusive: scenes solved only by that system. Per-episode values are means over all 15 outcomes unless marked median. Batches count action invocations that advance the environment, including early stops; steps are native actions, and steps/batch is their aggregate ratio. Model calls count RoboProbe LLM requests and Auriga completed responses.

The gain extends across matched scenes. Auriga completes 14 of the 15 scenes (93.3%), compared with 3 of 15 (20.0%) for RoboProbe, a 73.3-percentage-point increase in success rate. The paired outcomes identify where this difference occurs: Auriga succeeds on 11 scenes where RoboProbe fails, while RoboProbe succeeds on no scene where Auriga fails. Three scenes are solved by both configurations, and one by neither. Each success requires both blocks to reach the opposite initial pose, including orientation. The additional successes therefore represent completion of the coupled placement and orientation requirements across multiple initial arrangements.

Higher completion with lower model-call counts. Mean model calls per episode are 36.6 for RoboProbe and 25.4 for Auriga, with medians of 38 and 24. The 73.3-percentage-point increase in success rate thus accompanies a lower average frequency of model interaction. For pose exchange, each decision must connect a remembered destination to the object's current pose and the remaining transfer. The state and correction records above show how Auriga makes these decisions actionable: the model preserves object goals, revises the route or responsible arm, and submits the resulting stages as an executable operation.

Longer operations between execution returns. Across the 15 episodes, RoboProbe records 501 physical execution batches and Auriga records 47, a 90.6% reduction in execution invocations. Mean native steps per episode remain at 365.1 and 336.2, respectively, while the average executed batch spans 10.9 versus 107.3 native steps. These counts locate the change in execution granularity: a similar number of native steps is organized into substantially fewer execution calls. This pattern is consistent with the multi-stage operations in Tab. 13, where approach, grasp, transport, and placement can be specified together before execution. For pose exchange, this allows the order of access to the shared workspace and the timing of release to be expressed within an operation. Observations and execution receipts then provide the evidence for revising the next operation when an object slips or a placement needs correction.

The process evidence explains how the interface is used. The state trace in Tab. 12 shows preservation of a completed placement while the remaining transfer changes arms. The correction in Tab. 14 shows a remembered orientation becoming an explicit, previewed operation within the remaining motion budget. Together, these records connect the task-level result to concrete model decisions: maintaining object goals, revising beliefs from physical evidence, and reorganizing the actions that remain. The quantitative comparison

evaluates the resulting harness configurations, while the execution records reveal how Auriga's task-level objectives are carried into physical operations.

11 Case Studies

The pose-exchange analysis in Sec. 10 distinguishes a persistent task objective from the physical procedure used to achieve it. The following cases examine how that distinction shapes individual decisions in sequence reproduction, conveyor picking, stacking, packing, and insertion. For each case, we connect the task state to the new evidence, the resulting action revision, and the evaluator outcome. The first two groups examine recovery, the third follows geometric estimates into action parameters, and the last traces a correction sequence that exhausts its motion budget. Each group ends with episodes from RoboCasa365 (Sec. 8.2). The figures summarize the visible transitions; the accompanying analysis draws on the task plans and execution records.

11.1 Grasp Failure and Re-Grasp after Slippage

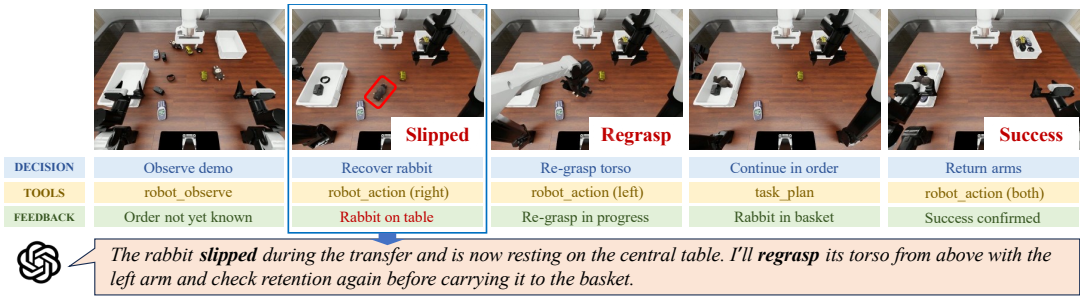
Grasp acquisition, retention during transport, and support after release are separate conditions. A failed acquisition calls for a new contact geometry; loss during transport also requires reconsidering how the object is moved; stable placement must be checked after the gripper ceases to support it. The RoboDojo and RoboCasa365 cases in Figs. 14 and 15 show how the model uses these distinctions to decide what to revise.

Sequence reproduction: keeping an unresolved placement in its original position in the order. The benchmark-provided observation sequence specifies wristwatch, black rectangular device, rabbit, phone, and gold vehicle. The model records this order before starting and verifies the first two releases. The rabbit is outside the left arm's planned reach, so the model uses the right arm to transfer it to the central table for a left-arm pickup (Fig. 14a). The rabbit slips during that transfer, and a subsequent placement leaves it outside the basket's far rim. The task plan continues to designate the rabbit as the third object; neither event advances the sequence to the phone.

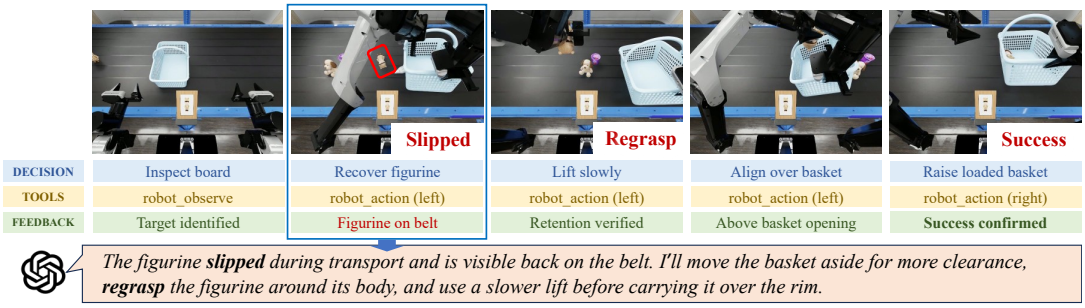
Recovery then changes more than the pickup position. A head grasp encounters the rim, further descents return `tracking_error`, and the wrist image shows the fingertips catching the rear edge of the head. The model changes the approach, tests a short lift, and ultimately turns the jaws across the torso after the rabbit has shifted into a region with more clearance. Only after the rabbit is visibly inside the basket does the plan advance to the phone and vehicle. The episode ends in task success. The important state transition is the completion of the third placement: physical retries can occur beneath that transition without changing the ordering constraint or repeating the two placements already completed.

Conveyor picking: separating acquisition, retention, and destination access. The model must match a figurine to the board image, place it inside the basket, and lift the basket by more than 8 cm. The moving conveyor first invalidates a basket pickup location, prompting a switch to right-arm interception. For the figurine, an approach contacts the hat before closure; a later empty closure leaves the body exposed. A torso grasp then acquires the object, but the figurine slips during transport (Fig. 14b). The model consequently changes both the grasp and the transfer conditions: it moves the basket aside, re-grasps across the torso, and verifies retention during a slower lift.

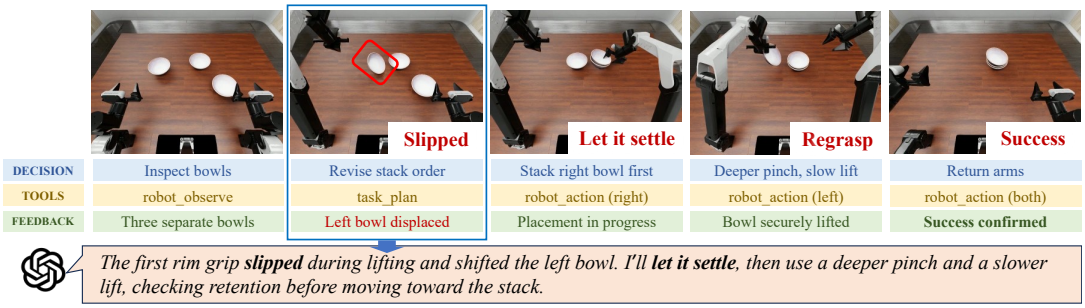
Once the figurine is retained, the remaining obstacle is the destination. Moving the basket underneath it causes contact with the image board and tilts the opening. The model keeps the figurine held, moves the basket upward and away from the board, and then completes the release. A later check against the initial image revises the height judgment because the earlier estimate followed a rim on a tilted basket. The loaded basket is raised farther before the evaluator signals success. These successive revisions address distinct predicates (target acquired, target retained, opening accessible, and required basket elevation), so progress on one predicate can be preserved while another is corrected.



(a) Sequence reproduction (`imitate_sorting_sequence`).



(b) Conveyor picking (`pick_from_conveyor_by_image`).

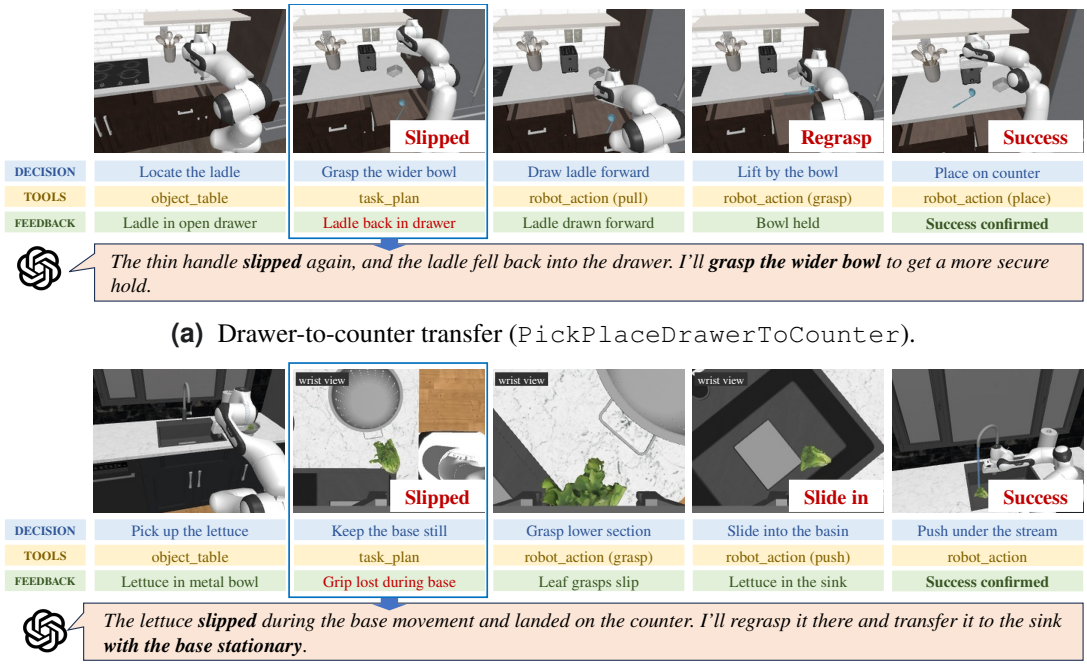


(c) Bowl stacking (`stack_bowls`).

Figure 14 Recovery from grasp failure and slippage. Each row follows one episode from its initial scene through the grasp disruption and corrective actions to successful completion. The model changes the grasped region for the rabbit and figurine, and uses a deeper pinch and slower lift for the bowl. Decision annotations condense the recorded model explanations; task success is confirmed by the evaluator.

Bowl stacking: changing the assembly order after a failed lift. The initial plan uses the rear middle bowl as the base, adds the left bowl, and then adds the right bowl. The first rim grip slips and displaces the left bowl. The revised plan lets that bowl settle while the robot approaches the right bowl, using a deeper pinch and slower lift (Fig. 14c). After the right bowl is nested into the base and remains supported after release, the model re-localizes the displaced left bowl and adds it last. Thus recovery changes the assembly order as well as the grasp.

This reordering is possible because the task constrains the final stack rather than the order in which bowls enter it. It also gives the displaced bowl time to settle before its next localization. The plan checks co-motion after lifting and upright support after release and withdrawal; the final return supplies another observation of whether the stack persists without gripper support. The evaluator confirms successful stacking. The distinction from sequence reproduction is consequential: the bowl order can change to simplify execution, whereas the observed object order must remain fixed.



(a) Drawer-to-counter transfer (`PickPlaceDrawerToCounter`).

(b) Lettuce washing (`WashLettuce`).

Figure 15 Recovery from grasp failure and slippage in RoboCasa365. Each row follows one episode from its initial scene through the slip and corrective actions to successful completion. The model moves the ladle grasp from the handle to the bowl, and keeps the base still before sliding the lettuce into the sink. Decision annotations condense the recorded model explanations; task success is confirmed by the evaluator.

Drawer-to-counter transfer: shifting contact from a thin handle to the wider bowl. The robot must take a ladle from an open drawer and place it on the counter, where the single task predicate tests only whether the ladle rests on the counter and remains false until final placement. An initial grasp on the thin handle slips during the lift, and after a lower re-grasp on a thicker section of the handle, the ladle slips again during mobile-base motion and falls back into the drawer (Fig. 15a). Both losses are reported by the execution receipts rather than the task predicate, and after the second slip the object table no longer finds the ladle in the wrist-camera view.

The model revises its task plan to grasp the ladle's wider bowl instead, though roughly ten attempts are blocked by the counter overhang or miss inside the drawer while it pulls the drawer out farther. It then uses the handle to draw the ladle forward within the drawer and grasps the bowl once clear of the overhang. Transferring the ladle by its bowl and releasing it on the counter turns the placement predicate true, and the evaluator confirms task success. This revision corresponds to the acquisition condition at the start of Sec. 11.1: when a thin handle fails to sustain lift and base transport, recovery changes the contact geometry to a wider region of the object.

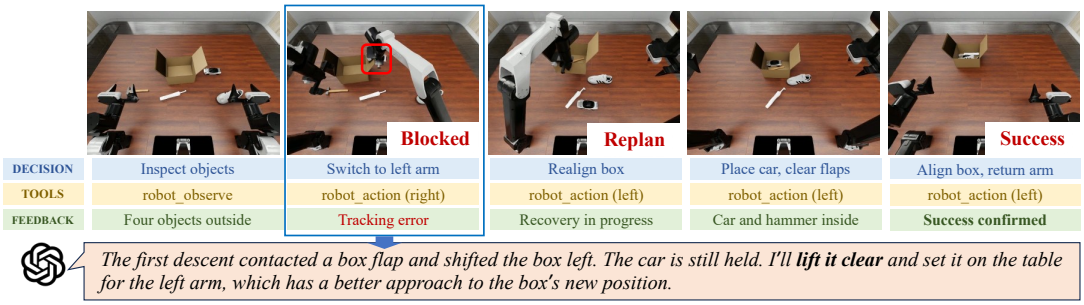
Lettuce washing: revising how the object is moved after transport slippage. The task requires placing a head of lettuce into the sink and rinsing it under running water, with a single predicate that turns true only after washing is complete. After the initial grasp, moving the mobile base causes the lettuce to slip onto the counter (Fig. 15b), as reported by the execution receipt and confirmed when the object table locates the lettuce in the wrist-camera view.

The model first revises the transfer by keeping the mobile base stationary, but three re-grasp attempts are blocked or slip from the upper leaves. It then turns to a narrower lower section and, with the lettuce beside the sink, slides it across the counter into the basin. It then turns on the faucet and pushes the lettuce under the

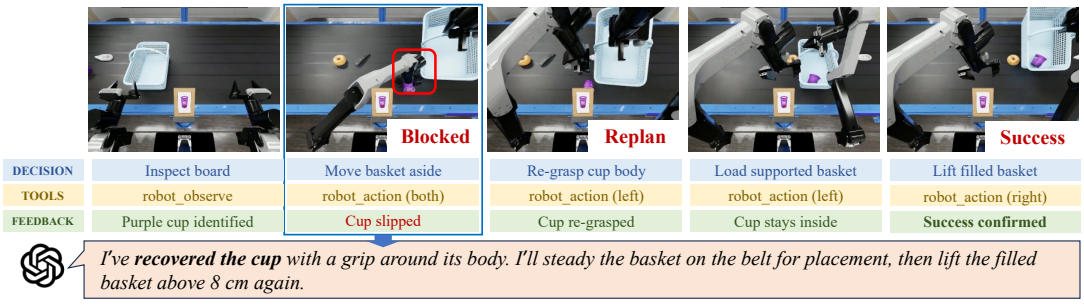
stream until the washing predicate passes and the evaluator confirms task success. This progression reflects the transport-retention condition in Sec. 11.1: loss during transport leads the model to change how the object is moved, first holding the base still and then replacing a lifted transfer with a supported slide.

11.2 Replanning after Contact and Execution Disruptions

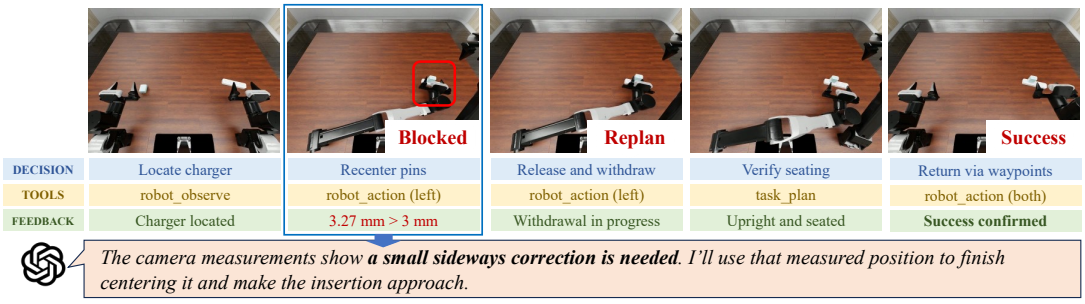
A new observation can invalidate a target coordinate, an approach route, or the ordering of operations. These require different scopes of revision. The RoboDojo and RoboCasa365 cases in Figs. 16 and 17 follow the model from a measured discrepancy to a change in the procedure, while retaining the task's final constraints.



(a) Object packing (`pack_objects_into_box`).



(b) Conveyor picking (`pick_from_conveyor_by_image`).



(c) Charger insertion (`plug_in_charger`).

Figure 16 Replanning after contact and execution disruptions. Each row shows the initial scene, the disrupted motion, subsequent adjustments, and successful completion. The model changes the arm used to approach the box, supports the basket during cup placement, and corrects the charger's alignment before insertion. Decision annotations condense the recorded model explanations; task success is confirmed by the evaluator.

Object packing: restoring a usable destination before resuming transfers. The task requires a car, shoe, hammer, and toothbrush inside the box with their fronts facing left. The first car descent returns `tracking_error`; the recorded vertical residual is about 8.2 cm, and the new image shows that contact with a flap has shifted the box. The car remains held, so the model lifts it clear and sets it on the table for transfer to the left arm, which can better approach the new box position (Fig. 16a).

The subsequent records show why changing arms alone is insufficient. The box continues to move when the wrists pass near its flaps, and the model temporarily sets down the held shoe while clearing the arms and restoring the box to the central workspace. The revised plan changes the approach geometry for later objects: keep wrists above the flaps, use short vertical releases, and withdraw vertically before lateral travel. For the toothbrush, the model further adjusts the wrist angle to reach over the front rim while keeping the brush end facing left. The episode completes all four placements. The central revision is to treat the receiving box as part of the state that must be restored, rather than repeatedly targeting coordinates measured before contact.

Conveyor picking: using support to decouple placement from basket motion. The cup initially remains held while the other arm raises the basket. During approach, the basket swings and the cup shifts in the left grip; a wrist-orientation change then loses the cup. After recovering it with a body grasp, the model preserves that wrist orientation and changes the basket procedure (Fig. 16b). The revised plan lowers the basket onto the belt, loads the cup, and then lifts the filled basket above the required height.

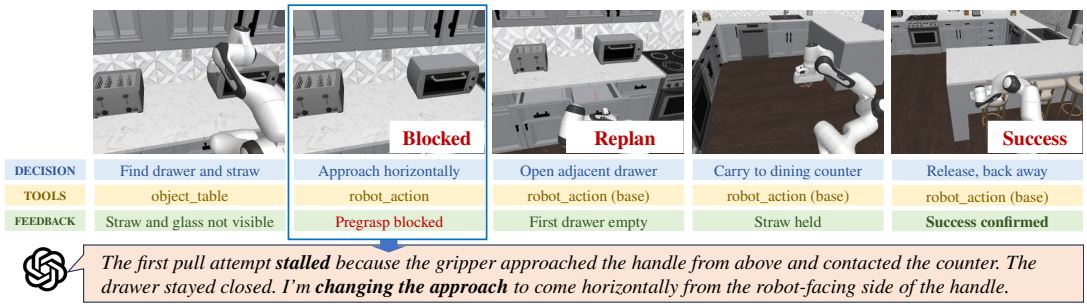
Supporting the basket changes the manipulation problem. Insertion can now be organized around a supported opening, while lifting becomes a separate operation on the loaded container. When a side adjustment loses the handle grip, the plan retains the cup grasp, clears the right arm for placement, and postpones handle acquisition until the cup has been released. The model then verifies that the cup stays inside after left-arm withdrawal, re-grasps the handle from above, and lifts the basket to complete the task. This sequence preserves the final object-in-basket and elevation requirements while removing the need to maintain a suspended basket throughout insertion.

Charger insertion: revising only the part of the procedure contradicted by evidence. The initial plan allows a table-supported handover if the left arm cannot reach the socket. A preview establishes a direct route, so the model keeps the charger in the left hand and moves the right arm outward to clear that route. Near the socket, the `clear_pins` clearance lift returns `tracking_error`: the position error is 3.27 mm, predominantly vertical, exceeding the stage's tight 3 mm tolerance. The charger remains held, and camera measurements identify a small lateral alignment correction (Fig. 16c). The next action uses the returned position to center the pins and complete the insertion, followed by release and a check that the charger stays seated.

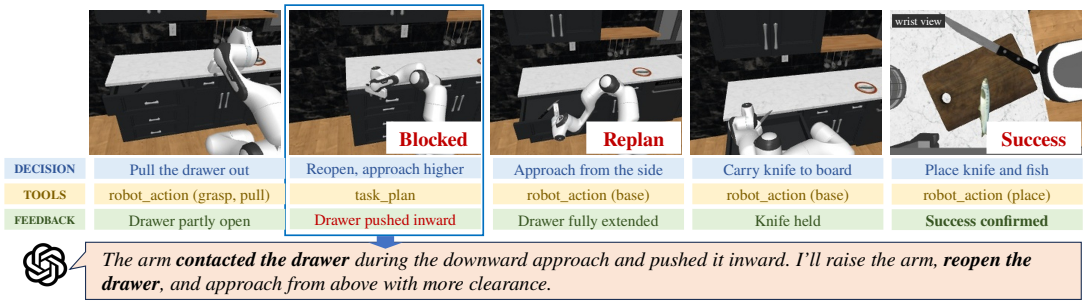
The later return-path failure calls for a different revision. Because seating remains visually supported, the model leaves the inserted charger undisturbed and splits the right-arm return into shorter movements through clear intermediate poses. The evaluator then reports success. This separates a local alignment correction from a route-planning problem: one changes the target relative to the socket, while the other changes how the arm reaches its home pose. Neither requires repeating the already verified insertion.

Straw delivery: matching the scope of revision to the invalidated assumption. The task requires taking a straw from a drawer to a glass cup on the dining counter; initially, the object table locates the drawer handles while neither the straw nor the cup is visible. Repeated attempts to open the first drawer return execution receipts for a blocked or stalled approach (Fig. 17a), as approaching the handle from above brings the gripper against the counter. The model switches to a horizontal approach from the robot-facing side of the handle, backs the mobile base away for clearance, and writes that direction into the grasp stage before pulling the drawer open with the base.

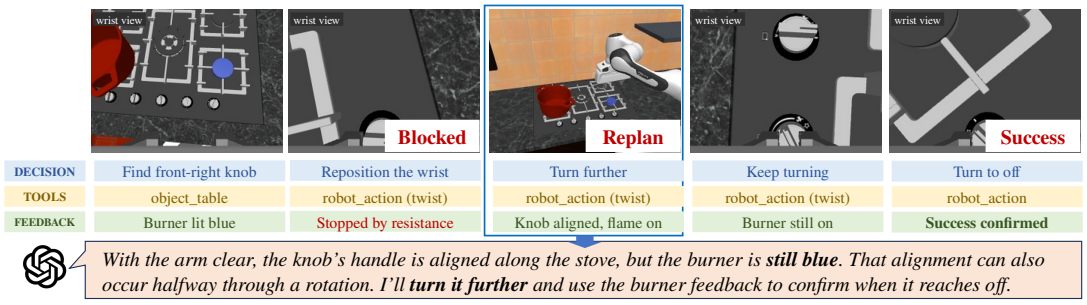
Inspecting the opened drawer reveals a different discrepancy when the object table finds no straw inside. The task plan records that this drawer is empty while the adjacent front drawer remains unchecked, and the robot opens that second drawer using the same horizontal handle approach and base pull. After grasping the straw and driving around the counter end, it rights the glass after a missed first insertion, places the straw inside to satisfy the straw-in-cup predicate, and backs away as the evaluator confirms success. As in charger insertion, the two disruptions call for different scopes of revision: a blocked grasp changes the approach route, whereas an empty drawer changes the target container without altering the delivery objective.



(a) Straw delivery (`DeliverStraw`).



(b) Cutting-station setup (`SetUpCuttingStation`).



(c) Turning off a burner (`TurnOffStove`).

Figure 17 Replanning after contact and execution disruptions in RoboCasa365. Each row shows the initial scene, the disrupted motion, subsequent adjustments, and successful completion. The model changes the approach to the drawer handle and then the drawer it opens, reopens the drawer that the arm pushed inward and approaches it from the side, and keeps turning the knob while the burner is still lit. Decision annotations condense the recorded model explanations; task success is confirmed by the evaluator.

Cutting-station setup: restoring a displaced container before revising the approach. The task requires taking a knife from a drawer and a fish from a plate and placing both onto a cutting board. When the arm reaches downward for the knife, the execution receipt reports a blocked approach, and the camera view shows that arm contact has pushed the partly open drawer inward (Fig. 17b). The model initially plans to raise the arm, reopen the drawer, and approach from above with more clearance.

Across roughly ten subsequent attempts, however, passing the arm over the drawer front repeatedly pushes the drawer inward and leaves grasps blocked or empty. The model then pulls the drawer fully open, withdraws the gripper clear of the handle before turning the wrist to approach from the side, and lowers its grasp slightly to place the finger pads around the knife handle. Delivering the knife satisfies the knife-on-board predicate, and after the fish is re-grasped following a slip during base motion, the evaluator signals task success during the final placement onto the board. As in object packing, the central revision treats the displaced drawer as state that must be restored and paired with a non-colliding approach route, rather than targeting coordinates

measured before contact.

Turning off a burner: updating wrist posture and target rotation as each assumption is contradicted. The task is to turn off the front-right burner, where the burner-on predicate remains true across all recorded steps while the flame stays lit. After roughly five grasp attempts miss the knob, stall on approach, or contact the stove surface, a secure grip turns only a few degrees before the execution receipt reports that rotation was stopped by resistance (Fig. 17c). Because the grip itself held, the model keeps the grasp target and releases only to reposition the wrist and re-grasp with room to turn.

After the next turn clears the arm, the camera view shows the knob handle aligned along the stove, a pose that could otherwise suggest the off position. Because the burner remains visibly blue in the image, consistent with the burner-on predicate remaining true, the model notes that the same alignment can occur halfway through a rotation. It commands a further turn guided by the burner state, during which the evaluator signals task success. Each revision addresses only the part of the procedure contradicted by the latest observation: mechanical resistance changes the wrist posture, whereas persistent flame invalidates the apparent target angle of the knob.

11.3 Geometry-Guided Manipulation with VGGT/SAM

The packing comparison in Fig. 18 follows two recorded harness configurations on the same randomized task scene, with VGGT/SAM geometric augmentation enabled in the lower row. The task requires all four objects inside the box with their fronts facing left. The augmented execution uses two distinct geometric outputs: an object-table orientation estimate for the receiving box and a queried 3D point for grasping the hammer. Tracing these outputs into the action specification explains how the additional geometry becomes useful. In RoboCasa365, Fig. 19 compares two configurations on the same cabinet-loading scene, with the SAM object table enabled in the lower row.

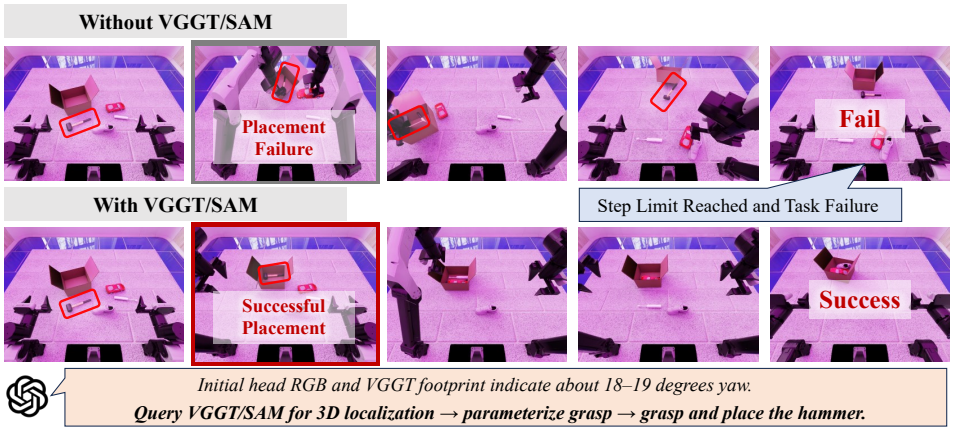


Figure 18 VGGT/SAM-assisted object packing (`pack_objects_into_box`). Two recorded configurations of the same randomized task scene. Top: the run without VGGT/SAM ends at the step limit without completing the task. Bottom: the model uses estimated box orientation and queried 3D geometry to guide alignment and grasping, and successfully packs all four objects. The rows show the initial scene, key intermediate states, and final outcome; annotations contain a recorded planning excerpt and a summary of the perception-to-action sequence.

Container orientation changes the preparation plan. The initial object table assigns the box a yaw of about 18 degrees. Together with the head image, this leads the task plan to record an approximately 18–19-degree misalignment and to align the box before packing. The model grasps the front wall near its rim; its lift and turn stages change the requested yaw from 109 to 90 degrees. An observation pause after the rim grasp provides an intermediate check before the turn. After release and withdrawal, the task ledger records that

the front edge appears horizontal. The orientation estimate therefore affects the order of work: establish the receiving geometry first, then use that geometry as the reference for successive placements.

From estimated height to grasp position. For the hammer, the model queries pixel $(267, 274)$ in the head image through `geometry_query`, using `pixel_plane` with `plane_source` set to `vggt`. The response uses a SAM point mask and a VGGT-derived estimated center-height plane. Back-projection through the calibrated camera produces a 3D point, which the model uses to set a nearby finger-referenced grasp target:

$$\mathbf{p}_{\text{query}} \simeq (-0.1051,\ -0.1827,\ 0.7795)\ \text{m}, \qquad \mathbf{p}_{\text{grasp}} = (-0.105,\ -0.183,\ 0.778)\ \text{m}.$$

The action retains the estimated horizontal location and places the grasp slightly lower, while separately specifying the wrist orientation and gripper closure. The previewed sequence is then executed, and its receipt records completion of the commanded grasp, lift, placement, and release stages. This provides a direct connection between a perception query and the parameters of a completed transfer.

The two estimates resolve different uncertainties. Box orientation concerns the reference frame for packing; hammer localization concerns where the fingers should make contact. The model also retains object-specific facing requirements in the task plan: hammer head, brush end, car front, and shoe toe toward the prescribed direction. It verifies object presence and orientation after placement, rather than using the numeric query result as a completion condition. The augmented run packs all four objects successfully; the execution without VGGT/SAM reaches the step limit without completing the task. The illustrated benefit is concrete: estimated geometry supports preparation of the destination and specification of contact, while execution observations determine whether those choices achieve the intended arrangement.

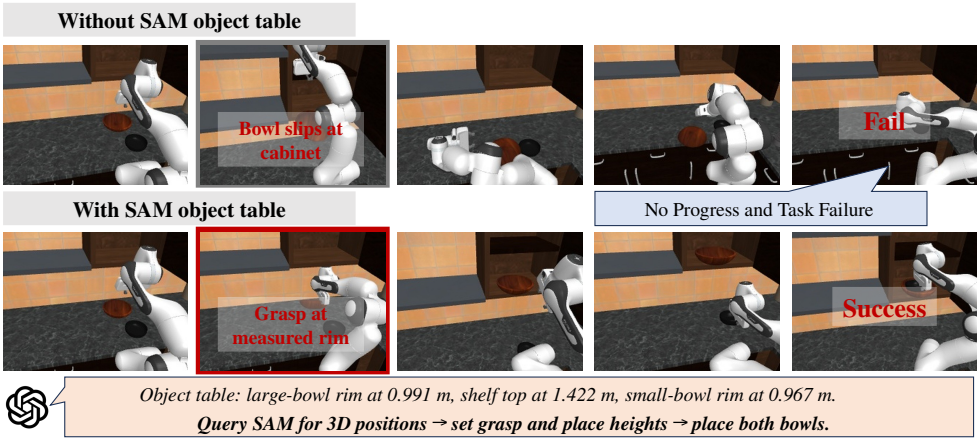


Figure 19 SAM-assisted bowl placement in RoboCasa365 (`StackBowlsCabinet`). Two recorded configurations of the same task scene. Top: the run without the SAM object table ends without progress after the large bowl slips at the cabinet opening and later falls to the floor. Bottom: the model uses object-table 3D measurements to set grasp and shelf-placement heights and places both bowls in the cabinet, with task success confirmed by the evaluator. The rows show the initial scene, key intermediate states, and final outcome; annotations report the measured heights and summarize the perception-to-action sequence.

Cabinet bowl placement: from object-table measurements to grasp and shelf parameters. In Fig. 19, the task requires placing a large bowl and a smaller bowl into an open cabinet, evaluated by six predicates testing cabinet containment, bowl-in-bowl contact, and gripper clearance for both bowls. With the SAM object table enabled, initial queries place the open shelf top at 1.422 m, the large-bowl rim at 0.991 m ($x = 5.431$ m), and the small-bowl rim at 0.967 m ($x = 5.502$ m). After two grasps near the large-bowl rim at 0.988 m and 0.985 m are blocked or slip, the model re-measures the rim at 0.990 m and grasps about 2 cm lower at

$0.968\ \mathrm{m}$ with $x = 5.433\ \mathrm{m}$, within $2\ \mathrm{mm}$ of the measured center. It plans the shelf placement at $1.477\ \mathrm{m}$, $5.5\ \mathrm{cm}$ above the measured shelf top, and seats the large bowl so its cabinet-containment predicate turns true.

The model then re-measures the placed large bowl at $x = 5.390\ \mathrm{m}$ to set the horizontal target for the second bowl. It grasps the small bowl at $0.948\ \mathrm{m}$, about $2\ \mathrm{cm}$ below its measured rim, at the measured $x = 5.502\ \mathrm{m}$, and moves it to $x = 5.39\ \mathrm{m}$ in the cabinet. Both cabinet-containment predicates turn true, and with both bowls in the cabinet, the evaluator confirms task success after 29 model responses. Without the SAM object table, the large bowl slips at the cabinet opening and settles upside down on the counter, where repeated side-grasp attempts on the inverted rim fail. After another slip at the cabinet approach, the bowl falls to the floor beyond the arm's reach during recovery, and the episode terminates without progress after 71 responses with all predicates false.

11.4 Failure after Repeated Corrections

The `stack_bowls` task requires an upright stack that remains supported after release and withdrawal. In Fig. 20, the model initially recovers from a slipped grasp and establishes a two-bowl stack. Adding the third bowl leaves it tilted, and repeated attempts to correct that tilt consume the remaining motion budget. Tab. 16 relates the observed state to the native-step counters returned during execution. In RoboCasa365, Fig. 21 and Tab. 17 trace a corresponding failure in kebab and sandwich heating, where repeated corrections instead exhaust the 100-request model budget.

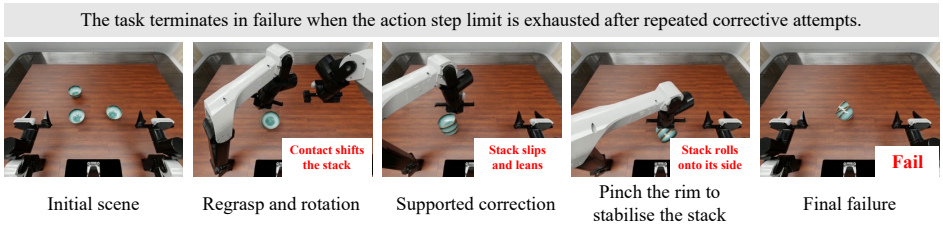


Figure 20 Failure after repeated corrections in `stack_bowls`. The frames show the initial scene, corrective manipulation, renewed slippage and rolling, and the final unstable stack. Re-grasping and supported corrections do not establish lasting stability before the 800-step native budget is exhausted; the evaluator records task failure with a Score of 15/100.

Table 16 Progress and remaining motion budget in the failed stacking episode. Step counts come from execution accounting; state descriptions condense the observation-linked model records. The final outcome is the evaluator's task-failure result.

Steps used	Steps left	State guiding the next decision
242	558	Two bowls remain nested after withdrawal; add the third.
440	360	The third bowl remains tilted after seating attempts; re-grasp and level it.
564	236	The nested stack was lifted and slipped; restore it using table support.
691	109	The stack has rolled onto its side again during the return; attempt a compact supported correction.
756	44	The rim pinch has not retained the stack; change contact to the raised base.
800	0	Native step limit reached; task failure, Score 15/100.

The correction changes the scope of the remaining problem. The plan at step 440 intends to lift and level the tilted top bowl. The subsequent model record instead reports that the grasp lifted the entire nested stack, which slipped during rotation. This observation changes the recovery target from seating one bowl to restoring support for the whole structure. It suggests that the correction did not sufficiently account for how the nested

bowls would move together. The already assembled configuration is therefore part of what the next action must preserve, not merely a destination for the top bowl.

Local completion does not ensure cumulative progress. The model recognizes the new state and tries to tip the stack upright against the table. That maneuver brings the bowls closer to upright, but the structure rolls again during the arm's return. Later supported pushes, a rim pinch, and a change of contact to the raised base continue to revise the strategy. Several motions receive `completed` receipts, yet the observation-linked states in Tab. 16 do not progress to stable support. The budget falls from 360 remaining steps before the re-grasp to 44 before the last contact change, leaving progressively less room for another acquisition, correction, release, and return.

The failure distinguishes detecting a discrepancy from selecting a correction that preserves progress. The records expose the instability and allow the task belief to be revised, but the chosen corrections do not establish a lasting upright stack before the evaluator terminates the episode. For recovery planning, the relevant question is consequently whether the next operation improves the supported object configuration after withdrawal, as well as whether the robot can complete the requested motion.

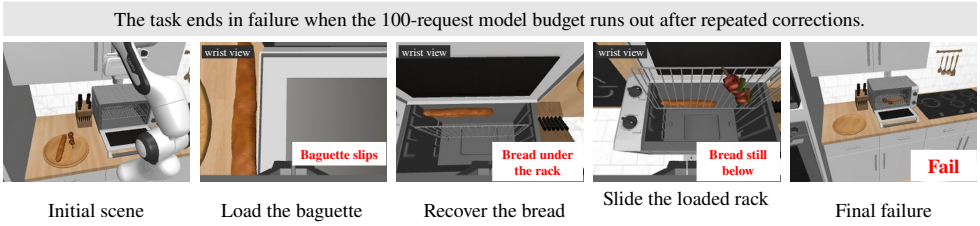


Figure 21 Failure after repeated corrections in RoboCasa365 (`HeatKebabSandwich`). The frames show the initial scene, the baguette slipping during loading, the baguette below the rack, the rack pulled out with the baguette still below it, and the final state. Corrections of the rack and the baguette do not bring the baguette into contact with the rack before the 100 model requests are used; the evaluator records task failure.

Table 17 Progress and remaining model budget in the failed RoboCasa365 episode (`HeatKebabSandwich`). Request counts are model responses out of the 100 allowed per episode; state descriptions condense the recorded model messages. The final outcome is the evaluator's task-failure result.

Requests used	Requests left	State guiding the next decision
14	86	The baguette has slipped onto the counter beside the open door; re-grasp it with an explicit approach direction.
41	59	The baguette is inside the oven but below the wire rack, so its rack-contact check is false; move the rack to expose the bread.
51	49	The kebab rests on the rack and its check passes; slide the loaded rack inward and recover the baguette from below.
63	37	The rack has not moved because the grip on its thin front rail was not secure; grasp the rail from a raised angle.
86	14	The rack has slid outward with the kebab, leaving the baguette behind it; lift the bread onto the rack.
97	3	The model reports the task unfinished: kebab on the rack, baguette below it, door open; the harness asks it to continue.
100	0	Model-request limit reached; task failure.

Kebab and sandwich heating: moving the rack expands the scope of the remaining problem. The task requires placing a kebab and a baguette into a toaster oven, closing the door, and turning the timer to start heating (Fig. 21 and Tab. 17). While the arm loads the baguette, the gripper contacts the oven entrance at request 9, and the baguette slips onto the counter between requests 12 and 14. The model repositions the base

and uses a horizontal grasp to return the bread to the entrance, but at request 41 the baguette lands inside the oven below the wire rack, leaving its rack-contact check false. To expose the bread underneath, the model decides to move the wire rack first, expanding the recovery from placing the baguette to manipulating the rack. As in the bowl-stacking episode where leveling one bowl displaces the stack, moving the rack becomes an obstacle of its own when grasps on the thin front rail fail to shift it between requests 63 and 86.

Local completion on the kebab and rack does not yield cumulative progress within the request budget. Meanwhile, the model places the kebab onto the rack at request 50, and the kebab-in-toaster predicate remains true for the rest of the episode (Tab. 17). When the loaded rack finally slides outward at request 86 with 14 requests remaining, the kebab stays on the rack, but the baguette still lies behind and below it without rack contact. At request 97, the model reports that the task remains unfinished with the kebab on the rack, the baguette below it, and the door open. The harness prompts it to continue at request 98, and the episode ends in evaluator-confirmed failure at the 100-request limit. Securing the kebab and eventually pulling out the rack thus represent local completions that do not resolve the baguette's rack-contact condition before the episode exhausts its 100-request model budget.

12 Conclusion

We have presented **Auriga**, a task-agnostic agent harness for zero-shot manipulation within a single episode, closing 68.3% of the planning–execution gap of vision-language models on RoboDojo (from 22.5% under the reference interface toward the 76.2% arm-free upper bound) without target-task exploration, demonstrations, or environment resets. In Auriga, the model makes all task decisions over an explicit task belief B_t and acts through direct tool calls or synthesized multi-step Python programs, while the harness supplies calibrated 3D perception, finger-referenced dual-arm control, and a single admission gateway $\mathcal{A}$ that previews trajectories while simulation is paused and verifies stage tolerances and grasp bounds during execution for in-place recovery. Developed across 13 cross-task capability iterations and frozen for evaluation, Auriga ranks first overall on the 42-task RoboDojo leaderboard with a 64.0 Score and 57.7% success rate, maintains first place under scene randomization ($G = 46.3$) while learned policies drop by up to 33.9%, and achieves a 73.2% success rate on RoboCasa365 mobile manipulation. Future work will extend this closed-loop execution architecture toward continuous force-sensitive manipulation, lower deliberation latency, and automated cross-task tool synthesis.

Author List

Contributors

Chengcheng Wang, Yuheng Shi, Siyu Xu, Yutong Wang, Jinjing Zhao, Chenghao Xia, Jinqi Liao, Jiankun Peng, Ligong Bi, Yuechuan Ma, Jiaqi Liu, Wenbo Wang, Zijian Wang, Yuemin Wu, Minjing Dong, Xiu Su

Project Leads

Jianyuan Guo, Tao Huang, Chengbin Du, Yunke Wang, Chang Xu

References

[1] Anthropic. System Card: Claude Opus 5.5. System card, September 2026. URL https://anthropic.com/claude-opus-5-5-system-card.

[2] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. SAM 3: Segment Anything with Concepts. arXiv preprint arXiv:2511.16719, 2026. URL https://arxiv.org/abs/2511.16719.

[3] Kaiyuan Chen, Shuangyu Xie, Letian Fu, Justin Yu, William Pacini, Sandeep Bajamahal, Hudson Kim, Jaimyn Drake, Daehwa Kim, Haoru Xue, Jonathan Francis, Christian Juette, Peter Schaldenbrand, Muhammet Yunus Seker, Ruwan Wickramarachchi, Uksang Yoo, Guanzhi Wang, Adithyavairavan Murali, Balakumar Sundaralingam, S. Shankar Sastry, Yuke Zhu, Linxi "Jim" Fan, Spencer Huang, and Ken Goldberg. GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks. arXiv preprint arXiv:2607.05369, 2026. URL https://arxiv.org/abs/2607.05369.

[4] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, Honghao Su, Zhiyang Dou, Kaixuan Wang, Dandan Zhang, Yunze Liu, Yan Qin, Qiwei Liang, Qiwei Wu, Zijian Lin, Wenwei Lin, Yuran Wang, Minghua He, Tianshu Wu, Ruihai Wu, Jingquan Zhou, Kai-Chong Lei, Haibao Yu, Yuanfeng Ji, Weiyang Jin, Guanyu Lin, Xiaofan Li, Qi Xiong, Renjing Xu, Zhongyu Li, Wenhao Chai, Enze Xie, Ziwei Wang, Yao Mu, Hao Dong, Wojciech Matusik, Mingyu Ding, Wenbo Ding, Ping Luo, and Masayoshi Tomizuka. RoboDojo: A Unified Sim-and-Real Benchmark for Comprehensive Evaluation of Generalist Robot Manipulation Policies. arXiv preprint arXiv:2607.04434, 2026. URL https://arxiv.org/abs/2607.04434.

[5] Dongzhou Cheng, Taoran Yi, Ye Fang, Xingwu Zhang, Fan Feng, Yixuan Li, Gengxiong Zhuang, Rongze Wang, Shuai Yang, Wei Song, Weizhi Xue, Minyan Wu, Jie Gui, Jiaqi Wang, and Tong Wu. In-Context Robot Learning with VLM Agents. arXiv preprint arXiv:2609.19138, 2026. URL https://arxiv.org/abs/2609.19138.

[6] DeepCybo Team, Yu Bin, Haipeng Cao, Zheng Chang, Kai Chen, Youning Chen, Kailin Deng, Yichao Du, Xiaotong Fu, Haoyang Ge, Yunlong Guo, Chenliu Hao, Jiyan He, Xuguo He, Yakun Hou, Kai Hu, Cong Huang, Tuopusen Huang, Yu Huang, Hong Li, Peize Li, Shijie Lian, Xiaopeng Lin, Yun Lin, Haibao Liu, Haochen Liu, Qiuzhi Liu, Shengcai Liu, Zhiqiang Liu, Tao Luo, Peng Ren, Shuo Ren, Chaoyi Ruan, Zhaolong Shen, Yukun Shi, Qiyuan Su, Yuxuan Tian, Yining Wang, Changti Wu, Hao Wu, Xueyin Xu, Ruoqi Yang, Zhaoyang Yang, Hang Yuan, Zhaoyang Zeng, Hanwen Zhang, Ruimeng Zhang, Yao Zhang, Yibo Zhang, Yuxiang Zhang, Zhirui Zhang, Ziyi Zhang, Zubin Zheng, and Zishen Zhuang. PhysBrain 1.5: From Vision-Language Models to Physical Foundation Models. arXiv preprint arXiv:2609.14973, 2026. URL https://arxiv.org/abs/2609.14973.

[7] Haoyuan Deng, Jiebin Liu, Tengxiao Zhang, Langning Yan, Hongye Cao, and Ziwei Wang. DynaHarness: A Dynamic Physical Harness for Self-Evolving Robot Agents. arXiv preprint arXiv:2609.40306, 2026. URL https://arxiv.org/abs/2609.40306.

[8] Yi Duan, Ying Liu, Zirui Tang, Haodong Chen, Jun Zhou, Yumou Liu, Bangrui Xu, Yukai Wu, Sidi Chen, Yuhan Zhou, Haoyu Wang, Xiaoyou Yu, Shaokun Han, Xuzhou Zhu, Le Zhou, Bolin Lu, Wei Zhou, Jiachen Liu, Nuozhou Fang, Jiaxin Tian, Ruoyu Chen, Yuxuan Li, Kai Zuo, Kaiyan Zhang, Qianyu Yang, Zijie Wang, Jiantao Qiu, Conghui He, Guoliang Li, Bowen Zhou, Zhiyuan Liu, Zhoufutu Wen, Jihua Kang, Xuanhe Zhou, and Fan Wu. The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement. arXiv preprint arXiv:2609.11873, 2026. URL https://arxiv.org/abs/2609.11873.

[9] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi "Jim" Fan. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. arXiv preprint arXiv:2603.22435, 2026. URL https://arxiv.org/abs/2603.22435.

[10] Galbot Team, Xuchuan Chen, Xiaoqian Cheng, Yu Deng, Lihe Ding, Shaocong Dong, Xiangjun Gao, Haozhe Jia, Zekai Li, Zhoujian Li, Yunrui Lian, Sikai Liang, Chenghuai Lin, Dairu Liu, Jiahang Liu, Qingtao Liu, Yuxuan Ma, Zekun Qi, Jiayi Su, He Wang, Ruochen Xu, Tianyu Xu, Xudong Xu, Zhe Xu, Mi Yan, Siming Yan, Li Yi, Ruixi Yu, Jinlu Zhang, Yintianrun Zhang, Zhikai Zhang, Zhizheng Zhang, Yixin Zheng, and Weiyi Zhu. Systematically Exploring the Capabilities of GPT-6 Astra as Embodied Policies. arXiv preprint arXiv:2609.38537, 2026. URL https://arxiv.org/abs/2609.38537.

[11] Hongcheng Gao, Alex Zhou, Jay Zhu, Zelin Zheng, Shijia Ge, Xiping Li, Di Wu, Yazhe Wang, Jianshu Zeng, Xuan Shangguan, Yongli Wang, Yexing Wan, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. HexaShell: A Thin Harness for Exploring the Limits of VLM Agents as Robot Coding Agents. arXiv preprint arXiv:2610.23479, 2026. URL https://arxiv.org/abs/2610.23479.

[12] Gemini Robotics Team, Abbas Abdolmaleki, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Ashwin Balakrishna, Nathan Batchelor, Alex Bewley, Jeff Bingham, Michael Bloesch, Konstantinos Bousmalis, Philemon Brakel, Anthony Brohan, Thomas Buschmann, Arunkumar Byravan, Serkan Cabi, Ken Caluwaerts, Federico Casarini, Christine Chan, Oscar Chang, London Chappellet-Volpini, Jose Enrique Chen, Xi Chen, Hao-Tien Lewis Chiang, Krzysztof Choromanski, Adrian Collister, David B. D'Ambrosio, Sudeep Dasari, Todor Davchev, Meet Kirankumar Dave, Coline Devin, Norman Di Palo, Tianli Ding, Carl Doersch, Adil Dostmohamed, Yilun Du, Debidatta Dwibedi, Sathish Thoppay Egambaram, Michael Elabd, Tom Erez, Xiaolin Fang, Claudio Fantacci, Cody Fong, Erik Frey, Chuyuan Fu, Ruiqi Gao, Marissa Giustina, Keerthana Gopalakrishnan, Laura Graesser, Oliver Groth, Agrim Gupta, Roland Hafner, Steven Hansen, Leonard Hasenclever, Sam Haves, Nicolas Heess, Brandon Hernaez, Alex Hofer, Jasmine Hsu, Lu Huang, Sandy H. Huang, Atil Iscen, Mithun George Jacob, Deepali Jain, Sally Jesmonth, Abhishek Jindal, Ryan Julian, Dmitry Kalashnikov, M. Emre Karagozler, Stefani Karp, Matija Kecman, J. Chase Kew, Donnie Kim, Frank Kim, Junkyung Kim, Thomas Kipf, Sean Kirmani, Ksenia Konyushkova, Li Yang Ku, Yuheng Kuang, Thomas Lampe, Antoine Laurens, Tuan Anh Le, Isabel Leal, Alex X. Lee, Tsang-Wei Edward Lee, Guy Lever, Jacky Liang, Li-Heng Lin, Fangchen Liu, Shangbang Long, Caden Lu, Sharath Maddineni, Anirudha Majumdar, Kevis-Kokitsi Maninis, Andrew Marmon, Sergio Martinez, Assaf Hurwitz Michaely, Niko Milonopoulos, Joss Moore, Robert Moreno, Michael Neunert, Francesco Nori, Joy Ortiz, Kenneth Oslund, Carolina Parada, Emilio Parisotto, Amaris Paryag, Acorn Pooley, Thomas Power, Alessio Quaglino, Haroon Qureshi, Rajkumar Vasudeva Raju, Helen Ran, Dushyant Rao, Kanishka Rao, Isaac Reid, David Rendleman, Krista Reymann, Miguel Rivas, Francesco Romano, Yulia Rubanova, Peter Pastor Sampedro, Pannag R Sanketi, Dhruv Shah, Mohit Sharma, Kathryn Shea, Mohit Shridhar, Charles Shu, Vikas Sindhwani, Sumeet Singh, Radu Soricut, Rachel Sterneck, Ian Storz, Razvan Surdulescu, Jie Tan, Jonathan Tompson, Saran Tunyasuvunakool, Jake Varley, Grace Vesom, Giulia Vezzani, Maria Bauza Villalonga, Oriol Vinyals, René Wagner, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Chengda Wu, Markus Wulfmeier, Fei Xia, Ted Xiao, Annie Xie, Jinyu Xie, Peng Xu, Sichun Xu, Ying Xu, Zhuo Xu, Jimmy Yan, Sherry Yang, Skye Yang, Yuxiang Yang, Hiu Hong Yu, Wenhao Yu, Wentao Yuan, Yuan Yuan, Jingwei Zhang, Tingnan Zhang, Zhiyuan Zhang, Allan Zhou, Guangyao Zhou, and Yuxiang Zhou. Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer. arXiv preprint arXiv:2510.03342, 2025. URL https://arxiv.org/abs/2510.03342.

[13] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4D World Action Modeling from Video Priors with Asynchronous Denoising. arXiv preprint arXiv:2604.26694, 2026. URL https://arxiv.org/abs/2604.26694.

[14] Yanjiang Guo, Haodong Yan, Zhide Zhong, Zhongru Zhang, Qingyuan Yang, Qingzhou Lu, Xiaoyu Chen, Yen-Jen Wang, Shuying Deng, Chenghan Yang, Puzhen Yuan, Chenxin Liu, Tun Ban, Xiang Zhu, Yichen Liu, Kun Feng, Haoang Li, and Jianyu Chen. Video Prediction Policy 2: Predict Better, Act Better. arXiv preprint arXiv:2610.10270, 2026. URL https://arxiv.org/abs/2610.10270.

[15] Wei Huang, Bohan Zhang, Chenzhi Liu, Isabella Liu, Shuai Yang, Weian Mao, Luozhou Wang, Yicheng Xiao, Weifeng Lin, Qixin Hu, Bryan Chu, Sifei Liu, Linxi Fan, Xiaojuan Qi, Song Han, and Yukang Chen. LongWAM: Scaling the Context of World-Action Models. arXiv preprint arXiv:2610.10528, 2026. URL https://arxiv.org/abs/2610.10528.

[16] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as Policy for Robotic Manipulation. arXiv preprint arXiv:2609.12541, 2026. URL https://arxiv.org/abs/2609.12541.

[17] Dongyoung Kim, Huiwon Jang, Myungkyu Koo, Suhyeok Jang, Taeyoung Kim, Beomjun Kim, Byungjun Yoon, Changsung Jang, Daewon Choi, Dongsu Han, Donguk Lee, Heeseung Kwon, Hojin Jeon, Jaehyun Kang, Jaekyoung Bae, Jihyuk Lee, Jimin Lee, John Won, Joonwoo Ahn, Junhyeong Park, Junyoung Sung, Kyungmin Lee, Minseong Han, Minsung Yoon, Sejune Joo, Seonil Son, Seungcheol Park, Seunggeun Cho, Seungjun Moon, Seungku Kim, Yonghoon Dong, Yongjin Cho, Youngchan Kim, Chang Hwan Kim, Dohyeon Kim, Heecheol Kim, Heewon Lee, Hensen Ahn, Hyungkyu Ryu, Hyunsoo Choi, Hyunsoo Shin, Jaeheon Jung, Jaewoo Kim, Jinwook Kim, Joochul Chang, Joonsoo Kim, Junghun Park, Jungwoo Park, Junho Cho, Junhyeok Park, Junwon Lee, Kangwook Lee, Kwanghoon Kim, Kyoungwhan Choe, Manoj Bhadu, Nayoung Oh, Sangjun Kim, Sangwoo Kim, Seunghoon Shim, Seunghyun Kim, Seungjun Lee, Seungyup Ka, Sungryol Yang, Wook Jung, Yashu Shukla, Yeonjae Lee, Yeonwoo Bae, and Jinwoo Shin. RLDX-1 Technical Report. arXiv preprint arXiv:2605.03269, 2026. URL https://arxiv.org/abs/2605.03269.

[18] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. arXiv preprint arXiv:2601.16163, 2026. URL https://arxiv.org/abs/2601.16163.

[19] Kehan Li, Bohan Hou, Minghao Zhu, Tianyi Zhang, Zesen Cheng, Zhikai Wang, Sicong Leng, Xin Li, Xiao Lin, Biying Yao, Minghua Zeng, Jiangpin Liu, Ronghao Dang, Jiayan Guo, Siteng Huang, Haoyu Zhao, Heng Ping, Yaxi Zhao, Tong Zhao, Kexiang Wang, Tong Lu, Shengke Xue, Jiahao Tang, Yulei Wang, Zejing Wang, Jianwei Gao, Shijian Lu, Chengju Liu, Jianfei Yang, Mingxiu Chen, and Deli Zhao. RynnBrain 1.1: Towards More Capable and Generalizable Embodied Foundation Model. arXiv preprint arXiv:2607.17977, 2026. URL https://arxiv.org/abs/2607.17977.

[20] Wenyi Li, Yizhe Chi, Youjie Zheng, Chunyu Liu, Xiaoqiu Wang, Mingju Gao, Zheng Jiang, Deyao Hong, Calvin Xiao, Kaisen Yang, and Qinhuai Na. EmbodiedEvo: Self-Evolving Coding Agent For Physical Intelligence. arXiv preprint arXiv:2610.18867, 2026. URL https://arxiv.org/abs/2610.18867.

[21] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as Policies: Language Model Programs for Embodied Control. arXiv preprint arXiv:2209.07753, 2023. URL https://arxiv.org/abs/2209.07753.

[22] Tao Lin, Yuxin Du, Haoxiao Wang, Zile Zhong, and Bo Zhao. FastHarness: Experience-Driven Review Scheduling for Vision-Language-Action Policies. arXiv preprint arXiv:2610.31427, 2026. URL https://arxiv.org/abs/2610.31427.

[23] Yihan Lin, Jiawei He, Shifeng Bao, Chen Zhao, Yang Li, Xiaobo Wang, Yan Wang, Cheng Chi, and Jing Zhang. JEPA-WAM: Learning Vision-Language-Action Policies with Joint-Embedding World Modeling. arXiv preprint arXiv:2608.09381, 2026. URL https://arxiv.org/abs/2608.09381.

[24] Fangcheng Liu, Yeqing Shen, Anda Cheng, Weishi Mi, Chao Tang, Chenyuan Liu, Yushun Xiang, Tingguang Li, Yong-Lu Li, and Yehui Tang. RoboICL: Embodied In-Context Learning with GPT-6 Astra. arXiv preprint arXiv:2609.34261, 2026. URL https://arxiv.org/abs/2609.34261.

[25] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, Xiao Liu, Dong Ke, Changxun Pan, Chenru Wu, Tailai Cheng, Xiaoshu Ren, Xinlei Zhang, Jianning Cui, Zijie Zhao, Haoyu Zhang, Kaiming Xu, Haodong Yang, Bowen Zhang, Jiahui Niu, Shaoting Zhu, Shiduo Zhang, and Hang Zhao. G0.5: One Autoregressive Stream for Robot Reasoning and Action. arXiv preprint arXiv:2608.11739, 2026. URL https://arxiv.org/abs/2608.11739.

[26] Runyu Lu, Yubo Wu, Ethan Kou, Haotian Lin, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Guanzhi Wang. ASPIRE: Agentic /Skills Discovery for Robotics. arXiv preprint arXiv:2607.00272, 2026. URL https://arxiv.org/abs/2607.00272.

[27] Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A Latent World-Action Model from Egocentric Videos. arXiv preprint arXiv:2605.00078, 2026. URL https://arxiv.org/abs/2605.00078.

[28] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui

Qiu, Zhaxizhuoma, Yuqiang Yang, Jiaqi Peng, Xueyuan Wei, Yangkun Zhu, Jiahao Jiang, Xing Gao, Hanqing Wang, Feng Yuan, Kailin Li, Xueyue Zhu, Tai Wang, Yan Ding, Jiangmiao Pang, Jia Zeng, Jingjing Zhang, Bowen Zhou, Yao Mu, Chunhua Shen, and Weinan Zhang. InternVLA-A1.5: Unifying Understanding, Latent Foresight, and Action for Compositional Generalization. arXiv preprint arXiv:2607.04988, 2026. URL https://arxiv.org/abs/2607.04988.

[29] Manifold AI Team. R^2SI: Recursive Robot Self-Improvement via a Bi-Level Agentic System with Agent–Model Co-Evolution. Technical report, Manifold AI, October 2026. URL https://r2si.worldscape.io/report.pdf.

[30] Xingyu Miao, Zizun Li, Baole Fang, Kaiwen Song, Tenghui Wang, Hanxue Zhang, Yating Wang, Xudong Li, Yuping He, Xueyuan Wei, Chao Gao, Xijie Yang, Yingxiang Xu, Kerui Ren, Wenqi Guo, Jianjun Zhou, Xinzhe Wang, Weiguang Zhao, Ni Yang, Zetao Cai, Yufei Xue, Hengjie Li, Zeyu He, Yuanzhen Zhou, Rong Fu, Jianyang Zhang, Siwei Cui, Fuxian Huang, Yunsong Zhou, Xing Gao, Yifei Yao, Qiaojun Yu, Kailin Li, Ming Zhou, Mu Huang, Xinyue Li, Wenze Cui, Bingqi Jiang, Xueyue Zhu, Junting Dong, Haoyu Guo, Tao Lu, Mulin Yu, Bowen Zhou, Bin Zhao, Tianfan Xue, Weinan Zhang, and Chunhua Shen. InternW0-Δ: A World Action Model Bridging Predictive Dynamics and Actions with 20K+ Hours of Open Data. arXiv preprint arXiv:2609.31394, 2026. URL https://arxiv.org/abs/2609.31394.

[31] MMLab & PhysicalRSI contributors. Physical RSI 1.0: Recursive Self-Harness for Scaling Embodied Skills. Project website and open-source software, 2026. URL https://mmlab.hk/research/PhysicalRSI. Preview release.

[32] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. arXiv preprint arXiv:2603.04356, 2026. URL https://arxiv.org/abs/2603.04356.

[33] Chang Nie, Zhe Liu, and Hesheng Wang. Teach and Grow: An Agent-Centered Architecture for General Robot Learning. arXiv preprint arXiv:2608.17209, 2026. URL https://arxiv.org/abs/2608.17209.

[34] OpenAI. GPT-6 Astra System Card. System card, September 2026. URL https://deploymentsafety.openai.com/gpt-6-astra.

[35] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi^*_{0.6}$: a VLA That Learns From Experience. arXiv preprint arXiv:2511.14759, 2025. URL https://arxiv.org/abs/2511.14759.

[36] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. arXiv preprint arXiv:2504.16054, 2025. URL https://arxiv.org/abs/2504.16054.

[37] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, Shihao Cao, Thomas Charbonnier, Vedant Choudhary, Foster Collins, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Maitrayee Dhaka, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachlan Groom, Haroun Habeeb, Hunter Hancock, Karol Hausman, Gashon Hussein, Victor Hwang, Brian Ichter, Connor Jacobsen, Szymon Jakubczak, Rowan Jen, Tim Jones, Gregg Kammerer, Ben Katz, Liyiming Ke, Mairbek Khadikov, Chandra

Kuchi, Marinda Lamb, Devin LeBlanc, Brendon LeCount, Sergey Levine, Xinyu Li, Adrian Li-Bell, Vladislav Lialin, Zhonglin Liang, Wallace Lim, Yao Lu, Enyu Luo, Vishnu Mano, Nandan Marwaha, Aikys Mongush, Liam Murphy, Suraj Nair, Tyler Patterson, Karl Pertsch, Allen Z. Ren, Gavin Schelske, Charvi Sharma, Baifeng Shi, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Jiaming Tang, Jimmy Tanner, Shalom Tekeste, Marcel Torne, Kyle Vedder, Quan Vuong, Anna Walling, Haohuan Wang, Jason Wang, XuDong Wang, Chris Whalen, Samuel Whitmore, Blake Williams, Charles Xu, Sukwon Yoo, Lili Yu, Wuming Zhang, Zhuoyang Zhang, and Ury Zhilinsky. $\pi_{0.7}$: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. arXiv preprint arXiv:2604.15483, 2026. URL https://arxiv.org/abs/2604.15483.

[38] Bince Qu, Wei Chen, and Bo Zhang. RoboHarness: A Simple Harness Outperforms VLA and World Action Models. arXiv preprint arXiv:2610.14018, 2026. URL https://arxiv.org/abs/2610.14018.

[39] Kerui Ren, Yingxiang Xu, Kaiwen Song, Linning Xu, Bo Dai, Mulin Yu, and Tao Lu. Real2Gym: Building Gyms from Videos, Bringing Skills to Robots. arXiv preprint arXiv:2609.37089, 2026. URL https://arxiv.org/abs/2609.37089.

[40] RoboCasa Team. RoboCasa365 leaderboard. https://robocasa.ai/leaderboard.html, 2026. Accessed 2026-10-30.

[41] RoboDojo. RoboDojo Leaderboard, 2026. URL https://robodojo-benchmark.com/leaderboard. Accessed 2026-10-31.

[42] RPent Authors. RPent leaderboard: RoboCasa365 Target50. https://rpent.readthedocs.io/en/latest/rst_source/leaderboard/index.html, 2026. Accessed 2026-10-30.

[43] SenseNova, Joint Research Team, Zhongang Cai, Ziang Cao, Tailai Chen, Han Chong Chua, Linjun Dai, Hanming Deng, Hokin Deng, Haiwen Diao, Chenjing Ding, Weichen Fan, Xiao Fu, Zerui Gong, Chenyang Gu, Zexin He, Zhi Hou, Bo Li, Qinyue Li, Quanzhou Li, Xiaowan Li, Zhichao Liao, Dahua Lin, Zilong Lin, Bo Liu, Chunxiao Liu, Pengfei Liu, Tianqi Liu, Ziwei Liu, Chen Change Loy, Lewei Lu, Jiapeng Luo, Yuexiao Ma, Liang Pan, Xingang Pan, Oscar Qian, Ziyi Shang, Yuyao Shao, Dawei Su, Dacheng Tao, Qingyi Tao, Quan Wang, Ruisi Wang, Xiaogang Wang, Ziyi Wang, Beichen Wen, Donghao Wu, Jiaxu Wu, Silei Wu, Xiaoming Wu, Zhonghua Wu, Hongcan Xiao, Weiye Xu, Atsushi Yamashita, Lei Yang, Zhe Yang, Baiqiao Yin, Wanqi Yin, Hanfei Zhao, Tianbo Zhong, Long Zhuo, and Jing Zuo. Has GPT-6-Astra Achieved Embodied Intelligence? arXiv preprint arXiv:2610.22471, 2026. URL https://arxiv.org/abs/2610.22471.

[44] Ruiyang Si, Jianxin Bi, Shunyu Yang, Rui Ni, Wenbo Huang, Qiang Wang, Shulong Jiang, Yintianrun Zhang, Feixiang Ruan, Kewei Zhang, Duomin Wang, Xiuyu Li, Haiwen Feng, Zhen Dong, and Daquan Zhou. Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens. arXiv preprint arXiv:2610.01939, 2026. URL https://arxiv.org/abs/2610.01939.

[45] Python Song, Zhixuan Liang, Kelsey Fu, Mengdi Wang, Junfeng Yang, and Shilong Liu. EmbodiedRSI: Active Continual Robot Learning Through Hypothesis-Guided Co-Evolution. arXiv preprint arXiv:2610.10498, 2026. URL https://arxiv.org/abs/2610.10498.

[46] Tencent Robotics X, HY Vision Team, Xumin Yu, Zuyan Liu, Ziyi Wang, He Zhang, Yongming Rao, Fangfu Liu, Yani Zhang, Ruowen Zhao, Oran Wang, Yves Liang, Haitao Lin, Minghui Wang, Yubo Dong, Kevin Cheng, Bolin Ni, Rui Huang, Han Hu, Zhengyou Zhang, Linus, and Shunyu Yao. HY-Embodied-0.5: Embodied Foundation Models for Real-World Agents. arXiv preprint arXiv:2604.07430, 2026. URL https://arxiv.org/abs/2604.07430.

[47] Jianyuan Wang, Minghao Chen, Shangzhan Zhang, Nikita Karaev, Johannes Schönberger, Patrick Labatut, Piotr Bojanowski, David Novotny, Andrea Vedaldi, and Christian Rupprecht. VGGT-Ω. arXiv preprint arXiv:2605.15195, 2026. URL https://arxiv.org/abs/2605.15195.

[48] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Yiqun Zhang, Zihan Wang, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-Evolving Embodied Agents via Skill-Harness Evolution. arXiv preprint arXiv:2608.11350, 2026. URL https://arxiv.org/abs/2608.11350.

[49] Xin Wang, Wenhao Wu, Menghao Zhang, Zhi Wang, Kun Shao, Jian Luan, Yang Li, Qing Li, Shangding Gu, Huichi Zhou, Shuqing Shi, Fei Ni, Shuo Lu, Weicheng Meng, Kang Li, Jin Wu, Kang Zhao, Shangmin Guo, Gen

Li, Yongqiang Tang, Zhizhong Zhang, Yuan Xie, and Heng Qu. HarnessPAI: An Evolving Harness for Physical AI. arXiv preprint arXiv:2609.29166, 2026. URL https://arxiv.org/abs/2609.29166.

[50] Xinyi Wang, Bangzheng Li, Hanyu Wang, Lin Zhang, Andrea Yaoyun Cui, Zhenggang Tang, Shyamal Buch, and Dejia Xu. SEAR: Self-Evolving Agentic Robotics. arXiv preprint arXiv:2610.18792, 2026. URL https://arxiv.org/abs/2610.18792.

[51] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining. arXiv preprint arXiv:2609.07398, 2026. URL https://arxiv.org/abs/2609.07398.

[52] Zhijie Wei, Ferris Tan, Jinghui Wang, and Hao Zhou. Scale and Selection: What Makes Automatic Harness Evolution Work for Visual-Interface Robot Agents. arXiv preprint arXiv:2609.39304, 2026. URL https://arxiv.org/abs/2609.39304.

[53] Wei Wu, Fangjing Wang, Fan Lu, He Sun, Shi Liu, Yunnan Wang, Yibin Yan, Yong Wang, Shuailei Ma, Xinyang Wang, Yibin Liu, Shuai Yang, Tianxiang Zhou, Kejia Zhang, Lei Zhou, Cheng Su, Nan Xue, Bin Tan, Han Zhang, Youchao Zhang, Fei Liao, Xing Zhu, Yujun Shen, and Kecheng Zheng. From Foundation to Application: Improving VLA Models in Practice. arXiv preprint arXiv:2607.06403, 2026. URL https://arxiv.org/abs/2607.06403.

[54] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian "Max" Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, Jimmy Wu, Guanzhi Wang, S. Shankar Sastry, Ken Goldberg, Linxi "Jim" Fan, Yuke Zhu, and Guanya Shi. ENPIRE: Agentic Robot Policy Self-Improvement in the Real World. arXiv preprint arXiv:2606.19980, 2026. URL https://arxiv.org/abs/2606.19980.

[55] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, Nan Sun, Qiao Sun, Runze Suo, Heyun Wang, Yunhong Wang, Rujie Wu, Caoyu Xia, Lina Zhang, Jack Zhao, Guoliang Chen, Wenlong Chen, Xinze He, Bin Li, Qing Li, Zhuorong Li, Heng Qu, Wenxuan Song, Diyun Xiang, Yifan Xie, Peiran Xu, Hangjun Ye, Wen Ye, Han Zhao, and Quanyun Zhou. Xiaomi-Robotics-1: Scaling Vision-Language-Action Models with over 100K Hours of Real-World Trajectories. arXiv preprint arXiv:2607.15330, 2026. URL https://arxiv.org/abs/2607.15330.

[56] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World Action Models are Zero-shot Policies. arXiv preprint arXiv:2602.15922, 2026. URL https://arxiv.org/abs/2602.15922.

[57] Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, Jie Zhang, Jingyang Fan, Gengze Zhou, Qihang Peng, Chenxu Lv, Xiaoyue Chen, An Yang, Fei Huang, Junyang Lin, Dayiheng Liu, Jingren Zhou, Chenfei Wu, and Xiong-Hui Chen. Qwen-RobotManip Technical Report: Alignment Unlocks Scale for Robotic Manipulation Foundation Models. arXiv preprint arXiv:2606.17846, 2026. URL https://arxiv.org/abs/2606.17846.

[58] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do World Action Models Need Test-time Future Imagination? arXiv preprint arXiv:2603.16666, 2026. URL https://arxiv.org/abs/2603.16666.

[59] Yifu Yuan, Yaoting Huang, Xianze Yao, Yutong Li, Shuoheng Zhang, Linqi Han, Pengyi Li, Jiangeng Sun, Wenting Jia, Zhao Zhang, Yuhao Liu, Ruihao Liao, Yucheng Hu, Qiyu Wu, Yuxiao Li, Zibin Dong, Fei Ni, Yan Zheng, Shuyang Gu, Yi Ma, Hongyao Tang, Han Hu, and Jianye Hao. Embodied-R1.5: Evolving Physical Intelligence via Embodied Foundation Models. arXiv preprint arXiv:2606.11324, 2026. URL https://arxiv.org/abs/2606.11324.

[60] Junyi Zhang, Jiaxin Ge, Hanjun Yoo, Letian Fu, Zihan Yang, Yaowei Liu, Raj Saravanan, Shaofeng Yin, Justin Yu,

Dantong Niu, Zirui Wang, Roei Herzig, Ken Goldberg, Yutong Bai, David M. Chan, Ion Stoica, Angjoo Kanazawa, Jiahui Lei, Haiwen Feng, and Trevor Darrell. Playful Agentic Robot Learning. arXiv preprint arXiv:2606.19419, 2026. URL https://arxiv.org/abs/2606.19419.

[61] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An Unexpected Robot Policy: Early Evaluations of GPT-6 Astra on RoboDojo and Beyond. arXiv preprint arXiv:2609.24170, 2026. URL https://arxiv.org/abs/2609.24170.

[62] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents. arXiv preprint arXiv:2607.08448, 2026. URL https://arxiv.org/abs/2607.08448.

[63] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyuan Zhan. X-VLA: Soft-Prompted Transformer as Scalable Cross-Embodiment Vision-Language-Action Model. arXiv preprint arXiv:2510.10274, 2025. URL https://arxiv.org/abs/2510.10274.

[64] Hai Zhou, Fan Yang, Zeyi Chen, Lejun Zhang, Chenyang Fu, Jianping Li, Jiayi Zhu, Xianyang Liu, Yingxuan Yang, Lili Meng, Lu Wang, Xinyang Han, Jiankai Sun, Weirui Ye, Guanchu Wang, Wanhua Li, Ming Zhou, Boyi Li, Ming Jin, Kun Shao, Muhao Chen, Guang Chen, Lei Ma, Joey Tianyi Zhou, Costas Spanos, Xia Hu, Mac Schwager, Weinan Zhang, Qian Feng, and Shangding Gu. Who Improves the Robot? Toward Physical Recursive Self-Improvement. arXiv preprint arXiv:2610.18842, 2026. URL https://arxiv.org/abs/2610.18842.

[65] Youwei Zhou, Bohan Hou, Yankai Fu, Yaoxu Lyu, Yifan Ye, Shilin Shan, Chen Guo, Jiaming Liu, Ran He, Caifeng Shan, Shanghang Zhang, Jianfei Yang, and Zhihe Lu. Large Multimodal Models for Direct Robot Control: GPT-6 as a Beginning. arXiv preprint arXiv:2610.28259, 2026. URL https://arxiv.org/abs/2610.28259.

A Benchmark Tasks

Tab. 18 lists the 42 tasks of RoboDojo by capability dimension (Sec. 8.1).

Table 18 The five capability dimensions of RoboDojo and their tasks.

Dimension	Tasks	What it covers	Task identifiers
Generalization	12	Manipulation across varied objects and layouts, including randomized scene variants	`stack_bowls`, `push_T`, `pack_objects_into_box`, `fold_clothes`, `hang_mugs`, `sweep_blocks`, `pour_liquid_into_cup`, `make_toast`, `arrange_largest_number`, `sort_nesting_dolls_by_size`, `store_laptop_and_headphones`, `stack_blocks`
Precision	8	Fine-grained manipulation under tight spatial constraints	`fasten_screws`, `plug_in_charger`, `insert_tubes`, `pour_balls_into_vase`, `play_Xylophone`, `deposit_coin`, `insert_key`, `build_tower`
Long-Horizon	8	Multi-step tasks that require completing several subgoals	`put_bottles_into_dustbin`, `fill_pen_holder`, `classify_objects`, `play_tic_tac_toe`, `fill_egg_holder`, `organize_table`, `make_kong`, `play_stacking_toy`
Memory	6	Tracking state, recalling a sequence, or matching after a delay	`cover_blocks`, `match_and_pick_from_conveyor`, `swap_blocks`, `swap_T`, `press_by_number`, `imitate_sorting_sequence`
Open	8	Open-ended tasks and manipulation conditioned on language or images	`align_blocks`, `general_pickup`, `stack_blocks_by_language`, `solve_equation`, `classify_objects_by_language`, `pick_from_conveyor_by_image`, `store_tools_in_toolbox`, `pour_by_language`

B Ablation Details

Tab. 19 lists the ten tasks of each ablation table in Sec. 9.5: five standard layouts per task (50 cases) in Tabs. 7 to 10, and 30 cases in Tab. 11.

Table 19 The ten tasks of each ablation table by dimension: five standard layouts per task (50 cases) in Tabs. 7 to 10, and 30 cases in Tab. 11.

Table	Tasks
Tab. 7	Generalization: `stack_blocks`, `stack_bowls`, `push_T`; Long-Horizon: `play_tic_tac_toe`; Memory: `cover_blocks`, `press_by_number`, `imitate_sorting_sequence`; Open: `align_blocks`, `stack_blocks_by_language`, `classify_objects_by_language`
Tab. 8	Generalization: `stack_bowls`, `sort_nesting_dolls_by_size`; Precision: `fasten_screws`, `insert_tubes`; Long-Horizon: `classify_objects`, `play_stacking_toy`; Memory: `cover_blocks`, `swap_T`; Open: `classify_objects_by_language`, `align_blocks`
Tab. 9	Generalization: `fold_clothes`, `arrange_largest_number`, `sort_nesting_dolls_by_size`; Precision: `fasten_screws`; Long-Horizon: `classify_objects`; Memory: `swap_T`, `cover_blocks`, `imitate_sorting_sequence`; Open: `pick_from_conveyor_by_image`, `classify_objects_by_language`
Tab. 10	Generalization: `fold_clothes`, `arrange_largest_number`; Precision: `fasten_screws`, `play_Xylophone`; Long-Horizon: `classify_objects`, `play_tic_tac_toe`; Memory: `press_by_number`, `cover_blocks`; Open: `classify_objects_by_language`, `align_blocks`
Tab. 11	Generalization: `sort_nesting_dolls_by_size`, `pack_objects_into_box`; Precision: `fasten_screws`; Long-Horizon: `play_stacking_toy`, `play_tic_tac_toe`, `classify_objects`; Memory: `press_by_number`, `imitate_sorting_sequence`; Open: `pick_from_conveyor_by_image`, `classify_objects_by_language`

C RoboCasa365 Results

Tab. 20 lists four entries of the official RoboCasa365 leaderboard [40], which are evaluated in the pretraining kitchens rather than in the target kitchens of Tab. 5; Xiaomi-Robotics-1 leads all four columns among these entries, with 57.4% Overall. Fig. 22 plots these four entries alongside Auriga, RPent, and RLDX-1 from Tab. 5. Fig. 23 gives the outcome of every task and episode of Auriga.

Table 20 Success rates (%) of four entries of the official RoboCasa365 leaderboard [40] on the 50 Target50 tasks, evaluated in the pretraining kitchens and ordered by ascending Overall SR. Overall is the mean over the 50 tasks.

Method	Type	Overall	Atomic-Seen	Composite-Seen	Composite-Unseen
π_0	VLA	14.8	34.6	6.1	1.1
$\pi_{0.5}$	VLA	16.9	39.6	7.1	1.2
WorldDreamer	WAM	35.3	66.3	26.7	9.0
Xiaomi-Robotics-1	VLA	57.4	80.2	57.1	32.1

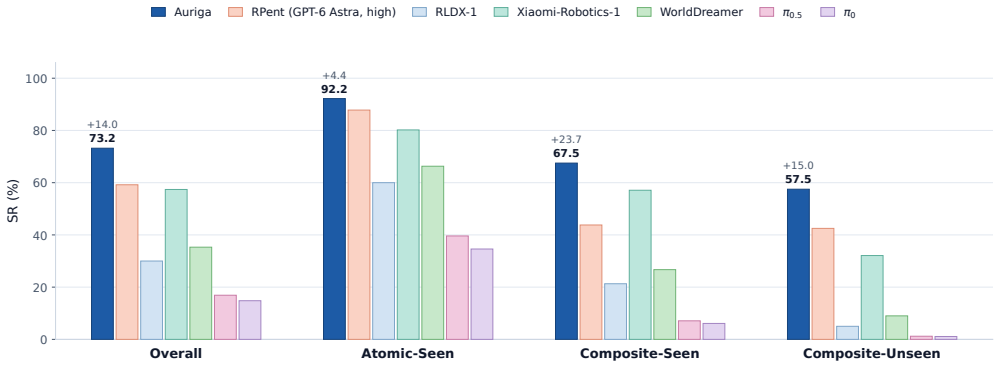


Figure 22 Success rates (%) on RoboCasa365 by group: Auriga, RPent, and RLDX-1 from Tab. 5 (target kitchens, 250 episodes) and the leaderboard entries of Tab. 20 (pretraining kitchens), with Auriga first. In each group, the bar of Auriga shows its rate and, above it, its difference from the highest other method of Tab. 5; another method's bar is labeled with its rate only where it reaches that of Auriga (Tabs. 5 and 20 list all values).

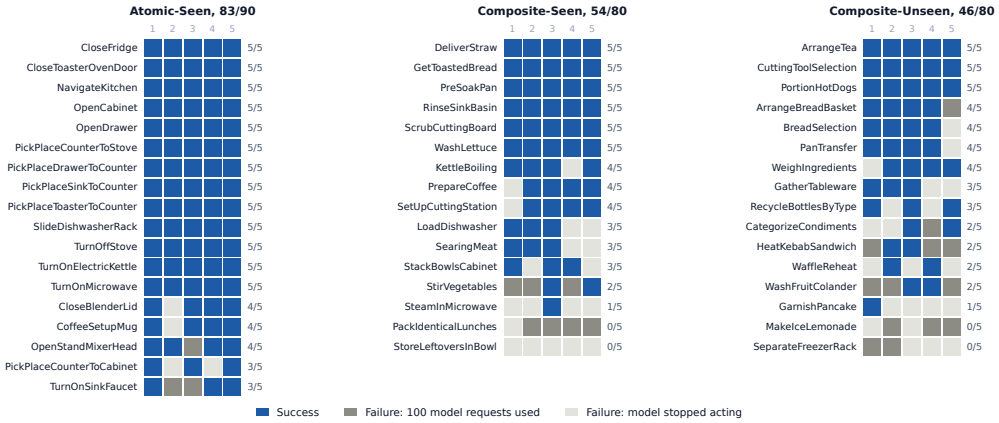


Figure 23 Outcome of every episode of Auriga on the 250 Target50 episodes, by task and seed: success, failure after the 100 model requests of the episode were used, and failure after the model stopped acting for several turns. Within each group, tasks are ordered by their number of successes.

D Additional Results

Tab. 21 lists the episode cost of Auriga by dimension corresponding to Fig. 11. Across the 6,299 episodes in Tab. 21, successful episodes (3,405 episodes) complete efficiently once the task subgoals are verified, averaging 30.3 model calls, 3.10 million input tokens, 11.5 minutes, and 65.6% of the native-step limit without ever exhausting the step budget. The remaining 2,894 episodes, which continue online recovery attempts until the budget expires, average 67.6 model calls, 8.61 million input tokens, 27.5 minutes, and 96.4% of the native-step limit, accounting for 65.5% of all model calls and 70.2% of all input tokens. Tab. 22 lists the SR and Score on the standard and random scenes of the 12 Generalization tasks together with the change between the two scene types (with the SR change and G plotted in Fig. 9), and Fig. 24 plots the Score and SR of the methods in Fig. 8 and of RoboProbe as bars.

Table 21 Cost of an Auriga episode by dimension across the 6,299 evaluated episodes: episodes, mean model calls, input tokens and their cached share, output tokens, the share of the native-step limit used, and wall-clock time.

Dimension	Episodes	Model calls	Input (M)	Cached (%)	Output (k)	Steps used (%)	Time (min)
Generalization	1,800	54.2	6.57	84.8	21.9	79.6	21.6
Precision	1,200	50.2	6.02	85.9	19.5	86.9	20.2
Long-Horizon	1,199	54.5	6.70	85.2	21.8	88.8	23.0
Memory	900	30.0	3.11	85.5	9.0	68.9	11.3
Open	1,200	40.7	4.67	86.1	15.4	71.8	14.9
All	6,299	47.5	5.63	85.4	18.3	79.7	18.8

Table 22 Generalization under scene randomization on the 12 Generalization tasks for the entries of Tab. 4: SR (%) and Score on the standard and random scenes, with the change (random − standard) and relative SR change. Above Auriga, entries are ordered by ascending composite generalization score G (Fig. 9). Bold: best entry per column.

	SR (%)				Score		
Method	Standard	Random	Change	Rel. change	Standard	Random	Change
PhysicalRSI	25.67	5.56	−20.1	−78%	31.57	11.04	−20.5
VPP2	46.44	12.56	−33.9	−73%	52.14	20.63	−31.5
Awomo-0.5	30.89	16.78	−14.1	−46%	37.70	22.92	−14.8
Rex-M1 Preview	34.78	17.33	−17.5	−50%	42.64	25.46	−17.2
GO-3 Nano Preview	43.00	24.33	−18.7	−43%	45.93	26.97	−19.0
Simate-beta	33.22	22.67	−10.6	−32%	40.54	29.63	−10.9
RoboProbe	32.67	28.33	−4.3	−13%	35.32	31.40	−3.9
HexaShell-Preview	38.00	31.00	−7.0	−18%	43.28	35.15	−8.1
WALL-OSS-1.0-lite	45.44	30.44	−15.0	−33%	53.44	39.88	−13.6
EmbodiedEvo	30.33	31.33	**+1.0**	**+3%**	34.55	35.88	**+1.3**
WorldScape-R^{2}SI	40.33	31.33	−9.0	−22%	46.62	39.28	−7.3
PyRUA-Lean	45.33	41.00	−4.3	−10%	50.47	46.07	−4.4
Auriga	**52.28**	**45.61**	−6.7	−13%	**56.22**	**49.58**	−6.6

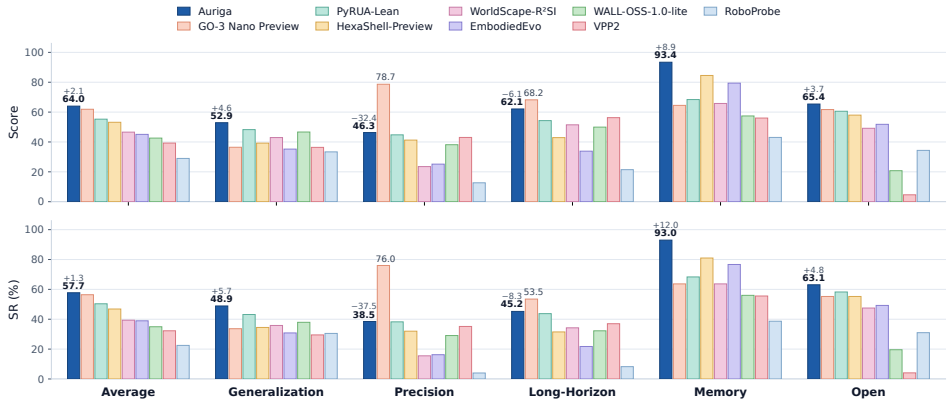


Figure 24 Score (top) and SR in % (bottom) of Auriga, the seven other methods of Fig. 8, and RoboProbe, on average and by dimension. Auriga is the first bar of each group, followed by the other methods in order of average Score. Above the bar of Auriga are its value and, higher up in grey, its difference from the highest of the other eight methods, whose value is given above its own bar where it is higher than Auriga.

E Per-Task Results

Tab. 23 lists the per-task Score and SR of Auriga alongside the per-task SR of the four other agent methods of Tab. 4 (PyRUA-Lean, HexaShell-Preview, EmbodiedEvo, and RoboProbe), and Fig. 25 compares Auriga with each of them task by task, in Score and in SR.

Table 23 Per-task results of Auriga (SR in % and Score) and per-task SR (%) of the four other agent methods in Tab. 4 (Sec. 8.1). Bold: highest SR of a task (none where all five are at 0.0).

	Auriga		SR (%) of the four other agent methods			
Task	SR	Score	PyRUA-Lean	HexaShell-Preview	EmbodiedEvo	RoboProbe
Generalization						
stack_bowls	**91.3**	92.5	70.0	86.0	56.0	62.0
push_T	86.0	86.0	86.0	**96.0**	86.0	60.0
pack_objects_into_box	14.7	27.3	**30.0**	16.0	0.0	0.0
fold_clothes	69.3	74.5	52.0	40.0	**82.0**	72.0
hang_mugs	4.0	19.5	**12.0**	4.0	0.0	0.0
sweep_blocks	**40.7**	40.7	6.0	8.0	2.0	4.0
pour_liquid_into_cup	7.3	7.3	4.0	2.0	**26.0**	4.0
make_toast	**26.0**	28.7	14.0	0.0	4.0	0.0
arrange_largest_number	**88.0**	90.9	74.0	22.0	32.0	60.0
sort_nesting_dolls_by_size	61.3	61.3	**70.0**	52.0	14.0	18.0
store_laptop_and_headphones	0.0	7.1	0.0	0.0	0.0	0.0
stack_blocks	98.0	98.3	**100.0**	88.0	68.0	86.0
Precision						
fasten_screws	**69.3**	80.7	68.0	18.0	38.0	16.0
plug_in_charger	20.7	20.7	16.0	**32.0**	6.0	0.0
insert_tubes	46.0	61.3	**50.0**	22.0	32.0	0.0
pour_balls_into_vase	38.0	38.0	66.0	**80.0**	22.0	4.0
play_Xylophone	**77.3**	77.3	62.0	58.0	10.0	8.0
deposit_coin	32.0	42.5	**40.0**	16.0	22.0	2.0
insert_key	0.0	14.8	0.0	0.0	0.0	0.0
build_tower	24.7	35.2	4.0	**30.0**	0.0	2.0
Long-Horizon						
put_bottles_into_dustbin	82.7	89.3	**88.0**	58.0	56.0	10.0
fill_pen_holder	**15.3**	43.0	8.0	0.0	0.0	0.0
classify_objects	**96.7**	97.7	96.0	78.0	16.0	56.0
play_tic_tac_toe	83.9	95.0	94.0	96.0	**100.0**	0.0
fill_egg_holder	0.7	11.1	**6.0**	0.0	0.0	0.0
organize_table	**19.3**	69.0	4.0	0.0	2.0	0.0
make_kong	**27.3**	27.3	0.0	2.0	0.0	0.0
play_stacking_toy	36.0	64.7	**54.0**	18.0	0.0	0.0
Memory						
cover_blocks	83.3	84.6	**100.0**	**100.0**	84.0	46.0
match_and_pick_from_conveyor	**96.7**	96.7	86.0	74.0	60.0	62.0
swap_blocks	**96.7**	96.7	40.0	78.0	80.0	0.0
swap_T	91.3	91.3	92.0	**96.0**	88.0	18.0
press_by_number	95.9	96.0	0.0	**96.0**	88.0	70.0
imitate_sorting_sequence	**94.0**	95.3	92.0	42.0	60.0	36.0
Open						
align_blocks	90.7	90.7	76.0	78.0	**98.0**	50.0
general_pickup	**93.3**	93.3	86.0	82.0	84.0	84.0
stack_blocks_by_language	96.0	96.8	**100.0**	96.0	50.0	40.0
solve_equation	**87.3**	87.3	72.0	86.0	70.0	40.0
classify_objects_by_language	88.7	92.1	**92.0**	72.0	40.0	30.0
pick_from_conveyor_by_image	44.7	44.7	22.0	28.0	**52.0**	4.0
store_tools_in_toolbox	0.0	13.3	0.0	0.0	0.0	0.0
pour_by_language	4.0	4.9	**18.0**	0.0	0.0	0.0

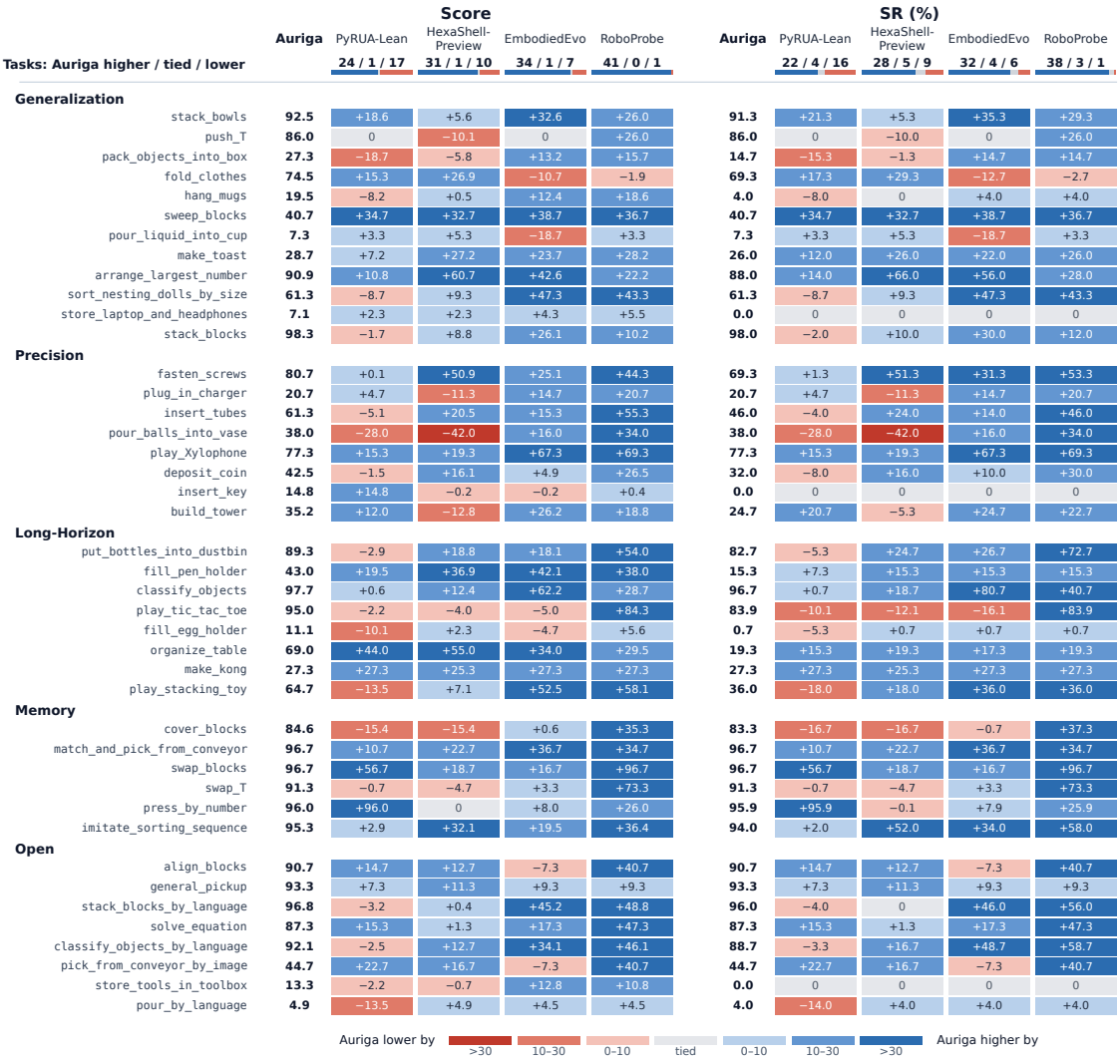


Figure 25 Auriga against PyRUA-Lean, HexaShell-Preview, EmbodiedEvo, and RoboProbe on each task, in Score (left) and SR in % (right). In each panel, the first column gives the value of Auriga and each remaining column gives the difference (Auriga minus that method): blue where Auriga is higher, red where it is lower, and grey where the two are tied, darker for larger differences. The top row counts the tasks on which Auriga is higher, tied, and lower, with a bar of their shares.